

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение
высшего образования
«УЛЬЯНОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ»

ФАКУЛЬТЕТ ИНФОРМАЦИОННЫХ СИСТЕМ И ТЕХНОЛОГИЙ

Кафедра «Информационные системы»

Дисциплина «Информационная безопасность»

Лабораторная работа № 2

Изучение криптографических алгоритмов.
Алгоритмы хэширования.

Выполнил:
студент гр. ПИБд-41
Белянин Н.Н.
Проверил:
Доцент, к.т.н.
Мытарев П. В.

Ульяновск, 2025г.

Лабораторная работа № 2

Изучение криптографических алгоритмов. Алгоритмы хэширования.

Содержание задания

1. Реализовать указанный преподавателем алгоритм хэширования.
2. Программа должна иметь интерфейс, в котором содержатся данные студента, вариант, справочная информация – алгоритм и его описание, возможности для сохранения и выбора обрабатываемого файла.
3. Отчет должен содержать вариант задания, блок-схему алгоритма, реализованного в программе, листинг программы с комментариями, расчет данных по одному циклу алгоритма.
4. Отчет и код программы с исполняемым кодом дополнительно представляются в электронном виде.
5. Обрабатываться должен файл любого формата(word, excel, графические форматы и т.д.), длиной не менее 1 кБ.
6. Для хэш-функций должны быть приведены результаты хэширования каких-либо значений, полученных в разрабатываемой программе, и результаты хэширования, рассчитанные с помощью общедоступных инструментов в сети «Интернет».
7. Ответить на вопросы преподавателя.

Вариант – 6

Хэш-функция MD5.

Ссылка на репозиторий: <https://github.com/Nikbeli/Information-Security>

Оглавление

Ход выполнения лабораторной работы.....	5
<i>Теоретическая часть выполнения</i>	5
Жизненный цикл атаки (Kill Chain)	9
Виды хэш-функций	10
<i>Практическая часть выполнения</i>	13
Применение MD5	14
Уязвимости MD5.....	14
Атаки MD-5.....	15
Как работает алгоритм?	18
Коллизии	21
Безопасное использование MD-5.....	24
Соль (salt) и её добавление к паролю	25
Декодирование кода MD5	27
Схемы алгоритма MD5.....	28
<i>Логическая схема выполнения</i>	28
<i>Схема шага алгоритма</i>	29
<i>Схема всего алгоритма MD5</i>	31
<i>Трассировка алгоритма MD5</i>	33
<i>Блок схема</i>	34
<i>Блок-схема раунда алгоритма MD5</i>	36
<i>Структурная схема</i>	37
Реализация алгоритма MD5.....	38
<i>Архитектура</i>	38
<i>Стек технологий</i>	39
Реализация на С.....	39
main.c	39
md5.h.....	43
md5.c	44
Реализация на C++	49
main.cpp	49
md5.h.....	52
md5.cpp	53
Реализация на Java	57

Main.java.....	57
MainWindow.java	57
MD5.java.....	60
Демонстрация работы программной системы	63
Результаты хэширования	71
Расчёт одного цикла алгоритма (Трассировка).....	76
Заключение.....	82
Приложение. Листинг кода	87

Ход выполнения лабораторной работы

Теоретическая часть выполнения

Что объединяет пароль пользователя, цифровую подпись, криптовалюту и контрольную сумму файла? Это хэширование!¹ Маленькая и казалось бы незначительная деталь лежит в основе этих технологий, без которых они бы не смогли существовать. Безопасность сегодня – это основополагающая вещь при взаимодействии с информационными технологиями.

Криптографические хэш-функции – фундаментальный инструмент информационной безопасности, где алгоритмы преобразуют некоторый массив входных данных произвольного размера (длины) в выходную битовую строку заданной длины (фиксированного размера) в соответствии с определённым алгоритмом. В результате мы получаем данные, которые как раз будут называться хэшем. Хэш-функции применяются:

- для контроля целостности;
- для построения цифровых (электронных) подписей (как хэш-образ сообщения);
- для аутентификации сообщений (в составе MAC);
- в индексации данных;
- при построении уникальных идентификаторов для наборов данных;
- для хранения паролей в системах защиты в виде хэш-кода;
- при построении ассоциативных массивов;
- при поиске дубликатов в последовательностях наборов данных;
- для вычисления контрольных сумм² от данных (сигнала) для последующего обнаружения в данных ошибок (возникших случайно или внесённых намеренно), возникающих при хранении и/или передаче данных.

¹**хэширование** – это преобразование входного массива данных произвольной длины в выходную битовую строку фиксированной длины.

²**Контрольная сумма** – некоторое значение, рассчитанное по набору данных путём применения определённого алгоритма и используемое для проверки целостности данных при их хранении или передаче.

Криптографические хэш-верификаторы ³считаются безопасными и используются для проверки целостности и подлинности. Таким образом, высокая *алгоритмическая сложность* и *однонаправленность* послужили основанием для легитимности электронных подписей и их аналогов в системах информационной безопасности. Существует 4 основных направления шифрования информации:

- Симметричное
- Ассиметричное
- Хэширование
- Беспроводное

Базовые требования к хэш-функции:

1. Хэш – всегда фиксированного размера вне зависимости от длины данных.
2. Все одинаковые входные данные выдают одинаковый хэш.
3. Все разные данные (отличающиеся на 1 байт, символ) должны выдавать абсолютно разный хэш.
4. Необратимое преобразование – невозможно узнать исходное сообщение, по которому он был сгенерирован.

Однако в случае с хэшированием мы имеем дело с бесконечностью, и эта бесконечность представляет собой поток входных данных, которые никак не ограничены в размерах и поэтому мы должны придумать их сколько угодно. А количество бит для хэша у нас фиксировано.

Пример: у нас 3 бита, значит хэш разместит $2^3=8$ уникальных значений. Как бы мы не сжимали, не перемешивали – всё равно периодически гарантировано будем получать одно из 8 значений (визуальное представление Рисунок 1).

³ **Хэш-верификаторы** — это инструменты, которые проверяют подлинность данных с помощью хэш-функций (хэширования)

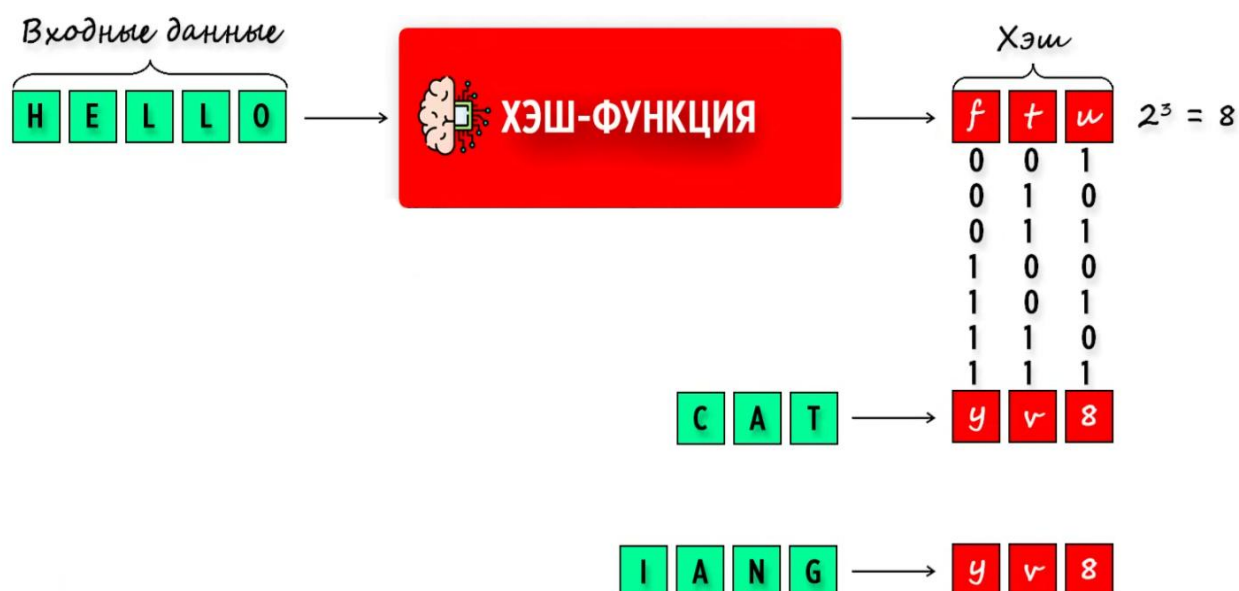


Рисунок 1. Работа хэш-функции.

Такое совпадение – есть **коллизия** (от лат. collisio — столкновение).

Коллизия (конфликт) в информационной безопасности — это ситуация, когда два различных набора данных дают одинаковый хэш (хэш-код или message digest). Это может привести к проблемам, особенно в системах, где хэш-коды используются для хранения данных, для безопасности (аутентификация). Коллизии могут снизить эффективность поиска и привести к ошибкам в работе системы.

Конечно, размер выходного хэша в реальных хэш-функциях намного больше: 32 бита, 64 бита, 128 бит, 256 бит и даже 512 бит, но даже на таких размерностях могут возникать коллизии. Изначально они могут возникать произвольно или случайно. Однако, если кто-то намерено пытается найти способ (алгоритм), который позволит ему генерировать эту самую коллизию для нужного ему хэша за какое-то приемлемое время, то такая хэш-функция будет считаться **взломанной** и появится **угроза конфиденциальности** и **целостности** информации, тем самым нарушится безопасность.

- **Угроза** – это потенциальная возможность определённым образом нарушить информационную безопасность.

- **Взлом (*hacking*)** – это реализация (метод) угрозы, а точнее это процесс поиска и использования уязвимостей в информационной системе, в её сетях, инфраструктуре, приложениях или устройствах с целью получения несанкционированного доступа, изменения, кражи или разрушения информации или ресурсов.

Виды взлома по мотивации:

1. *Криминальный (black-hat)* — кража данных, вымогательство, мошенничество, саботаж.
2. *Этический (white-hat)* — тестирование на проникновение с разрешения (pentest), поиск уязвимостей для их устранения.
3. *Серый (gray-hat)* — действия между законным и преступным: нашёл уязвимость и опубликовал её без согласия владельца.
4. *Государственный/разведывательный* — кибершпионаж, атака на инфраструктуру.

Типы атак:

- Социальная инженерия (фишинг⁴, подмена).
- Вредоносное ПО (трояны).
- Web-уязвимости (SQL-инъекции – это метод кибератаки, при котором злоумышленник внедряет вредоносный SQL-код в запросы к базе данных. Такая уязвимость веб-безопасности позволяет злоумышленнику вмешиваться в запросы, которые приложение делает к своей базе данных).
- Перебор паролей, брутфорс⁵ — метод взлома, при котором хакер подбирает разные варианты логинов, паролей и ключей шифрования. В криптографии на вычислительной сложности полного перебора основывается оценка криптостойкости шифров. Шифр считается криптостойким, если не

⁴ **Фишинг (fishing или «выуживание»)** – вид интернет-мошенничества или кибератаки, целью которой является получение доступа к конфиденциальным данным пользователя (логин и пароль). Осуществляется путём проведения массовой рассылки электронных писем от различных сервисов и от имени популярных брендов.

⁵ **Полный перебор (метод «грубой силы» или brute force)** – метод решения математических задач, который относится к классу методов поиска решения исчерпыванием всевозможных вариантов. Сложность полного перебора зависит от количества всех возможных решений задачи.

существует метода «взлома» существенно более быстрого чем полный перебор всех ключей. Криптографические ключи, основанные на методе полного перебора, являются самыми универсальными, но и очень долгими.

- Использование уязвимых конфигураций и сервисов
- Эксплуатация уязвимостей ПО/протоколов.
- MITM (man-in-the-middle) – вид атаки в криптографии и компьютерной безопасности, когда злоумышленник тайно ретранслирует и при необходимости изменяет связь между двумя сторонами, которые считают, что они непосредственно общаются друг с другом.

Жизненный цикл атаки (Kill Chain)

Жизненный цикл кибератаки — это процесс или модель, в соответствии с которой типичный злоумышленник продвигается по цепочке событий, чтобы успешно проникнуть в сеть организации и похитить из неё информацию, данные или коммерческую тайну. Это схема, описывающая этапы, которые проходит злоумышленник, чтобы осуществить успешную кибератаку. Понимание этого цикла помогает организациям выявлять и устранять угрозы на каждом этапе.

1. Разведка (reconnaissance). Сбор информации о цели: сети, IP, ПО, сервисах и персонале.

2. Сканирование / Создание вредоносного ПО. Поиск открытых портов и уязвимых сервисов, либо создание вирусной программы с последующей доставкой к нужной цели.

3. Эксплуатация. Использование найденной уязвимости для получения доступа.

4. Эскалация привилегий. Получение более высокого уровня прав и установка доступа к защищённым ресурсам и инфраструктуре.

5. Установка вредоносного ПО / Удержание доступа. Установка бекдоров ("задняя дверь") – уязвимость, которая даёт несанкционированный доступ к системе и последующее создание удалённого доступа. Цель –

скрытное и быстрое получение доступа к данным, зачастую зашифрованным и защищённым.

6. Управление и контроль. Подчинение программной системы

7. Движение по сети. Доступ к другим системам и данным.

8. Действия с целью (exfiltration / sabotage). Кража данных, шифрование, уничтожение.

9. Соккрытие следов. Очистка логов, маскировка активности

Сейчас никто не хранит пароль в открытом виде, потому что в случае, если базу данных сольют в сеть, то злоумышленникам будет достаточно ввести пароль и получить доступ к данным. В базе данных хранятся хэши паролей и, очевидно, их использование не даст никакого результата, поскольку введенные данные для пароля преобразуются в хэш, затем будут сравниваться с хэшем, который сохранён в системе. Ничего не произойдёт, нужно знать исходные данные, из которых этот хэш получился. Однако злоумышленник может разработать алгоритм, который сгенерирует ему другие входные данные, которые приведут к слитому хэшу, то есть образует коллизию.

Виды хэш-функций

Сегодня придумано и разработано большое количество хэш-функций (md5, sha-1, sha-256, crc-32), их так много, потому что каждая из них была когда-то неуязвима, но в дальнейшем их всех постигла участь быть взломанными. Нашлись такие алгоритмы, которые находили коллизии для этих алгоритмов за приемлемое время.

Хэш-функция устойчива к коллизиям, когда вероятность её вычислить настолько мала, что для этого потребуются миллионы лет.

Все основные используемые хэш-функции можно разбить на 3 больших семейства: семейство MD, семейство CRC, семейство SHA.

Каждая из этих групп содержит множество хэш-функций в хронологическом порядке. Они отличаются между собой разрядностью выходного хэша, криптостойкостью и скоростью вычисления. Чем выше разрядность и криптостойкость алгоритма, тем больше времени он будет выполняться. Выбор будет зависеть от конкретной задачи, однако некоторые из них вообще не являются криптостойкими, а, следовательно, они сильно подвержены коллизиям. Однако в случае с хэш-функциями для вычисления контрольной суммы, этого и не требуется.

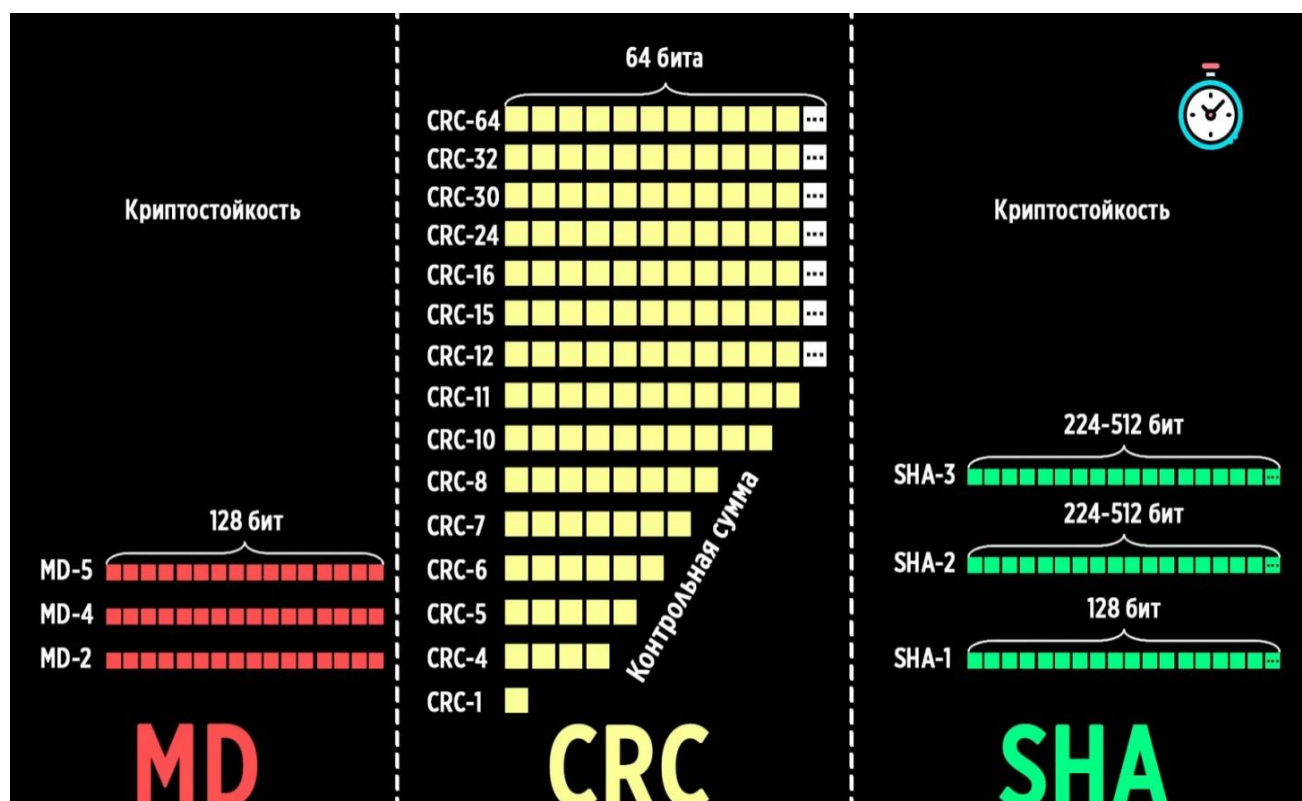


Рисунок 2. Семейства хэш-функций.

Контрольная сумма – это метод проверки целостности информации при передаче данных. Используется повсеместно, начиная от проверки целостности PNG-файла, заканчивая передачей данных по сети через протокол TCP/IP⁶. Дело в том, что ходе отправки пакета с данными, часть битов может исказиться. В результате каких-то внешних воздействий,

⁶ **TCP/IP** — это набор сетевых протоколов, который используется для передачи данных в интернете и других компьютерных сетях. Название расшифровывается как Transmission Control Protocol/Internet Protocol («Протокол управления передачей/Интернет-протокол»)

поэтому до их отправки на основе этих данных формируется контрольная сумма алгоритмом CRC32, которая записывается в конец передаваемой информации. Компьютер получателя, получив пакет, так же формирует контрольную сумму и сверяет полученные данные. Если контрольные суммы не совпадают, то данные не достоверны, и запрашиваем повторную передачу данных. Алгоритмы для вычисления контрольных сумм не предназначены для защиты от преднамеренного взлома, но они необходимы для защиты данных при случайных искажениях в ходе передачи данных.

1. Криптографические хэш-функции – используются в системах безопасности (шифрование, цифровые подписи, хранение паролей, генерация уникальных идентификаторов). Они должны обладать стойкостью к коллизиям, невозможностью восстановления исходных данных и равномерным распределением.

Алгоритмы:

- MD5 (128 бит), SHA-1 (160 бит) – на данный момент устаревшие, небезопасные и уязвимые к коллизиям.
- SHA-2 (SHA-224, SHA-256, SHA-384, SHA-512) – современный стандарт шифрования.
- BLAKE2 / BLAKE3 (до 512 бит) – быстрее SHA-2 и SHA-3, современный и безопасный.
- SHA-3 (Кессак) – современный стандарт NIST, устойчив к многим атакам. Система блокчейна⁷

2. Некриптографические хэш-функции – используются там, где безопасность не требуется, а важна скорость и равномерное распределение (хэш-таблицы, базы данных, индексация, контроль целостности без криптографической защиты).

⁷ **Блокчейн** – это технология хранения данных, в которой записи объединены в последовательную цепочку блоков, защищённую от подделки с помощью криптографических хэш-функций.

Алгоритмы:

- CRC32 – проверка целостности файлов (быстрый, легко предсказуем).
- FNV (Fowler-Noll-Vo) – хэширование строк (прост в реализации, равномерно распределён).

3. Специализированные хэш-функции – разработаны для узких задач и встроены в некоторые языки программирования с целью быстрой и удобной разработки программного обеспечения.

Алгоритмы:

- bcrypt – хэширование паролей (медленный с солью).
- scrypt – повышенная устойчивость к аппаратным атакам (GPU/ASIC)
- Argon2 – современный стандарт хэширования паролей.
- HMAC (Hash-based Message Authentication Code) – проверка целостности и подлинности сообщений.

Практическая часть выполнения

Переходим к выполнению практической части лабораторной работы с разбором работы алгоритма, особенностями, схемами отображения и реализацией алгоритма хэширования в соответствии с вариантом задания. По техническому заданию необходимо реализовать алгоритм хэширования (хэш-функцию) MD-5.

MD5 (Message Digest 5) – это 128 битный алгоритм хэширования и криптографическая хэш-функция, разработанная профессором Рональдом Л. Ривестом из Массачусетского технологического института (MIT) в 1991 году. Хэш-функция принимает сообщение любой длины и выдаёт 128-битный хэш (32 шестнадцатеричных символа или в случае с сообщениями, то пакета). Алгоритм предназначен для создания контрольных сумм или «отпечатков» сообщения произвольной длины и последующей проверки их подлинности.

Данная хэш-функция позволяет создавать уникальное хэш-значение для произвольного объёма данных. Основное назначение MD5 – это проверка целостности данных, чтобы убедиться, что файл не повреждён и не изменён.

Ранее алгоритм широко использовался для хэширования паролей в базу данных и создания цифровых подписей, однако сейчас из-за своей уязвимости и возможности коллизий, этот метод больше не рекомендуется, а предпочтительней стал более надёжный SHA-256. Алгоритм MD5 основан на алгоритме MD4 и является развитием идей предшественников (MD2, MD3, MD4), которые решали задачи безопасности.

Применение MD5

1. Создание контрольных сумм (Checksum). Контрольная сумма – короткий код, который позволяет проверить, что данные не изменились и не повредились. Она создаётся из файла перед его передачей или записью, а потом проверяется у получателя. Если контрольная сумма совпала, то всё в порядке.

2. Хэширование паролей. Раньше MD5 использовали для хранения паролей в виде хэшей, что помогало защитить данные, а в базе данных хранился пароль в виде «отпечатка», а не в чистом виде. Однако сейчас из-за уязвимостей, он потерял свою пользу и его заменяют на Argon2, SHA-256.

3. Построение ассоциативных массивов. MD5 применяется для быстрого поиска данных по ключу, что ускоряет работу программ, так как хэш служит уникальным идентификатором для каждой записи.

4. Сравнение данных и поиск дубликатов. Вместо анализа самих данных сравнивают их хэши, что помогает быстрее находить совпадения, особенно в больших наборах информации.

Уязвимости MD5

Алгоритм MD5 уязвим к некоторым атакам, к примеру, создание двух сообщений с одинаковой хэш-суммой. На данный момент существуют несколько видов взлома хэшей MD5:

- подбора сообщения с заданным хэшем;
- перебор по словарю;
- Brute-force
- RainbowCrack;
- Коллизия хэш-функции;

При этом методы перебора по словарю и brute-force могут использоваться для взлома хэша других хэш-функций (с небольшими изменениями алгоритма). RainbowCrack требует предварительной подготовки радужных таблиц, которые создаются для заранее определённой хэш-функции. Поиск коллизий специфичен для каждого алгоритма. Поскольку алгоритм очень быстро работает – это делает его уязвимым к атакам методом перебора.

Атаки MD-5

Криптоанализ алгоритма MD-5 предоставляет следующие виды хакерского «взлома» хэшей.

- **Атака полного перебора (исчерпывающий поиск ключей)** в криптографии – это стратегия, которая используется против зашифрованных данных и теоретически справляется с любыми шифрами. В случае, если злоумышленник не может найти уязвимость в системе шифрования, то он реализовывает атаку перебором. Она включает в себя систематическую проверку всех возможных ключей, пока не будет найден правильный. Для взлома надёжной системы с большей вероятностью придётся задействовать всю вычислительную мощность.

- **Перебор по словарю** – атака на систему защиты, применяющая метод полного перебора предполагаемых паролей, используемых для аутентификации. Она осуществляется путём последовательного перебора всех слов (паролей в чистом виде) определённого вида и длины из словаря с целью последующего взлома системы и получения доступа к секретной информации. Атаки по словарю являются атаками полного перебора, однако есть отличие в том, что данный взлом обычно более эффективен, так как становится не нужным перебирать все комбинации символов, чтобы добиться успеха. Злоумышленники используют обширные списки наиболее часто используемых в пароле (такие как: имена животных, персонажей), отсюда и название атаки. Если же пароль действительно уникален (не является комбинацией слов), атака по словарю не сработает. В таком случае остаётся использовать только атаку полным перебором.

- **Радужные таблицы** – это более эффективные тип атаки, состоящий из хэш-цепочек. Данный способ основан на генерировании большого набора хэшей из набора символов, чтобы по получившейся базе в поиске ввести заданный хэш. Радужные таблицы оптимизируют требования к хранению, хотя поиск выполняется немного медленнее. Они отличаются от хэш-таблиц тем, что они создаются с использованием, как хэш-функций, так и функций редукции.

Суть радужных таблиц – это вскрытие паролей, преобразованных при помощи сложнообратимой хэш, а также атака на симметричные шифры на основе известной открытого текста. Всё это осуществляется путём создания больших наборов данных, в которых хранятся различные распространённые слабые пароли и хэши, созданные из этих паролей.

Один из распространённых методов защиты от взлома с помощью радужных таблиц – использование необратимых хэш-функций, которые включают salt («соль» или «модификатор»)

$$\text{Хэш} = \text{MD5}(\text{пароль} + \text{соль})$$

Для восстановления такого пароля взломщику необходимы таблицы для всех возможных значений соли. При использовании такой схемы соль должна быть достаточно длинной (6–8 символов), иначе злоумышленник может вычислить таблицы для каждого значения соли, случайной и различной для каждого пароля. Таким образом, два одинаковых пароля будут иметь разные значения хэшей, если только не будет использоваться одинаковая соль. По сути, соль увеличивает длину пароля и отчасти увеличения сложности пароля. Если таблица рассчитана на некоторую длину или на некоторый ограниченный набор символов, то соль может предотвратить восстановление пароля. Примером, служат старые Unix-пароли, где размер соли составлял лишь 12 бит, для взлома таких паролей злоумышленнику нужно было бы посчитать всего 4096 таблиц, которые уместятся на современных жёстких дисках. В современных приложениях использую уже более длинную соль, так в специализированном алгоритме bcrypt используется соль размером в 128 бит и подобная длина делает предварительные вычисления бессмысленными и бесполезными.

- **Цепочки хэшей** – это метод для уменьшения требований к объёму памяти (уменьшение потребности в памяти). Главная идея заключается в определении функции редукции, которая сопоставляет значениям хэша значения из таблицы. Редукционная функция не является **инверсией хэш-функции**. Начиная с исходного пароля и поочерёдно применяя функции хэширования и редукции к каждому полученному значению, мы получим цепочку чередующихся паролей и хэшей.

Радужные таблицы являются развитием идеи таблицы хэш-цепочек. Функция редукции применяются по очереди, перемежаясь с функцией хэширования.

Использование последовательной функции редукции изменяет способ поиска по таблице, поскольку хэш может быть найден в любом месте цепочки, необходимо сгенерировать несколько различных цепочек.

Существует множество систем взлома паролей, использующие подобные таблицы. Основная идея – это достижение компромисса между временем поиска по таблице и занимаемой памятью. Радужные таблицы не дают гарантии взлома всей системы паролей, но чем больше набор символов, используемый для создания радужной таблицы, и чем продолжительнее хэш-цепочки, тем больше будет шансов получить доступ к базе данных исходных паролей.

Как работает алгоритм?

Метод хэширования MD5 описан в стандарте Интернета RFC 1321, во многом реализация похожа на предшественника MD4 и набивка идентична. На вход алгоритма поступает какой-то входной поток данных произвольного размера (сообщение, текстовый файл, изображение, символ, строка или что-то другое), хэш которого нужно найти. Длина сообщения измеряется в битах и может быть любой (в том числе нулевой). Записывается длина сообщения и это число, как правило, целое и неотрицательное. Кратность каким-либо числам необязательна. После поступления данных переходим к процессу подготовки потока к вычислениям.

Шаг 1. Выравнивание потока

Для обработки данных с помощью MD5 их длина должна быть кратной **512 бит (64 байта)**. Сначала к концу потока дописывается единичный бит. Затем добавляют некоторое число *нулевых бит* такое, чтобы новая длина потока стала сравнима с 448 по модулю 512. Нули дописываются до тех пор, пока не будет получен нужный размер для обработки. Выравнивание происходит в любом случае, даже если длина исходного потока, уже сравнима с 448

Шаг 2. Добавление длины сообщения

После выравнивания в конец сообщения дописывают 64-битное представление длины данных (количество бит в сообщении). Это позволяет алгоритму учитывать первоначальный объём информации, даже если данные были дополнены. Сначала записывают младшие 4 байта, затем старшие. Если длина превосходит $2^{64}-1$, то дописывают младшие биты (эквивалентно взятию по модулю 264) После этого длина массива слов по 512 бит. Вычисления будут основываться на представлении этого потока данных в виде массива слов по 512 бит.

Затем происходит разделение на блоки. Дополненные данные разбиваются на блоки по 512 бит (64 байта)

Шаг 3. Инициализация буфера

Для вычислений инициализируются четыре **буфера или регистра** (в коде называют переменными) размером по 32 бита, начальные значения которых задаются шестнадцатеричными числами. И затем MD5 использует 4 буфера: A, B, C, D. Они получают фиксированные стартовые значения и будут служить основой для дальнейших вычислений.

В этих переменных или **регистрах** будут храниться результаты промежуточных вычислений. Начальное состояние ABCD называется *инициализирующим вектором*.

A = 0x67452301 B = 0xEFCDAB89

C = 0x98BADCFE D = 0x10325476

Шаг 4. Вычисления в цикле.

Самый важный этап MD5. Здесь алгоритм выполняется в общей сложности за 4 раунда, где каждый блок данных обрабатывается через 64 операции, которые включают в себя битовые операции: И, ИЛИ, исключающее ИЛИ и НЕ. Эти операции выполняются с использованием

значений из блока и текущих значений A, B, C, D. Ниже представлены функции, применяемые к регистрам.

$$\begin{aligned}F(X, Y, Z) &= (X \wedge Y) \vee (\neg X \wedge Z) \\G(X, Y, Z) &= (X \wedge Y) \vee (Y \wedge \neg Z) \\H(X, Y, Z) &= X \oplus Y \oplus Z \\I(X, Y, Z) &= Y \oplus (X \vee \neg Z)\end{aligned}$$

Рисунок 3. Функции для алгоритма хэширования.

где $\oplus, \wedge, \vee, \neg$ побитовые логические операции XOR, AND, OR и NOT соответственно. После разбиения сообщения на блоки по 512 бит (64 байта), обрабатываемые в цикле. Каждый блок проходит этапы: регистры A, B, C, D обновляются с помощью математических операций (побитовых вычислений, перестановок и сдвигов). А затем используются заранее заданные константы, которые были получены из дробных частей синусов целых чисел. Определяется таблица констант от 1 до 64. 64-элементная таблица данных.

В каждом раунде выполняется по 16 операций. Мы применяем разные функции на каждом этапе, то есть последовательно применим F, G, H, I к каждому соответствующему этапу (к 1-му – F, ко 2-му – G и так далее).

Шаг 5. Результат вычислений.

После обработки всех блоков, получим значения, которые объединяются в 128-битный хэш. Результат вычислений находится в буфере регистров ABCD – это и есть хэш (уникальный «отпечаток»). Если выводить побайтово, начиная с младшего байта A и заканчивая старшим байтом D, то получим MD5-хэш (представление в виде 32-значного шестнадцатеричного числа).

Алгоритм MD5 происходит от MD4. В новый алгоритм добавили ещё один раунд, теперь их стало 4 вместо 3 в MD4. Добавили новую константу для того, чтобы свести к минимуму влияние входного сообщения, в каждом раунде на каждом шаге и каждый раз константа разная, она суммируется с результатом F и блоком данных. Изменилась функция

Результат каждого шага складывается с результатом предыдущего шага, из-за этого происходит более быстрое изменение результата. Для этой же цели оптимизирована величина сдвига на каждом круге. Изменился порядок работы с входными словами в раундах 2 и 3

Коллизии

Коллизия хэш-функции — это ситуация при которой происходит получение одинакового значения функции для разных сообщений и идентичного начального буфера. В отличие от коллизий, псевдоколлизии определяются как разные значения хэша для разных значений начального буфера, причём сами сообщения могут совпадать или отличаться. В MD5 вопрос коллизий не решается.

В 1996 году Ганс Доббертин нашёл псевдоколлизии в MD5, используя определённые **инициализирующие векторы**⁸, а не стандартные. Оказалось, что можно для известного сообщения построить второе такое, что оно будет иметь такой же хэш, как и исходное. MD5 был тщательно изучен криптографическим сообществом с момента его первоначального выпуска и до 2004 года демонстрировал лишь незначительные недостатки. Однако летом 2004 года криптографы Ван Сяюнь и Фэн Дэнго продемонстрировали алгоритм способный генерировать MD5-коллизии с использованием стандартного вектора инициализации.

⁸ **Инициализирующий вектор** (initialization vector, IV) в криптографии — это случайное или псевдослучайное значение, которое используется вместе с секретным ключом для инициализации процесса шифрования. Цель IV — это гарантировать, что идентичные открытые тексты, зашифрованные одним и тем же ключом, создают разные зашифрованные тексты.

Позже данный алгоритм был усовершенствован, как следствие время поиска пары сообщений значительно уменьшилось, что позволило находить коллизии с приемлемой вычислительной сложностью. Как оказалось, в MD5 вопрос коллизий не решается. Можно создать два 512-битных входных блока и изменить определенные биты внутри этих блоков, чтобы создать два слегка отличающихся сообщения с одинаковым хэш-значением. Время создания пары сообщений MD5 составляло в среднем 1 час.

Коллизии существуют для большинства хэш-функций, но для криптографически стойких хэш-функций частота их возникновения минимальна. Если множество различных входных данных конечно, можно задать *инъективную хэш-функцию*⁹, по определению не имеющую коллизий. Однако для хэш-функций, принимающих вход переменной длины и возвращающих хэш постоянной длины (таких как MD5), коллизии обязаны существовать, поскольку хотя бы для одного значения хэш-функции соответствующее ему множество входных данных будет бесконечно. Любые два набора данных из этого множества образуют коллизию.

Мерой криптостойкости хэш-функции является вычислительная сложность нахождения коллизии.

В идеале не должно существовать способа отыскания коллизий более быстрого, чем полный перебор.

Если для некоторой хэш-функции находится способ получения коллизий существенно более быстрый, чем полный перебор, то эта хэш-функция перестаёт считаться криптостойкой и использоваться для передачи и хранения секретной информации.

Вычислительная сложность и математическое обоснование коллизий

⁹ **Инъективная хэш-функция** (идеальная хэш-функция) — это функция, которая отображает каждый ключ из набора S во множество целых чисел без коллизий. В математике такое преобразование называется инъективным отображением.

Коллизия – это ситуация, когда $M_1 \neq M_2$ дают одинаковый хэш:

Формула коллизии: $MD5(M_1) = MD5(M_2)$

Для MD5 выход имеет длину 128 бит, то есть возможных хэшей всего 2^{128} . Если хэш-функция идеальна, то вероятность того, что два случайных сообщения совпадут по хэшу.

$$p = \frac{1}{2^{128}}$$

Однако для нахождения хотя бы одной коллизии не нужно проверять все 2^{128} комбинаций, поскольку работает *парадокс дней рождения*. Вероятность того, что среди k случайных хэшей есть хотя бы одна коллизия:

$$P(k) \approx 1 - e^{-k^2/(2N)}$$

Где $N = 2^{128}$ – количество возможных хэшей и для того, чтобы вероятность найти коллизию составляла примерно 50% нужно:

$$k \approx \sqrt{2N \ln(2)} \approx 1.177\sqrt{N} - \text{а теперь подставим для MD5 (128 бит)}$$

$$k \approx 1.177 \times \sqrt{2^{128}} = 1.177 \times 2^{64} \approx 2^{64}...$$

Таким образом, теоретическая вычислительная сложность нахождения коллизии для MD5 равно: $O(2^{64})$

На практике MD5 не является идеальной функцией и в ней много структурных уязвимостей. Современные атаки используют особенности внутренних преобразований (разность в состоянии блоков). По оценке Стивенса, сложность поиска коллизии в одном 512-битном блоке по его алгоритму оценивается в $2^{49,81}$. А сейчас есть коллизии MD5, найденные за несколько секунд на обычном ПК с GPU. Они не требуют перебора всего, потому что используется анализ слабых мест функций.

Безопасное использование MD-5

Сейчас уже существует много алгоритмов хэширования, однако MD5 не перестаёт быть одним из распространённых способов защиты информации в сфере различных прикладных исследований и в области разработки веб-приложений. В любом случае хэш необходимо обезопасить от всевозможных хакерских атак.

Информационная энтропия

$$H(x) = - \sum_{i=1}^n p(i) \log_2 p(i)$$

Рисунок 4. Формула информационной энтропии.

Информационная энтропия (энтропия Шеннона) — это мера неопределённости или случайности данных, поступающих из определённого источника. Количественно определяет среднее количество информации, содержащееся в сообщении или наборе данных. Энтропия дискретного источника равна среднему количеству информации, приходящейся на один символ (сообщение, элемент) источника и характеризует непредсказуемость появления какого-либо символа алфавита.

Выбор основания логарифма определяет единицу измерения информации. Для информационных систем, основанных на двоичной системе счисления, единицей информации является двоичный символ — бит и основание логарифма $r = 2$. В этом случае энтропия называется двоичной энтропией.

Надёжность и сложность пароля в сфере информационных технологий обычно измеряют в информационной энтропии (бит, нат, хартли) и чем выше она, тем надёжнее пароль и, следовательно, труднее её взломать.

Чем длиннее пароль и больше набор символов, из которого он получен, тем он надёжнее. При увеличении длины пароля на 1 бит количество возможных паролей удвоится, что сделает задачу атакующего в 2 раза сложнее. В среднем, получается так, что атакующий должен проверить половину из всех возможных паролей до того как найдёт правильный.

Считается наилучшей практикой, когда программная система настаивает на том, чтобы пользователь использовал надёжный пароль в процессе регистрации. Такое предварительное требование делает криптографический алгоритм надёжнее.

Соль (salt) и её добавление к паролю

Соль – это случайная последовательность байтов, которая добавляется к входному значению (обычно паролю) перед хэшированием, чтобы:

- предотвратить использование предвычисленных таблиц (**rainbow tables** – это специальный вариант таблиц поиска для обращения криптографических хэш-функций).
- сделать одинаковые пароли у разных пользователей различными по хэшу (что ещё вдобавок предотвратит коллизии, добавит разнообразия и безопасности данным).
- увеличить стоимость атак перебором (brute-force) при наличии уникальной соли для каждого хэша.

Соль – это вторичный фрагмент информации, состоящий из строки символов, которые добавляются к открытому тексту (исходному паролю пользователя), а затем хэшируется. Соление делает пароли более устойчивыми к атакам типа радужных таблиц, так как подобный пароль будет содержать более высокую информационную энтропию. Следовательно, меньше вероятность, что в предварительно вычисленной радужной таблице будет существовать наше значение.

Алгоритм MD5 сам по себе не содержит механизма соли и её мы добавляем сами, сверху. К примеру, мы можем захешировать MD5(salt || password) или MD5(password || salt).

Рекомендации:

1. Рекомендуется задавать длину соли ≥ 64 бит, а лучше 128 бит, 256 бит или 512 бит.
2. Предположительно энтропия соли равна количеству её случайных бит (128-битная соль = 128 бит энтропии).
3. Соль должна быть криптографически случайной, уникальной для каждого хэша.

Варианты использования соли:

1. Использование единой соли для всех паролей (соль достаточно длинная и сложная и стандартная радужная таблица не сможет справиться с расшифровкой солёных хэшей). Однако 2 одинаковых пароля будут иметь один и то же хэш.
2. Использование различной (случайной) соли для каждого пароля и хранение соли в самой базе данных. Если мы используем различные соли для каждого пароля, то 2 одинаковых пароля будут иметь разные хэши. Хакеру придётся генерировать различные радужные таблицы для каждого отдельного пользователя, что значительно увеличивает вычислительную сложность для взлома. Соли могут храниться в открытом виде в базе данных, однако лучше и её хэшировать. Цель соли не в её неизвестности, а в случайности, чтобы препятствовать использованию радужных таблиц.
3. Использование существующего значения столбца, то есть существующее значение столбца (пример: email, username) можно использовать в качестве соли. Решение аналогично второму и оно препятствует использованию стандартной радужной таблице, но минус в том, что хакер будет создавать таблицу для определённого имени пользователя, что повысит уязвимость.

Энтропия и оценка устойчивости

- Если пароль имеет энтропию H бит, то средняя стоимость перебора $\approx 2^{(H-1)}$ проверок.
- Соль не увеличит энтропию пароля, она лишь превращает одну цель атаки (один хэш) в множество независимых целевых хэшей (чтобы атаковать N хэшей с солью, требуется затратить N стоимость вычисления 1ого хэша).
- Для оценки устойчивости нужны следующие метрики: длина пароля, набор символов (алфавит) и вычисленная энтропия ($H \approx \text{length} \times \log_2(|N|)$)

Влияние соли на безопасность MD5

- Соли защищает от табличных атак. Но не снимает фундаментальных криптографических проблем MD5 (коллизии и другие уязвимости).
- Однако для хранения паролей всё равно лучше использовать специализированные алгоритмы с солью и «замедлением» (bcrypt, scrypt, Argon2).

Одна из наиболее распространённых причин успешных хакерских атак на базы данных заключается в том, что разработчики не используют добавление соли к исходному паролю. Безусловно, это сильно облегчает работу хакерам и взлом системы осуществляется с помощью атак типа радужных таблиц. Вдобавок, к этому пользователи зачастую используют очень распространённые простые пароли, которые имеют одинаковый хэш.

Декодирование кода MD5

При работе с информационными системами или в частности с повреждёнными базами данных требуется декодировать зашифрованные сообщения, с помощью значения хэша. Можно использовать специализированные онлайн ресурсы:

- md5.web-max.ca — данный сервис предназначен для получения декодированного значения нужно ввести хэш и заполнить поле проверочной капчи.

- md5decrypter.com -аналогичный сервис.
- msurf.ru – данный ресурс позволяет не только расшифровывать значения хэш-кодов, но и создавать их.

Однако, анализируя значения декодинга, процесс расшифровки почти не даёт результатов. Эти ресурсы – это либо объединённые между собой базы данных с занесёнными расшифровками, либо попытки поиска коллизии, а это не то что нам нужно. Если хэши каких-то сообщений состоят из более сложных и длинных комбинаций символов, то это невозможно будет расшифровать.

Создание хэша MD5 является односторонним процессом. Поэтому не подразумевает обратного декодирования первоначального значения.

Схемы алгоритма MD5

Логическая схема выполнения

Алгоритм получает на входе сообщение произвольной длины и создает в качестве выхода дайджест сообщения длиной 128 бит. Алгоритм состоит из шагов и имеет следующую логическую схему:

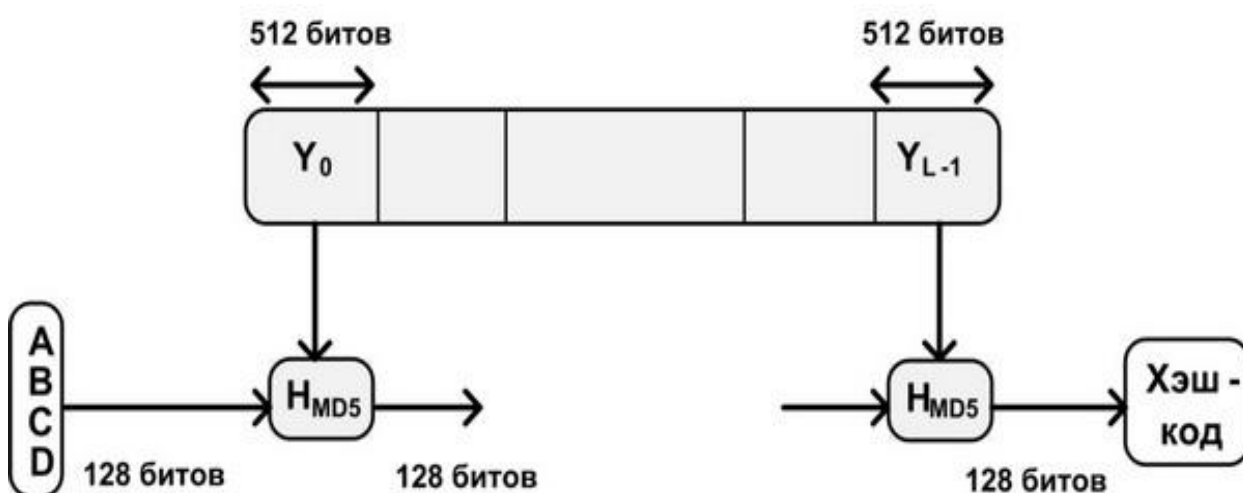


Рисунок 5. Логическое отображение алгоритма.

Схема отображает итеративный характер алгоритма MD5 – результат обработки каждого блока зависит от предыдущего.

Схема шага алгоритма

Рассмотрим схему выполнения процесса вычисления.

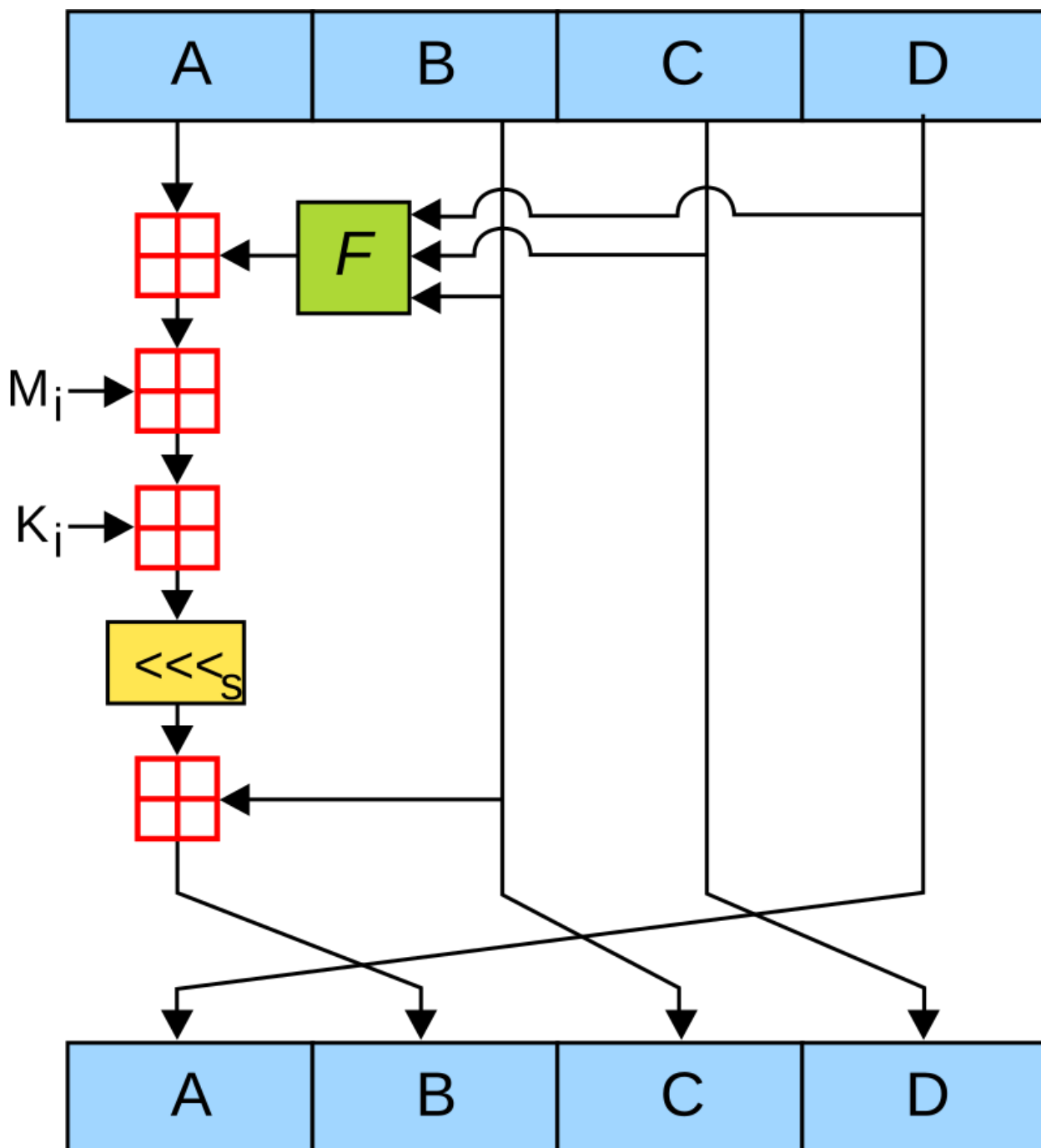


Рисунок 6. Схема процесса (одного шага (step) алгоритма).

Алгоритм MD5 – итеративная хэш-функция, основанная на обработке данных блоками по 512 бит и для каждого блока 64 шага, сгруппированные в 4 раунда по 16 операций. Результатом обработки одного блока является обновлённое состояние (A, B, C, D) – четыре 32-битных регистра.

Описание операции процесса:

Для генерации дайджеста сообщения используется буфер из четырех слов (A, B, C, D). Здесь каждый из A, B, C, D является 32-битным буфером для использования. Эти слова буфера инициализируются следующими значениями в шестнадцатеричном формате следующим образом: сначала мы определим 4 вспомогательные функции, которые используют в буфере, каждая из которых принимает на вход три 32-битных слова и выдает на выходе одно 32-битное слово.

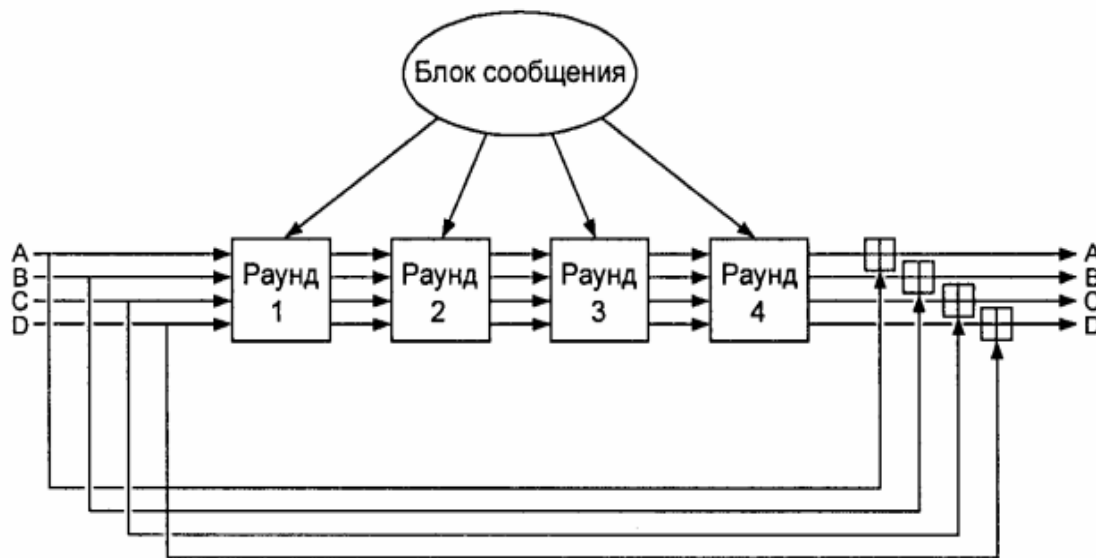
Использование 4 буферов (A, B, C и D) теперь комбинируется со словами ввода с помощью четырех вспомогательных функций (F, G, H и I). Здесь проводится четыре раунда, каждый из которых включает в себя 16 основных операций для выполнения. На рисунке показано, как вспомогательная функция "F" используется для четырех буферов (A, B, C и D) с помощью слова "M(i)" и константы "K(i)" и элемент " $\ll n$ " $M[i]$ — это 32-битное сообщение, $K[i]$ — это 32-битная константа, $\ll n$ — это сдвиг влево на n бит. Входные данные для инициализации буфера.

1. Берётся одно 32-битное слово $M[k]$ из блока сообщения ($512 \text{ бит} = 16 \text{ слов} \times 32 \text{ бита}$) и берётся одна из 64 predetermined констант $T[i]$, то есть $K[i]$.
2. Применяется логическая функция F, зависящая от раунда.
3. Добавляем битовое сообщение $M[i]$ к результату и затем добавляем константу $K[i]$ к результату.
4. Результат проходит циклический сдвиг ($\ll n$), где n может быть любым числом.
5. Обновляются регистры A, B, C, D

После того, как мы выполним все раунды, буферы A, B, C и D содержат MD5 дайджест исходного входа. В основном, MD5 состоит из пяти

шагов с четырьмя раундами вычислений, которые вычисляют хэш входного значения и выдают буфер на выходе.

Схема всего алгоритма MD5



Главный цикл обработки алгоритма MD5

A = 0x01234567
B = 0x89abcdef
C = 0xfedcba98
D = 0x76543210

$$F(X,Y,Z) = (X \wedge Y) \vee ((\neg X) \wedge Z)$$

$$G(X,Y,Z) = (X \wedge Z) \vee (Y \wedge (\neg Z))$$

$$H(X,Y,Z) = X \oplus Y \oplus Z$$

$$I(X,Y,Z) = Y \oplus (X \vee (\neg Z))^1$$

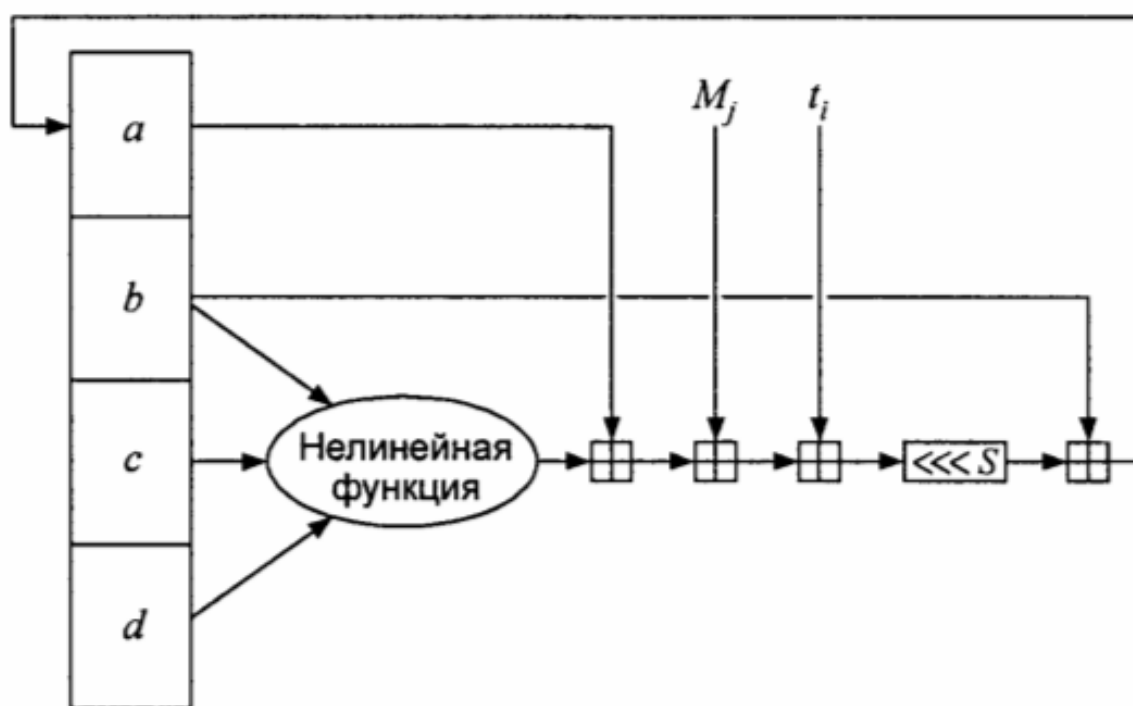
$$F(X,Y,Z) = ((Y \oplus Z) \wedge X) \oplus Z$$

$$G(X,Y,Z) = ((X \oplus Y) \wedge Z) \oplus Y$$

Рисунок 7. Схема алгоритма с функциями.

На представленной схеме рисунка 7 показана структура основного цикла обработки алгоритма MD5, предназначенного для вычисления хэш-суммы фиксированной длины (128 бит) из произвольного сообщения. На вход подаётся блок сообщения размером 512 бит, который разбивается и последовательно обрабатывается в четырёх раундах, каждый из которых использует одну из четырёх нелинейных логических функций — F, G, H, I. Эти функции выполняют различные комбинации логических операций (И, ИЛИ, НЕ, XOR) над переменными A, B, C, D, которые инициализируются заданными константами. Каждый раунд модифицирует состояние регистра, выполняя операции сложения по модулю 2^{32} , циклические сдвиги и

использование констант, зависящих от номера шага. После завершения всех четырёх раундов полученные значения А, В, С и D суммируются с исходными и формируют промежуточный хэш. После обработки всех блоков сообщение сворачивается в окончательное 128-битное значение — MD5-хэш, уникально характеризующее исходные данные.



Одна операция алгоритма MD5

$FF(a,b,c,d,M_j,s,t_i)$ означает $a = b + ((a + F(b,c,d) + M_j + t_i) \lll s)$

$GG(a,b,c,d,M_j,s,t_i)$ означает $a = b + ((a + G(b,c,d) + M_j + t_i) \lll s)$

$HH(a,b,c,d,M_j,s,t_i)$ означает $a = b + ((a + H(b,c,d) + M_j + t_i) \lll s)$

$II(a,b,c,d,M_j,s,t_i)$ означает $a = b + ((a + I(b,c,d) + M_j + t_i) \lll s)$

Рисунок 8. Схема одной операции алгоритма.

Ниже будет представлена трассировка алгоритма MD5, состоящая из всех 4 раундов, каждый из которых включает 16 операций. С применением функции F, G, H, I для каждого соответствующего раунда. А также блок-схема, которая содержит общее отображение и более подробное.

Трассировка алгоритма MD5

Раунд 1:

$FF(a, b, c, d, M_0, 7, 0xd76aa478)$
 $FF(d, a, b, c, M_1, 12, 0xe8c7b756)$
 $FF(c, d, a, b, M_2, 17, 0x242070db)$
 $FF(b, c, d, a, M_3, 22, 0xc1bdcee)$
 $FF(a, b, c, d, M_4, 7, 0xf57c0faf)$
 $FF(d, a, b, c, M_5, 12, 0x4787c62a)$
 $FF(c, d, a, b, M_6, 17, 0xa8304613)$
 $FF(b, c, d, a, M_7, 22, 0xfd469501)$
 $FF(a, b, c, d, M_8, 7, 0x698098d8)$
 $FF(d, a, b, c, M_9, 12, 0x8b44f7af)$
 $FF(c, d, a, b, M_{10}, 17, 0xffff5bb1)$
 $FF(b, c, d, a, M_{11}, 22, 0x895cd7be)$
 $FF(a, b, c, d, M_{12}, 7, 0x6b901122)$
 $FF(d, a, b, c, M_{13}, 12, 0xfd987193)$
 $FF(c, d, a, b, M_{14}, 17, 0xa679438e)$
 $FF(b, c, d, a, M_{15}, 22, 0x49b40821)$

Раунд 2:

$GG(a, b, c, d, M_1, 5, 0xf61e2562)$
 $GG(d, a, b, c, M_6, 9, 0xc040b340)$
 $GG(c, d, a, b, M_{11}, 14, 0x265e5a51)$
 $GG(b, c, d, a, M_0, 20, 0xe9b6c7aa)$
 $GG(a, b, c, d, M_5, 5, 0xd62f105d)$
 $GG(d, a, b, c, M_{10}, 9, 0x02441453)$
 $GG(c, d, a, b, M_{15}, 14, 0xd8a1e681)$
 $GG(b, c, d, a, M_4, 20, 0xe7d3fbc8)$
 $GG(a, b, c, d, M_9, 5, 0x21e1cde6)$
 $GG(d, a, b, c, M_{14}, 9, 0xc33707d6)$
 $GG(c, d, a, b, M_3, 14, 0xf4d50d87)$
 $GG(b, c, d, a, M_8, 20, 0x455a14ed)$
 $GG(a, b, c, d, M_{13}, 5, 0xa9e3e905)$
 $GG(d, a, b, c, M_2, 9, 0xfcfa3f8)$
 $GG(c, d, a, b, M_7, 14, 0x676f02d9)$
 $GG(b, c, d, a, M_{12}, 20, 0x8d2a4c8a)$

Раунд 3:

$HH(a, b, c, d, M_5, 4, 0xffffa3942)$
 $HH(d, a, b, c, M_8, 11, 0x8771f681)$
 $HH(c, d, a, b, M_{11}, 16, 0x6d9d6122)$
 $HH(b, c, d, a, M_{14}, 23, 0xfde5380c)$
 $HH(a, b, c, d, M_1, 4, 0xa4beea44)$
 $HH(d, a, b, c, M_4, 11, 0x4bdecfa9)$
 $HH(c, d, a, b, M_7, 16, 0xf6bb4b60)$
 $HH(b, c, d, a, M_{10}, 23, 0xbebfb7c0)$
 $HH(a, b, c, d, M_{13}, 4, 0x289b7ec6)$
 $HH(d, a, b, c, M_0, 11, 0xea127fa)$
 $HH(c, d, a, b, M_3, 16, 0xd4ef3085)$
 $HH(b, c, d, a, M_6, 23, 0x04881d05)$
 $HH(a, b, c, d, M_9, 4, 0xa9d4d039)$
 $HH(d, a, b, c, M_{12}, 11, 0xe6db99e5)$
 $HH(c, d, a, b, M_{15}, 16, 0x1fa27cf8)$
 $HH(b, c, d, a, M_2, 23, 0xc4ac5665)$

Раунд 4:

$II(a, b, c, d, M_0, 6, 0xf4292244)$
 $II(d, a, b, c, M_7, 10, 0x432aff97)$
 $II(c, d, a, b, M_{14}, 15, 0xab9423a7)$
 $II(b, c, d, a, M_5, 21, 0xfc93a039)$
 $II(a, b, c, d, M_{12}, 6, 0x655b59c3)$
 $II(d, a, b, c, M_3, 10, 0x8f0ccc92)$
 $II(c, d, a, b, M_{10}, 15, 0xffeff47d)$
 $II(b, c, d, a, M_1, 21, 0x85845dd1)$
 $II(a, b, c, d, M_8, 6, 0x6fa87e4f)$
 $II(d, a, b, c, M_{15}, 10, 0xfe2ce6e0)$
 $II(c, d, a, b, M_6, 15, 0xa3014314)$
 $II(b, c, d, a, M_{13}, 21, 0x4e0811a1)$
 $II(a, b, c, d, M_4, 6, 0xf7537e82)$
 $II(d, a, b, c, M_{11}, 10, 0xbd3af235)$
 $II(c, d, a, b, M_2, 15, 0x2ad7d2bb)$
 $II(b, c, d, a, M_9, 21, 0xeb86d391)$

Рисунок 9. Трассировка раундов алгоритма.

Блок схема

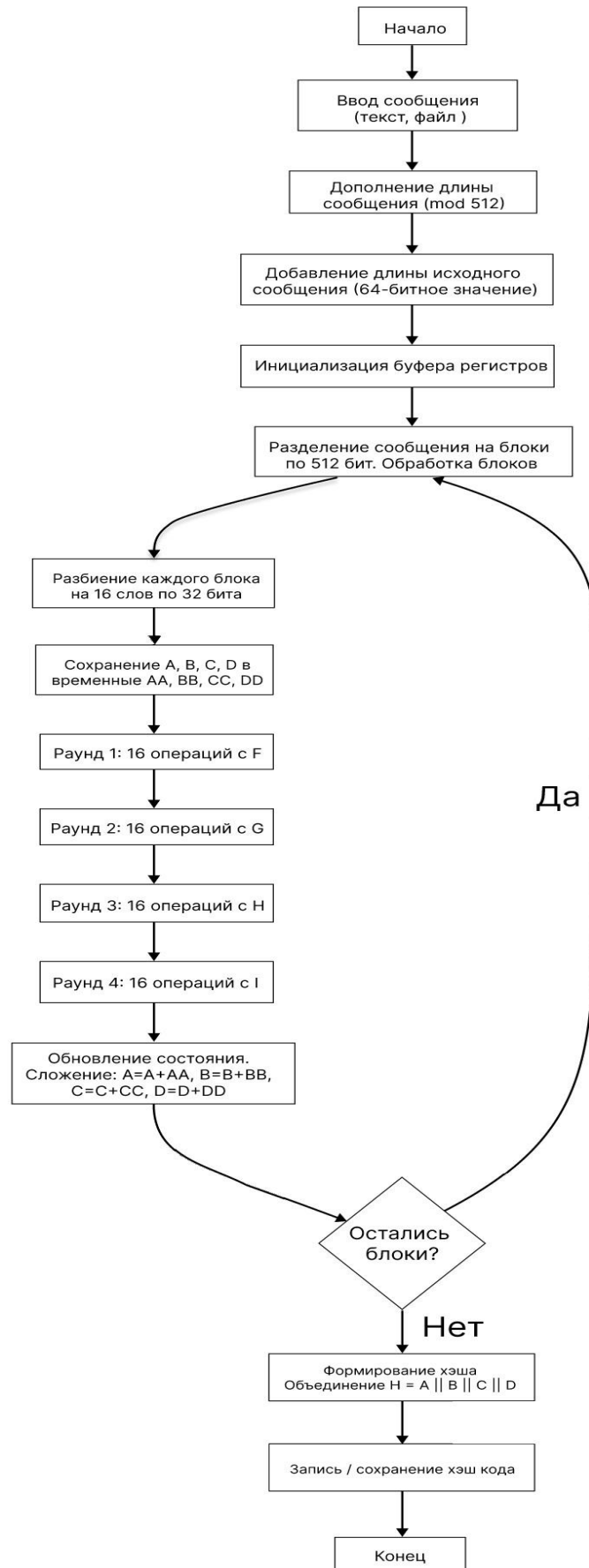


Рисунок 10. Блок-схема.

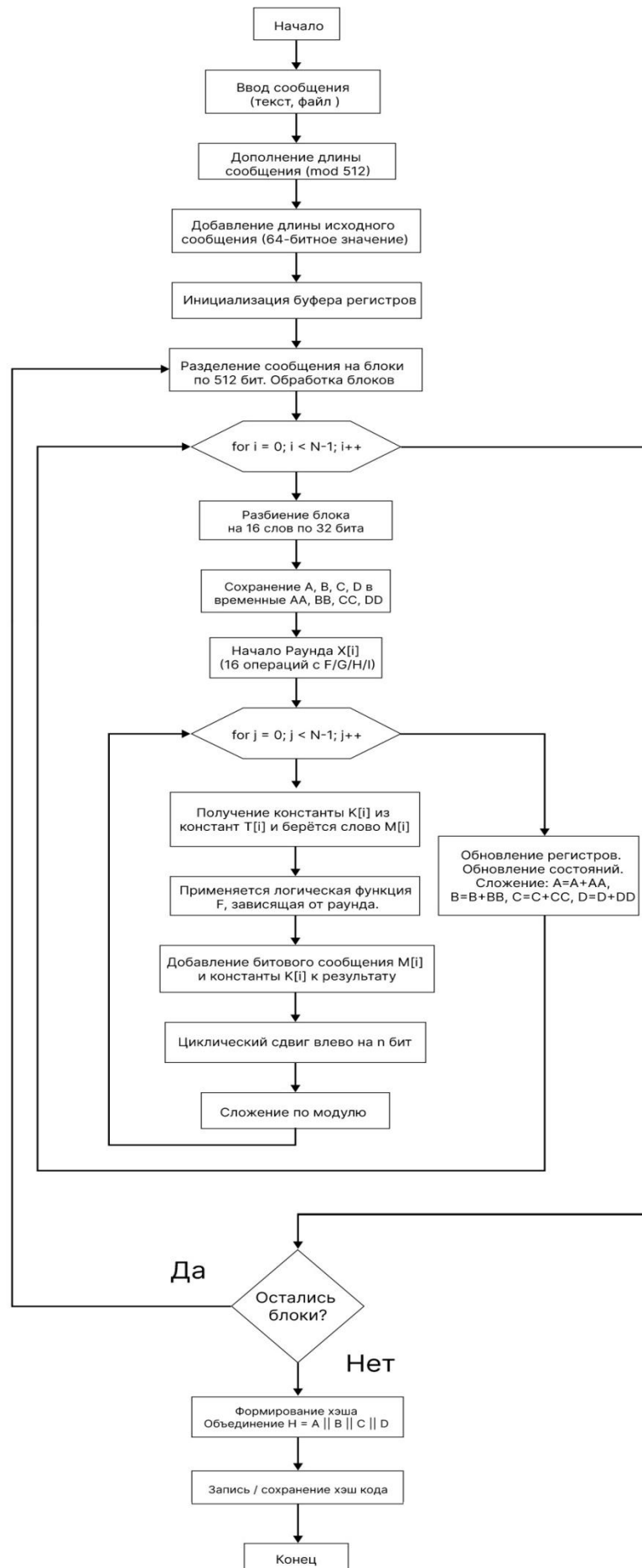


Рисунок 11. Блок-схема более подробно на циклах.

Блок-схема раунда алгоритма MD5

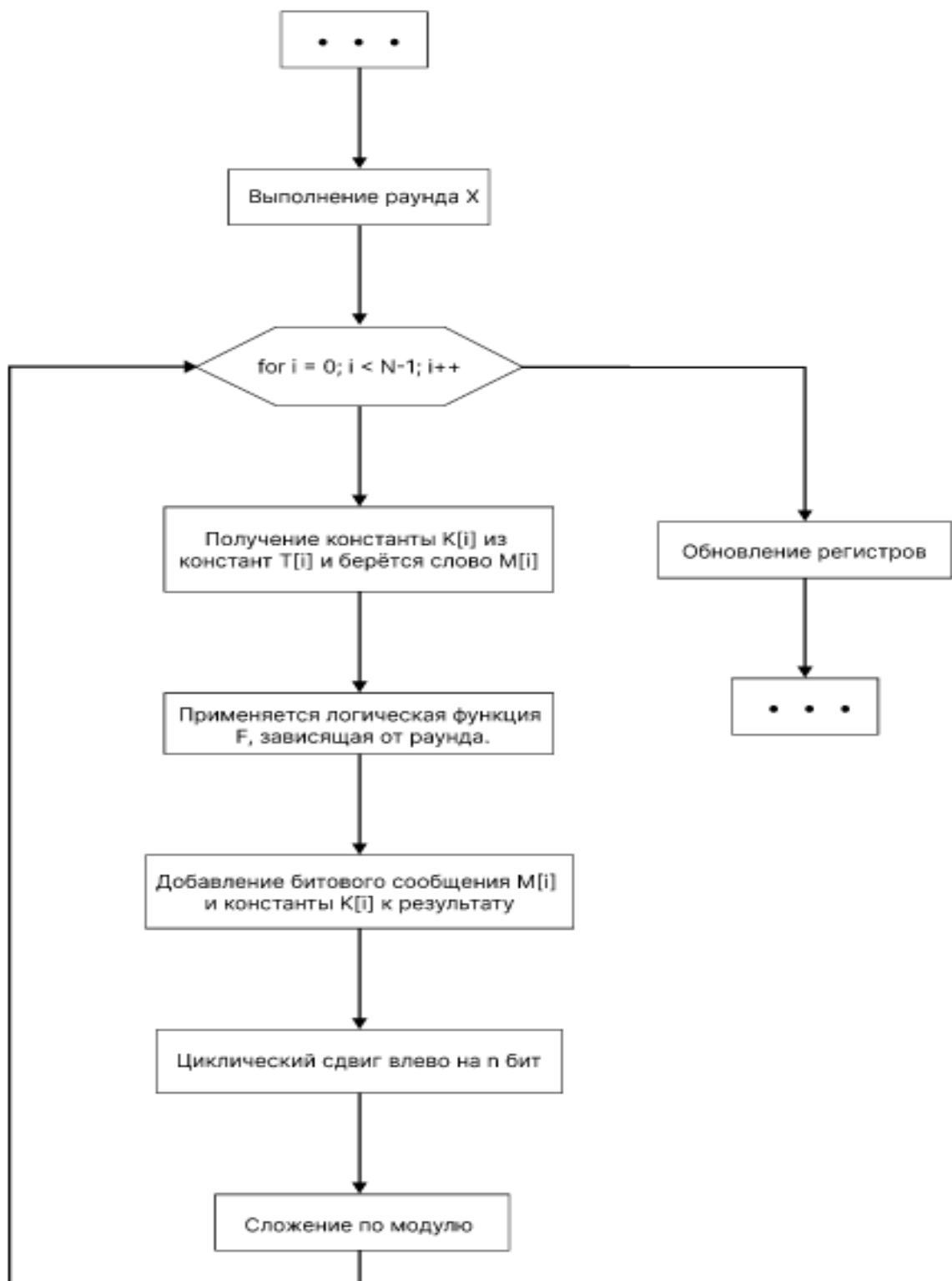


Рисунок 11. Блок-схема цикла раунда.

Первая блок-схема содержит действия и в конце базируется на условии, которое проверяет полную обработку всех блоков алгоритма. Вторая блок-схема визуализирует выполнение алгоритма на циклах. Циклы по блокам и

по раундам с подробной обработкой, а затем с проверкой обработки всех блоков. Третья блок-схема визуализирует обработку цикла раунда и выполнение действий.

Структурная схема

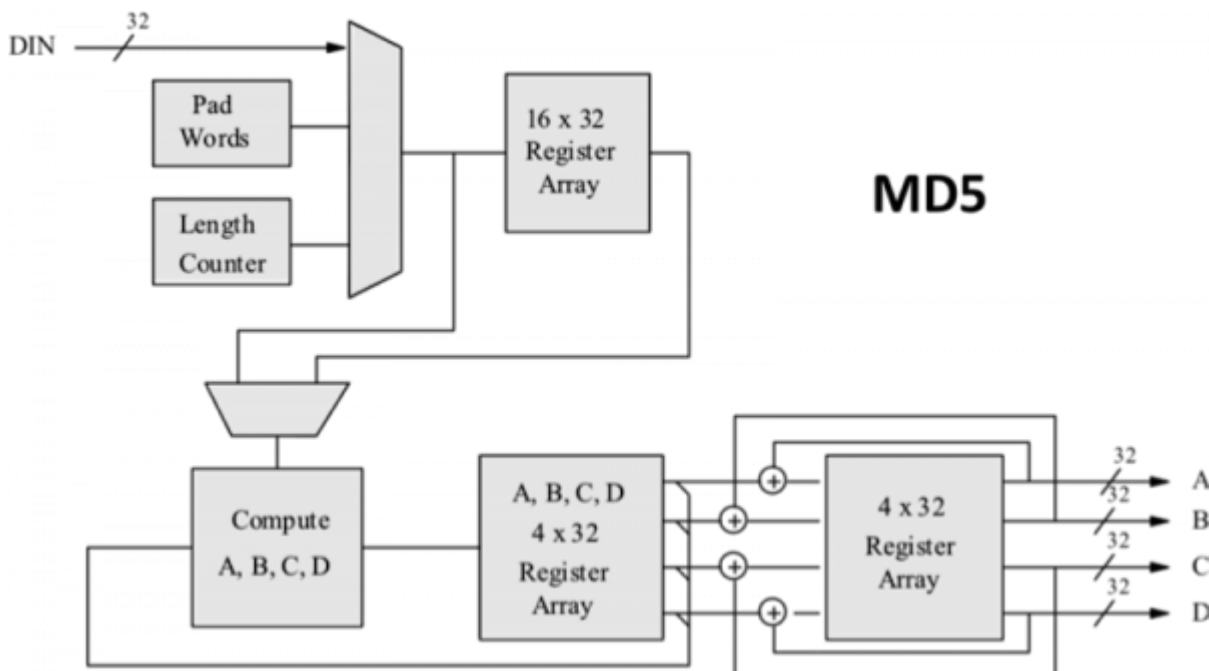


Рисунок 12. Схема структуры.

На схеме показаны блоки, которые представляют собой этапы или компоненты алгоритма хэширования. Эта схема отражает общую структуру алгоритма MD5, но не показывает все детали внутренних операций. Она фокусируется на потоке данных и основных компонентах.

На основе этой схемы алгоритм работает следующим образом:

1. **Подготовка сообщения (Padding).** Входное сообщение сначала дополняется ("Pad Words"), чтобы его длина стала кратной 512 битам минус 64 бита. Затем добавляется 64-битное представление исходной длины сообщения ("Length Counter"). Это гарантирует, что длина сообщения всегда будет кратна 512 битам.
2. **Обработка блоков.** Подготовленное сообщение разбивается на 512-битные блоки ("16 x 32 Register Array").

3. **Основной цикл вычисления.** Каждый 512-битный блок обрабатывается в 4 раундах по 16 операций в каждом. Основная функция "Compute A, B, C, D" выполняет эти операции. Она использует четыре 32-битных регистра (A, B, C, D), которые инициализируются фиксированными значениями. На каждом шаге регистры обновляются с помощью побитовых операций (AND, OR, XOR, NOT) и сложения по модулю 2^{32} .

4. **Формирование результата.** После обработки всех блоков содержимое регистров A, B, C, D конкатенируется, образуя окончательный 128-битный хэш ("4 x 32 Register Array").

Реализация алгоритма MD5

В ходе работы над лабораторной работой был реализован криптографический алгоритм MD5 (Message Digest 5) — это стандартный алгоритм хэширования, используемый для получения 128-битного (16-байтного) дайджеста сообщений произвольной длины. Задача реализации заключалась не только в вычислении хэша, но и в разработке программы с интерфейсом, позволяющим студенту вводить данные, выбирать файлы для обработки и сохранять результаты.

Архитектура

Для реализации выбрана архитектура двухмодульной системы, состоящая из таких модулей:

- ***Логический уровень (Logic Core)***, который содержит реализацию алгоритма MD5, включая все стадии: дополнение сообщения до кратности 512 бит, добавление длины исходного сообщения, инициализация регистров A, B, C, D, выполнение 4 раундов по 16 итераций и формирование финального хэша.

- ***Интерфейсный уровень (UI Layer)***, который отвечает за взаимодействие с пользователем, ввод данных студента, выбор входного файла,

отображение справочной информации об алгоритме MD5 и вывод вычисленного хэша. Реализуется в десктоп форме. А также дополнен уровнем данных (I/O Layer), который реализует операции чтения и записи файлов, сохранение результатов хэширования и отчётных данных. Данный уровень обеспечивает возможность обработки файлов любого формата. Проверено на word/pdf/jpg форматах.

Стек технологий

Было реализовано 3 программы на следующих языках программирования:

- Чистый C с реализацией классического Windows приложения в качестве GUI. Данная реализация на низкоуровневом языке программирования, позволяет увеличить производительность и поработать с памятью при разработке программы.
- Язык программирования C++ с реализацией GUI отображения через технологию классического Windows приложения. Стек и реализация во многом похожи на предшественника, однако в нём есть небольшая разница в исходном коде.
- Язык программирования Java использовался для реализации алгоритма MD5 для третьей программы на другом стеке. В качестве UI отображения и интерфейса использовался Swing.
- Среда разработки Visual Studio и IntelliJ IDEA, которые удобны для отладки и визуализации структуры проекта

Реализация на C

main.c

Это основной файл программы, который отвечает за запуск и выполнение приложения с использованием различных обработчиков ввода, вывода, вызова функций для вычисления, записи и сохранения данных. Он же отвечает за управление хэшированием. Это главный UI всей программы.

В файле в функции определяется строка, которая будет хэшироваться, либо берётся файл (ЛЮБОЙ) для хэширования, но с заданным размером не более 1КБ. Затем вызывается функция MD5 хэша строки.

```
#define _CRT_SECURE_NO_WARNINGS

#include <windows.h>
#include "md5.h"
#include <stdio.h>

#define ID_TEXTBOX 1
#define ID_BUTTON_TEXT 2
#define ID_BUTTON_FILE 3
#define ID_RESULT 4
#define ID_BUTTON_SAVE 5
#define ID_BUTTON_NEXT 6

LRESULT CALLBACK WndProc(HWND, UINT, WPARAM, LPARAM);
void ComputeMD5Text(HWND);
void ComputeMD5File(HWND);
void SaveResultToFile(HWND);
void ShowInfoScreen(HWND);
void ShowMainScreen(HWND);

BOOL mainScreenShown = FALSE;

char lastHash[256] = "";

// --- Точка входа программы с GUI (Регистрация окна) --- //
int WINAPI WinMain(HINSTANCE hInst, HINSTANCE hPrev, LPSTR args, int nCmdShow) {
    WNDCLASSW wc = { 0 };
    wc.hbrBackground = (HBRUSH)COLOR_WINDOW;
    wc.hCursor = LoadCursor(NULL, IDC_ARROW);
    wc.hInstance = hInst;
    wc.lpszClassName = L"MD5Application";
    wc.lpfnWndProc = WndProc;
    RegisterClassW(&wc);

    CreateWindowW(L"MD5Application", L"MD5 Hash Analyzer (C)",
        WS_OVERLAPPEDWINDOW | WS_VISIBLE,
        100, 100, 600, 400, NULL, NULL, NULL, NULL);

    MSG msg = { 0 };
    while (GetMessage(&msg, NULL, 0, 0)) {
        TranslateMessage(&msg);
        DispatchMessage(&msg);
    }

    return 0;
}

// --- Обработчик событий (кнопки, окна, ввод) --- //
LRESULT CALLBACK WndProc(HWND hwnd, UINT msg, WPARAM wp, LPARAM lp) {
    static HWND hTextBox, hButtonText, hButtonFile, hResult, hButtonSave, hInfo;

    switch (msg) {
        case WM_CREATE:
            ShowInfoScreen(hwnd);
            break;
    }
}
```



```

        case WM_COMMAND:
            switch (LOWORD(wp)) {
                case ID_BUTTON_NEXT:
                    if (!mainScreenShown) {
                        mainScreenShown = TRUE;
                        ShowMainScreen(hwnd);
                    }

                    break;

                case ID_BUTTON_TEXT:
                    ComputeMD5Text(hwnd);
                    break;

                case ID_BUTTON_FILE:
                    ComputeMD5File(hwnd);
                    break;

                case ID_BUTTON_SAVE:
                    SaveResultToFile(hwnd);
                    break;
            }

            break;

        case WM_DESTROY:
            PostQuitMessage(0);
            break;
    }

    return DefWindowProcW(hwnd, msg, wp, lp);
}

// === Экран со справочной информацией === //
void ShowInfoScreen(HWND hwnd) {
    CreateWindowW(L"static",
        L"Студент гр. ПИБд-41 Белянин Никита Николаевич\nВариант: №6\n\n"
        L"\nАлгоритм MD5 — это криптографическая хэш-функция,\n"
        L"создающая 128-битное (32-символьное) представление данных.\n\n"
        L"Принцип работы:\n"
        L"1□. Дополнение входного сообщения (padding)\n"
        L"2□. Добавление длины сообщения\n"
        L"3□. Инициализация буфера (A, B, C, D)\n"
        L"4□. Обработка по блокам по 512 бит\n"
        L"5□. Формирование 128-битного хэша\n\n"
        L"Используется для проверки целостности данных и цифровых подписей.",
        WS_VISIBLE | WS_CHILD | ES_LEFT, 20, 20, 540, 240,
        hwnd, NULL, NULL, NULL);

    CreateWindowW(L"button", L"Перейти к программе",
        WS_VISIBLE | WS_CHILD | WS_BORDER,
        200, 280, 180, 35, hwnd, (HMENU)ID_BUTTON_NEXT, NULL, NULL);
}

// === Основное окно для хэширования === //
void ShowMainScreen(HWND hwnd) {
    // Очистить предыдущее содержимое
    HWND hChild = GetWindow(hwnd, GW_CHILD);

    while (hChild) {
        DestroyWindow(hChild);
        hChild = GetWindow(hwnd, GW_CHILD);
    }
}

```

```

// === Поле ввода текста === //
CreateWindowW(L"static", L"Введите текст: ",
    WS_VISIBLE | WS_CHILD, 20, 20, 120, 20, hwnd, NULL, NULL, NULL);

CreateWindowW(L"edit", L"",
    WS_VISIBLE | WS_CHILD | WS_BORDER, 20, 50, 440, 25, hwnd, (HMENU)ID_TEXTBOX,
    NULL, NULL);

// --- Кнопки --- //
CreateWindowW(L"button", L"Хэшировать текст",
    WS_VISIBLE | WS_CHILD, 20, 90, 180, 30, hwnd, (HMENU)ID_BUTTON_TEXT, NULL,
    NULL);

CreateWindowW(L"button", L"Хэшировать файл",
    WS_VISIBLE | WS_CHILD, 220, 90, 180, 30, hwnd, (HMENU)ID_BUTTON_FILE, NULL,
    NULL);

CreateWindowW(L"button", L"Сохранить результат", WS_VISIBLE | WS_CHILD,
    400, 200, 160, 30, hwnd, (HMENU)ID_BUTTON_SAVE, NULL, NULL);

// --- Результат --- //
CreateWindowW(L"edit", L"", WS_VISIBLE | WS_CHILD | WS_BORDER | ES_READONLY,
    20, 140, 440, 25, hwnd, (HMENU)ID_RESULT, NULL, NULL);
}

// --- Обработка действий пользователя --- //
void ComputeMD5Text(HWND hwnd) {
    wchar_t input[256];
    char buffer[512];
    GetDlgItemTextW(hwnd, ID_TEXTBOX, input, 256);

    char text[256];
    wcstombs(text, input, sizeof(text));

    char* hash = MD5String(text);
    wprintfA(buffer, "%s", hash);
    SetDlgItemTextA(hwnd, ID_RESULT, buffer);
    strncpy_s(lastHash, sizeof(lastHash), hash, _TRUNCATE);
    free(hash);
}

void ComputeMD5File(HWND hwnd) {
    OPENFILENAMEW ofn = { 0 };
    wchar_t file[260] = L"";
    file[0] = '\0';

    ofn.lStructSize = sizeof(ofn);
    ofn.hwndOwner = hwnd;
    ofn.lpstrFilter = L"All Files\0*.*\0";
    ofn.lpstrFile = file;
    ofn.nMaxFile = sizeof(file);
    ofn.Flags = OFN_FILEMUSTEXIST | OFN_PATHMUSTEXIST;

    if (GetOpenFileNameW(&ofn)) {
        char path[260];
        wcstombs(path, file, sizeof(path));
        char* hash = MD5File(path);
        SetDlgItemTextA(hwnd, ID_RESULT, hash);
        strncpy_s(lastHash, sizeof(lastHash), hash, _TRUNCATE);
        free(hash);
    }
}

```

```

void SaveResultToFile(HWND hwnd) {
    if (strlen(lastHash) == 0) {
        MessageBoxW(hwnd, L"Нет данных для сохранения!", L"Ошибка", MB_ICONERROR);
        return;
    }

    OPENFILENAMEW ofn = { 0 };
    wchar_t file[260] = L"hash_result.txt";
    ofn.lStructSize = sizeof(ofn);
    ofn.hwndOwner = hwnd;
    ofn.lpstrFilter = L"Text Files\0*.txt\0All Files\0*.*\0";
    ofn.lpstrFile = file;
    ofn.nMaxFile = sizeof(file);
    ofn.Flags = OFN_OVERWRITEPROMPT;

    if (GetSaveFileNameW(&ofn)) {
        FILE* f;
        char path[260];
        wcstombs(path, file, sizeof(path));
        f = fopen(path, "w");

        if (f) {
            fprintf(f, "MD5 Hash Result:\n%s\n", lastHash);
            fclose(f);
            MessageBoxW(hwnd, L"Результат успешно сохранён!", L"Готово",
MB_ICONINFORMATION);
        }
    }
}

```

md5.h

Это заголовочный файл, который содержит объявления функций, структур и макросов, которые необходимы для реализации алгоритма MD5. Этот файл необходим для того, чтобы другие исходные файлы могли работать с алгоритмом. Благодаря заголовочным файлам другие исходные файлы могут вызывать функции для хэширования данных.

```

#pragma once

#ifndef MD5_H
#define MD5_H

#include <stdint.h>
#include <stddef.h>

// == Контекст MD5 (состояние вычислений) ==
/*
    state - 4 регистра (a,b,c,d);
    count - количество обработанных бит
    buffer - текущий блок данных (512 бит) */
typedef struct {

```

```

uint32_t state[4];
uint32_t count[2];
unsigned char buffer[64];
} MD5_CTX;

// ===== Основные функции ===== //

// Инициализация контекста (регистры)
void MD5Init(MD5_CTX* context);

// Добавление данных
void MD5Update(MD5_CTX* context, const unsigned char* input, size_t inputLen);

// Завершение вычисления
void MD5Final(unsigned char digest[16], MD5_CTX* context);

// Один раунд преобразования блока
void MD5Transform(uint32_t state[4], const unsigned char block[64]);

// Хэширование строки и хэширование файла
void *MD5String(const char* string);
void *MD5File(const char* filename);

#endif

```

md5.c

Это реализация самого алгоритма MD5. Он принимает данные, которые нужно хэшировать и возвращает их хэш. Функция MD5Init отвечает за инициализацию контекста, чтоб алгоритм заработал нужно регистрам дать данные. Функция MD5Update – это добавление данных, суммирует до кратности и деление на блоки. MD5Transform – это непосредственно само преобразование данных в ходе 4 раундов по 16 операций с использованием математических операций и фиксированных значений (констант). Функция выполняет хэширование данных: добавляет паддинг (если длина данных не кратна 64), затем для каждого блока производит обработку. Также используются макросы для выполнения операций в каждом из 4 раундов алгоритма MD5. Каждый макрос изменяет значение переменной а в процессе хэширования, используя операции с другими переменными и константами. Пояснения: b,c,d – промежуточные результаты, х – входные данные, s – сдвиг, ас – одна из констант для каждого шага. Также есть вспомогательные функции для обработки данных в алгоритме MD5. Функция Encode –

функция для преобразования данных в формат, пригодный для обработки в MD5 (преобразуются 32-битные числа в байтовый массив, после обработки данных представляются как массив байтов). Decode – функция для преобразования байтов в 32-битные целые числа. Это нужно, чтобы декодировать 64-байтовые блоки данных, поступающие на вход MD5, в массив целых чисел, которые далее обрабатываются алгоритмом. MD5-memcpy – эта функция выполняет копирование данных из одного места в другое. Это эквивалент стандартной функции memcpy, но в контексте MD5 она может быть использована для работы с массивами данных. А также есть две функции для вспомогательной работы с текстовой строкой и файлом.

```
#define _CRT_SECURE_NO_WARNINGS

#include "md5.h"
#include <string.h>
#include <stdlib.h>
#include <stdio.h>

#define F(x,y,z) ((x & y) | (~x & z))
#define G(x,y,z) ((x & z) | (y & ~z))
#define H(x,y,z) (x ^ y ^ z)
#define I(x,y,z) (y ^ (x | ~z))
#define ROTATE_LEFT(x,n) ((x << n) | (x >> (32 - n)))

// Макросы
#define FF(a,b,c,d,x,s,ac) { (a) += F((b),(c),(d)) + (x) + (uint32_t)(ac); (a) = ROTATE_LEFT((a), (s)); (a) += (b); }
#define GG(a,b,c,d,x,s,ac) { (a) += G((b),(c),(d)) + (x) + (uint32_t)(ac); (a) = ROTATE_LEFT((a),(s)); (a) += (b); }
#define HH(a,b,c,d,x,s,ac) { (a) += H((b),(c),(d)) + (x) + (uint32_t)(ac); (a) = ROTATE_LEFT((a),(s)); (a) += (b); }
#define II(a,b,c,d,x,s,ac) { (a) += I((b),(c),(d)) + (x) + (uint32_t)(ac); (a) = ROTATE_LEFT((a),(s)); (a) += (b); }

static void Encode(unsigned char* output, const uint32_t* input, size_t len);
static void Decode(uint32_t* output, const unsigned char* input, size_t len);
static void MD5_memcpy(unsigned char* output, const unsigned char* input, size_t len);
static void MD5_memset(unsigned char* output, int value, size_t len);

static const unsigned char PADDING[64] = { 0x80, 0 };

// == Инициализация контекста == //
void MD5Init(MD5_CTX* context) {
    context->count[0] = context->count[1] = 0;
    context->state[0] = 0x67452301;
    context->state[1] = 0xefcdab89;
    context->state[2] = 0x98badcfe;
    context->state[3] = 0x10325476;
}

// == Обновление (добавление данных, делится входные данные на блоки по 512 бит) == //
void MD5Update(MD5_CTX* context, const unsigned char* input, size_t inputLen) {
    size_t i, index, partLen;

    index = (unsigned int)((context->count[0] >> 3) & 0x3f);
    if (((context->count[0] += ((uint32_t)inputLen << 3)) < ((uint32_t)inputLen << 3))
```

```

        context->count[1]++;
        context->count[1] += ((uint32_t)inputLen >> 29);

    partLen = 64 - index;

    if (inputLen >= partLen) {
        MD5_memcpy(&context->buffer[index], input, partLen);
        MD5Transform(context->state, context->buffer);

        for (i = partLen; i + 63 < inputLen; i += 64)
            MD5Transform(context->state, &input[i]);

        index = 0;
    }
    else i = 0;

    MD5_memcpy(&context->buffer[index], &input[i], inputLen - i);
}

// === Завершение (финализация) === //
void MD5Final(unsigned char digest[16], MD5_CTX* context) {
    unsigned char bits[8];
    unsigned int index, padLen;

    Encode(bits, context->count, 8);

    index = (unsigned int)((context->count[0] >> 3) & 0x3f);
    padLen = (index < 56) ? (56 - index) : (120 - index);
    MD5Update(context, PADDING, padLen);

    MD5Update(context, bits, 8);

    Encode(digest, context->state, 16);

    MD5_memset((unsigned char*)context, 0, sizeof(*context));
}

// === Преобразование блока (ядро, где 64 шага, 4 раунда * 16 операций для функций F,G,H,I) === //
/* Шаг: перемешивание A,B,C,D; добавление констант, сдвиги и суммирование в 32-битной арифметике */
void MD5Transform(uint32_t state[4], const unsigned char block[64]) {
    uint32_t a = state[0], b = state[1], c = state[2], d = state[3], x[16];
    Decode(x, block, 64);

    // === Раунд 1 === //
    FF(a, b, c, d, x[0], 7, 0xd76aa478);
    FF(d, a, b, c, x[1], 12, 0xe8c7b756);
    FF(c, d, a, b, x[2], 17, 0x242070db);
    FF(b, c, d, a, x[3], 22, 0xc1bdceee);
    FF(a, b, c, d, x[4], 7, 0xf57c0faf);
    FF(d, a, b, c, x[5], 12, 0x4787c62a);
    FF(c, d, a, b, x[6], 17, 0xa8304613);
    FF(b, c, d, a, x[7], 22, 0xfd469501);
    FF(a, b, c, d, x[8], 7, 0x698098d8);
    FF(d, a, b, c, x[9], 12, 0x8b44f7af);
    FF(c, d, a, b, x[10], 17, 0xffff5bb1);
    FF(b, c, d, a, x[11], 22, 0x895cd7be);
    FF(a, b, c, d, x[12], 7, 0x6b901122);
    FF(d, a, b, c, x[13], 12, 0xfd987193);
    FF(c, d, a, b, x[14], 17, 0xa679438e);
    FF(b, c, d, a, x[15], 22, 0x49b40821);

    // === Раунд 2 === //
    GG(a, b, c, d, x[1], 5, 0xf61e2562);

```

```

GG(d, a, b, c, x[6], 9, 0xc040b340);
GG(c, d, a, b, x[11], 14, 0x265e5a51);
GG(b, c, d, a, x[0], 20, 0xe9b6c7aa);
GG(a, b, c, d, x[5], 5, 0xd62f105d);
GG(d, a, b, c, x[10], 9, 0x02441453);
GG(c, d, a, b, x[15], 14, 0xd8a1e681);
GG(b, c, d, a, x[4], 20, 0xe7d3fbc8);
GG(a, b, c, d, x[9], 5, 0x21e1cde6);
GG(d, a, b, c, x[14], 9, 0xc33707d6);
GG(c, d, a, b, x[3], 14, 0xf4d50d87);
GG(b, c, d, a, x[8], 20, 0x455a14ed);
GG(a, b, c, d, x[13], 5, 0xa9e3e905);
GG(d, a, b, c, x[2], 9, 0xfcefa3f8);
GG(c, d, a, b, x[7], 14, 0x676f02d9);
GG(b, c, d, a, x[12], 20, 0x8d2a4c8a);

```

```
// === Раунд 3 === //
```

```

HH(a, b, c, d, x[5], 4, 0xfffa3942);
HH(d, a, b, c, x[8], 11, 0x8771f681);
HH(c, d, a, b, x[11], 16, 0x6d9d6122);
HH(b, c, d, a, x[14], 23, 0xfde5380c);
HH(a, b, c, d, x[1], 4, 0xa4beea44);
HH(d, a, b, c, x[4], 11, 0x4bdecfa9);
HH(c, d, a, b, x[7], 16, 0xf6bb4b60);
HH(b, c, d, a, x[10], 23, 0xbebfb70);
HH(a, b, c, d, x[13], 4, 0x289b7ec6);
HH(d, a, b, c, x[0], 11, 0xeeaa127fa);
HH(c, d, a, b, x[3], 16, 0xd4ef3085);
HH(b, c, d, a, x[6], 23, 0x04881d05);
HH(a, b, c, d, x[9], 4, 0xd9d4d039);
HH(d, a, b, c, x[12], 11, 0xe6db99e5);
HH(c, d, a, b, x[15], 16, 0x1fa27cf8);
HH(b, c, d, a, x[2], 23, 0xc4ac5665);

```

```
// === Раунд 4 === //
```

```

II(a, b, c, d, x[0], 6, 0xf4292244);
II(d, a, b, c, x[7], 10, 0x432aff97);
II(c, d, a, b, x[14], 15, 0xab9423a7);
II(b, c, d, a, x[5], 21, 0xfc93a039);
II(a, b, c, d, x[12], 6, 0x655b59c3);
II(d, a, b, c, x[3], 10, 0x8f0ccc92);
II(c, d, a, b, x[10], 15, 0xffeff47d);
II(b, c, d, a, x[1], 21, 0x85845dd1);
II(a, b, c, d, x[8], 6, 0x6fa87e4f);
II(d, a, b, c, x[15], 10, 0xfe2ce6e0);
II(c, d, a, b, x[6], 15, 0xa3014314);
II(b, c, d, a, x[13], 21, 0x4e0811a1);
II(a, b, c, d, x[4], 6, 0xf7537e82);
II(d, a, b, c, x[11], 10, 0xbd3af235);
II(c, d, a, b, x[2], 15, 0x2ad7d2bb);
II(b, c, d, a, x[9], 21, 0xeb86d391);

```

```

state[0] += a;
state[1] += b;
state[2] += c;
state[3] += d;

```

```
MD5_memset((unsigned char*)x, 0, sizeof(x));
```

```
}
```

```
// === Утилиты для конвертации uint32 и байты === //
```

```

static void Encode(unsigned char* output, const uint32_t* input, size_t len) {
    for (size_t i = 0, j = 0; j < len; i++, j += 4) {

```

```

        output[j] = (unsigned char)(input[i] & 0xff);
        output[j + 1] = (unsigned char)((input[i] >> 8) & 0xff);
        output[j + 2] = (unsigned char)((input[i] >> 16) & 0xff);
        output[j + 3] = (unsigned char)((input[i] >> 24) & 0xff);
    }
}

```

```

static void Decode(uint32_t* output, const unsigned char* input, size_t len) {
    for (size_t i = 0, j = 0; j < len; i++, j += 4)
        output[i] = ((uint32_t)input[j]) |
            (((uint32_t)input[j + 1]) << 8) |
            (((uint32_t)input[j + 2]) << 16) |
            (((uint32_t)input[j + 3]) << 24);
}

```

```

static void MD5_memcpy(unsigned char* output, const unsigned char* input, size_t len) {
    memcpy(output, input, len);
}

```

```

static void MD5_memset(unsigned char* output, int value, size_t len) {
    memset(output, value, len);
}

```

```

char* MD5String(const char* string) {
    MD5_CTX context;
    unsigned char digest[16];
    char* output = (char*)malloc(33);
    if (!output) return NULL;

    MD5Init(&context);
    MD5Update(&context, (const unsigned char*)string, strlen(string));
    MD5Final(digest, &context);

    for (int i = 0; i < 16; ++i)
        sprintf(&output[i * 2], "%02x", digest[i]);

    output[32] = '\0';
    return output;
}

```

```

char* MD5File(const char* filename) {
    FILE* file = fopen(filename, "rb");
    if (!file) return _strdup("Ошибка открытия файла");

    MD5_CTX context;
    unsigned char buffer[1024], digest[16];
    size_t bytes;
    char* output = (char*)malloc(33);
    if (!output) return NULL;

    MD5Init(&context);
    while ((bytes = fread(buffer, 1, sizeof(buffer), file)) != 0)
        MD5Update(&context, buffer, bytes);
    MD5Final(digest, &context);
    fclose(file);

    for (int i = 0; i < 16; ++i)
        sprintf(&output[i * 2], "%02x", digest[i]);

    output[32] = '\0';
    return output;
}

```


Реализация на C++

В этой реализации используется так же несколько файлов, каждый из которых используется для вычисления MD5 хэш-суммы.

main.cpp

Этот файл является точкой входа в программу. В нем реализована функция WinMain, которая является основной для Windows-приложений. Основная задача этого файла — это инициализация и запуск графического интерфейса программы, а также настройка основного окна.

Регистрация окна — это создание и регистрация класса окна, который будет использоваться в дальнейшем. Затем происходит создание главного окна программы, в котором будут размещены все элементы интерфейса.

Основной цикл обработки сообщений — это фрагмент программы использует цикл сообщений (GetMessage, DispatchMessage), который обрабатывает все события, такие как нажатия кнопок, обновления окон. Этот файл управляет созданием графического интерфейса пользователя (GUI), который предоставляет интерактивные элементы для ввода данных и получения результатов хэширования MD5.

```
#define _CRT_SECURE_NO_WARNINGS
#include <windows.h>
#include <commdlg.h>
#include <string>
#include <fstream>
#include <chrono>
#include <ctime>
#include "md5.h"

LRESULT CALLBACK WndProc(HWND, UINT, WPARAM, LPARAM);
void ShowReference();
void SaveHashToFile(const std::string& hash, const std::wstring& source);

// элементы интерфейса
HWND hEditInput, hEditOutput, hBtnText, hBtnFile, hBtnInfo;

int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE, LPSTR, int nCmdShow) {
    const wchar_t CLASS_NAME[] = L"MD5HashApp";

    WNDCLASS wc = {};
    wc.lpfnWndProc = WndProc;
    wc.hInstance = hInstance;
```

```

wc.lpszClassName = CLASS_NAME;
wc.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);

RegisterClass(&wc);

HWND hwnd = CreateWindowEx(
    0,
    CLASS_NAME,
    L"MD5 Hash Analyzer (WinAPI)",
    WS_OVERLAPPEDWINDOW ^ WS_THICKFRAME,
    CW_USEDEFAULT, CW_USEDEFAULT, 500, 300,
    nullptr, nullptr, hInstance, nullptr
);

if (!hwnd) return 0;
ShowWindow(hwnd, nCmdShow);

MSG msg = {};
while (GetMessage(&msg, nullptr, 0, 0)) {
    TranslateMessage(&msg);
    DispatchMessage(&msg);
}
return (int)msg.wParam;
}

// === создание окна и элементов ===
LRESULT CALLBACK WndProc(HWND hwnd, UINT msg, WPARAM wParam, LPARAM lParam) {
    switch (msg) {
        case WM_CREATE: {
            CreateWindow(L"STATIC", L"Введите текст:", WS_VISIBLE | WS_CHILD,
                20, 20, 100, 20, hwnd, nullptr, nullptr, nullptr);

            hEditInput = CreateWindow(L"EDIT", L"", WS_VISIBLE | WS_CHILD | WS_BORDER,
                130, 20, 320, 25, hwnd, nullptr, nullptr, nullptr);

            hBtnText = CreateWindow(L"BUTTON", L"Хэшировать текст", WS_VISIBLE | WS_CHILD,
                20, 60, 150, 30, hwnd, (HMENU)1, nullptr, nullptr);

            hBtnFile = CreateWindow(L"BUTTON", L"Хэшировать файл", WS_VISIBLE | WS_CHILD,
                180, 60, 150, 30, hwnd, (HMENU)2, nullptr, nullptr);

            hBtnInfo = CreateWindow(L"BUTTON", L"Справка", WS_VISIBLE | WS_CHILD,
                340, 60, 110, 30, hwnd, (HMENU)3, nullptr, nullptr);

            CreateWindow(L"STATIC", L"Результат MD5:", WS_VISIBLE | WS_CHILD,
                20, 110, 120, 20, hwnd, nullptr, nullptr, nullptr);

            hEditOutput = CreateWindow(L"EDIT", L"", WS_VISIBLE | WS_CHILD | WS_BORDER |
                ES_AUTOHSCROLL,
                150, 110, 300, 25, hwnd, nullptr, nullptr, nullptr);
            break;
        }

        case WM_COMMAND: {
            switch (LOWORD(wParam)) {
                case 1: { // хэшировать текст
                    wchar_t buffer[1024];
                    GetWindowText(hEditInput, buffer, 1024);
                    std::wstring text(buffer);
                    if (text.empty()) {
                        MessageBox(hwnd, L"Введите текст для хэширования!", L"Ошибка", MB_ICONWARNING);
                    }
                    break;
                }
            }
        }
    }
}

```

```

MD5 md5;
md5.update(std::string(text.begin(), text.end()));
std::string result = md5.finalize();
std::wstring wresult(result.begin(), result.end());
SetWindowText(hEditOutput, wresult.c_str());
SaveHashToFile(result, L"Введённый текст");
break;
}

```

```

case 2: { // хэшировать файл
    OPENFILENAME ofn = {};
    wchar_t fileName[MAX_PATH] = L"";
    ofn.lStructSize = sizeof(ofn);
    ofn.hwndOwner = hwnd;
    ofn.lpstrFile = fileName;
    ofn.nMaxFile = MAX_PATH;
    ofn.lpstrFilter = L"Все файлы\0*.*\0";
    ofn.Flags = OFN_FILEMUSTEXIST;

```

```

    if (GetOpenFileName(&ofn)) {
        try {
            std::string path(fileName, fileName + wcslen(fileName));
            std::string result = MD5::fromFile(path);
            std::wstring wresult(result.begin(), result.end());
            SetWindowText(hEditOutput, wresult.c_str());
            SaveHashToFile(result, fileName);
        }
        catch (const std::exception& e) {
            MessageBoxA(hwnd, e.what(), "Ошибка", MB_ICONERROR);
        }
    }
    break;
}

```

```

case 3:
    ShowReference();
    break;
}
break;
}

```

```

case WM_DESTROY:
    PostQuitMessage(0);
    break;
}
return DefWindowProc(hwnd, msg, wParam, lParam);
}

```

```

// === окно со справочной информацией ===
void ShowReference() {
    MessageBox(
        nullptr,
        L"Студент гр. ПИБд-41 Белянин Никита Николаевич\nВариант: №6\n\n"
        L"\nАлгоритм MD5 — это криптографическая хэш-функция,\n"
        L"создающая 128-битное (32-символьное) представление данных.\n\n"
        L"MD5 (Message Digest 5)\n\n"
        L"• Разработан Рональдом Ривестом в 1991 году\n"
        L"• Длина хэша: 128 бит (32 шестнадцатеричных символа)\n"
        L"• Применяется для проверки целостности данных\n"
        L"• Не рекомендуется для криптографической защиты\n"
        L"\nПрограмма позволяет вычислить MD5 хэш строки или файла.",
        L"Справочная информация"
    );
}

```

```

        MB_ICONINFORMATION
    );
}

// === сохранение хэша в файл ===
void SaveHashToFile(const std::string& hash, const std::wstring& source) {
    std::ofstream file("md5_result.txt", std::ios::app);
    if (!file.is_open()) return;

    // текущее время
    auto now = std::chrono::system_clock::now();
    std::time_t now_c = std::chrono::system_clock::to_time_t(now);

    file << "=== Result MD5 ===\n";
    file << "Date and time: " << std::ctime(&now_c);

    file << "hash: " << hash << "\n";
    file << "=====\n\n";

    file.close();

    MessageBox(nullptr, L"Результат успешно сохранён в файлик",
        L"Сохранение", MB_OK | MB_ICONINFORMATION);
    file << "MD5: " << hash << "\n";
}

```

md5.h

Этот файл заголовка содержит объявления классов и методов, которые реализуют алгоритм MD5. Он позволяет другим частям программы (например, main.c) взаимодействовать с функциональностью. md5.h – это файл с прототипами функций, которые необходимы для работы с MD5. Этот заголовочный файл обеспечивают доступ к необходимым функциям Windows, стандартным библиотекам C++, а также позволяют реализовать взаимодействие с пользователем через графический интерфейс.

Класс MD5 — это объявление класса, который инкапсулирует все функции для работы с алгоритмом MD5. В классе содержатся методы обновления состояния алгоритма новыми данными (update()), завершение хэширования и получение результата (finalize()) и вычисление хэша для содержимого файла (fromFile()). А так же есть другие вспомогательные методы для кодирования и декодирования данных, а также преобразования данных в шестнадцатеричную строку.

Этот файл служит интерфейсом для взаимодействия с функциональностью MD5, а также описывает, как использовать алгоритм MD5 в других частях программы. А так же есть соответствующие стандартные библиотеки для работы со строками, потоками, файловыми операциями.

```
#pragma once

#ifndef MD5_H
#define MD5_H

#include <string>
#include <array>
#include <stdint>
#include <vector>

class MD5 {
public:
    MD5();
    void update(const std::string& data);
    std::string finalize();
    static std::string fromFile(const std::string& path);

private:
    void transform(const uint8_t block[64]);
    void encode(uint8_t* output, const uint32_t* input, size_t len);
    void decode(uint32_t* output, const uint8_t* input, size_t len);
    void reset();
    static std::string bytesToHex(const uint8_t* bytes, size_t length);

    bool finalized;
    uint64_t count;
    std::array<uint32_t, 4> state;
    std::array<uint8_t, 64> buffer;
    std::array<uint8_t, 16> digest;
};

#endif
```

md5.cpp

Этот файл содержит основную логику алгоритма MD5. В нем реализованы все функции для вычисления хэш-суммы по алгоритму MD5, начиная от преобразования входных данных до вычисления окончательной хэш-суммы. Функционал:

- MD5::update – функция, которая обновляет состояние хэш-суммы с новыми данными (текстом или содержимым файла).

- MD5::transform – обрабатывает 64-байтовый блок данных и применяет к нему основную логику MD5 через четыре раунда с использованием функций F, G, H и I.
- MD5::finalize – завершается процесс хэширования, дополняя данные, если это необходимо, и вычисляет окончательную хэш-сумму.
- MD5::bytesToHex – преобразует полученный результат в шестнадцатеричную строку для удобства отображения.
- MD5::fromFile – позволяет вычислить хэш-сумму для содержимого файла, читая его блоками.

Этот файл представляет собой реализацию самого алгоритма MD5, который используется для вычисления хэш-сумм как для строк, так и для файлов. Он является основным вычислительным компонентом программы.

```
#include "md5.h"
#include <fstream>
#include <cstring>
#include <cmath>
#include <sstream>
#include <iomanip>
#include <stdexcept>

static inline uint32_t F(uint32_t x, uint32_t y, uint32_t z) { return (x & y) | (~x & z); }
static inline uint32_t G(uint32_t x, uint32_t y, uint32_t z) { return (x & z) | (y & ~z); }
static inline uint32_t H(uint32_t x, uint32_t y, uint32_t z) { return x ^ y ^ z; }
static inline uint32_t I(uint32_t x, uint32_t y, uint32_t z) { return y ^ (x | ~z); }

static inline uint32_t ROTATE_LEFT(uint32_t x, uint32_t n) { return (x << n) | (x >> (32 - n)); }

// Макросы
#define FF(a,b,c,d,x,s,ac) { a += F(b, c, d) + x + ac; a = ROTATE_LEFT(a, s); a += b; }
#define GG(a,b,c,d,x,s,ac) { a += G(b,c,d) + x + ac; a = ROTATE_LEFT(a, s); a += b; }
#define HH(a,b,c,d,x,s,ac) { a += H(b,c,d) + x + ac; a = ROTATE_LEFT(a, s); a += b; }
#define II(a,b,c,d,x,s,ac) { a += I(b,c,d) + x + ac; a = ROTATE_LEFT(a, s); a += b; }

MD5::MD5() { reset(); }

void MD5::reset() {
    finalized = false;
    count = 0;
    state = { 0x67452301, 0xefcdab89, 0x98badcfe, 0x10325476 };
    std::fill(buffer.begin(), buffer.end(), 0);
}

void MD5::update(const std::string& data) {
    if (finalized) throw std::runtime_error("MD5 already finalized");
    const uint8_t* input = reinterpret_cast<const uint8_t*>(data.data());
    size_t len = data.size();
```

```
size_t index = (count / 8) % 64;
count += static_cast<uint64_t>(len) * 8;
```

```
size_t partLen = 64 - index;
size_t i = 0;
```

```
if (len >= partLen) {
    std::memcpy(&buffer[index], input, partLen);
    transform(buffer.data());
    for (i = partLen; i + 63 < len; i += 64)
        transform(&input[i]);
    index = 0;
}
```

```
std::memcpy(&buffer[index], &input[i], len - i);
}
```

```
void MD5::transform(const uint8_t block[64]) {
    uint32_t a = state[0], b = state[1], c = state[2], d = state[3], x[16];
    decode(x, block, 64);
```

```
// === Раунд 1 === //
```

```
FF(a, b, c, d, x[0], 7, 0xd76aa478); FF(d, a, b, c, x[1], 12, 0xe8c7b756);
FF(c, d, a, b, x[2], 17, 0x242070db); FF(b, c, d, a, x[3], 22, 0xc1bdcee5);
FF(a, b, c, d, x[4], 7, 0xf57c0faf); FF(d, a, b, c, x[5], 12, 0x4787c62a);
FF(c, d, a, b, x[6], 17, 0xa8304613); FF(b, c, d, a, x[7], 22, 0xfd469501);
FF(a, b, c, d, x[8], 7, 0x698098d8); FF(d, a, b, c, x[9], 12, 0x8b44f7af);
FF(c, d, a, b, x[10], 17, 0xffff5bb1); FF(b, c, d, a, x[11], 22, 0x895cd7be);
FF(a, b, c, d, x[12], 7, 0x6b901122); FF(d, a, b, c, x[13], 12, 0xfd987193);
FF(c, d, a, b, x[14], 17, 0xa679438e); FF(b, c, d, a, x[15], 22, 0x49b40821);
```

```
// === Раунд 2 === //
```

```
GG(a, b, c, d, x[1], 5, 0xf61e2562); GG(d, a, b, c, x[6], 9, 0xc040b340);
GG(c, d, a, b, x[11], 14, 0x265e5a51); GG(b, c, d, a, x[0], 20, 0xe9b6c7aa);
GG(a, b, c, d, x[5], 5, 0xd62f105d); GG(d, a, b, c, x[10], 9, 0x02441453);
GG(c, d, a, b, x[15], 14, 0xd8a1e681); GG(b, c, d, a, x[4], 20, 0xe7d3fbc8);
GG(a, b, c, d, x[9], 5, 0x21e1cde6); GG(d, a, b, c, x[14], 9, 0xc33707d6);
GG(c, d, a, b, x[3], 14, 0xf4d50d87); GG(b, c, d, a, x[8], 20, 0x455a14ed);
GG(a, b, c, d, x[13], 5, 0xa9e3e905); GG(d, a, b, c, x[2], 9, 0xfcefa3f8);
GG(c, d, a, b, x[7], 14, 0x676f02d9); GG(b, c, d, a, x[12], 20, 0x8d2a4c8a);
```

```
// === Раунд 3 === //
```

```
HH(a, b, c, d, x[5], 4, 0xfffa3942); HH(d, a, b, c, x[8], 11, 0x8771f681);
HH(c, d, a, b, x[11], 16, 0x6d9d6122); HH(b, c, d, a, x[14], 23, 0xfde5380c);
HH(a, b, c, d, x[1], 4, 0xa4beea44); HH(d, a, b, c, x[4], 11, 0x4bdecfa9);
HH(c, d, a, b, x[7], 16, 0xf6bb4b60); HH(b, c, d, a, x[10], 23, 0xbebfbf70);
HH(a, b, c, d, x[13], 4, 0x289b7ec6); HH(d, a, b, c, x[0], 11, 0xeaad127fa);
HH(c, d, a, b, x[3], 16, 0xd4ef3085); HH(b, c, d, a, x[6], 23, 0x04881d05);
HH(a, b, c, d, x[9], 4, 0xd9d4d039); HH(d, a, b, c, x[12], 11, 0xe6db99e5);
HH(c, d, a, b, x[15], 16, 0x1fa27cf8); HH(b, c, d, a, x[2], 23, 0xc4ac5665);
```

```
// === Раунд 4 === //
```

```
II(a, b, c, d, x[0], 6, 0xf4292244); II(d, a, b, c, x[7], 10, 0x432aff97);
II(c, d, a, b, x[14], 15, 0xab9423a7); II(b, c, d, a, x[5], 21, 0xfc93a039);
II(a, b, c, d, x[12], 6, 0x655b59c3); II(d, a, b, c, x[3], 10, 0x8f0ccc92);
II(c, d, a, b, x[10], 15, 0xffff47d); II(b, c, d, a, x[1], 21, 0x85845dd1);
II(a, b, c, d, x[8], 6, 0x6fa87e4f); II(d, a, b, c, x[15], 10, 0xfe2ce6e0);
II(c, d, a, b, x[6], 15, 0xa3014314); II(b, c, d, a, x[13], 21, 0x4e0811a1);
II(a, b, c, d, x[4], 6, 0xf7537e82); II(d, a, b, c, x[11], 10, 0xbd3af235);
II(c, d, a, b, x[2], 15, 0x2ad7d2bb); II(b, c, d, a, x[9], 21, 0xeb86d391);
```

```
state[0] += a; state[1] += b; state[2] += c; state[3] += d;
```

```

}

void MD5::encode(uint8_t* output, const uint32_t* input, size_t len) {
    for (size_t i = 0, j = 0; j < len; ++i, j += 4) {
        output[j] = (uint8_t)(input[i] & 0xff);
        output[j + 1] = (uint8_t)((input[i] >> 8) & 0xff);
        output[j + 2] = (uint8_t)((input[i] >> 16) & 0xff);
        output[j + 3] = (uint8_t)((input[i] >> 24) & 0xff);
    }
}

void MD5::decode(uint32_t* output, const uint8_t* input, size_t len) {
    for (size_t i = 0, j = 0; j < len; ++i, j += 4)
        output[i] = (((uint32_t)input[j]) |
                     (((uint32_t)input[j + 1]) << 8) |
                     (((uint32_t)input[j + 2]) << 16) |
                     (((uint32_t)input[j + 3]) << 24));
}

std::string MD5::bytesToHex(const uint8_t* bytes, size_t length) {
    std::ostringstream oss;
    oss << std::hex << std::setfill('0');

    for (size_t i = 0; i < length; ++i)
        oss << std::setw(2) << static_cast<int>(bytes[i] & 0xff);

    return oss.str();
}

std::string MD5::finalize() {
    if (finalized) return bytesToHex(digest.data(), 16);

    uint8_t bits[8];

    encode(bits, (uint32_t*)&count, 8);

    // вычисление кол-ва байт в буфере
    size_t index = (count / 8) % 64;
    size_t padLen = (index < 56) ? (56 - index) : (120 - index);
    static const uint8_t PADDING[64] = { 0x80 };

    update(std::string((char*)PADDING, padLen));
    update(std::string((char*)bits, 8));

    // Формирование итогового ответа (state -> digest)
    encode(digest.data(), state.data(), 16);

    finalized = true;
    return bytesToHex(digest.data(), 16);
}

std::string MD5::fromFile(const std::string& path) {
    std::ifstream file(path, std::ios::binary);

    if (!file.is_open())
        throw std::runtime_error("Не удалось открыть файл.");

    MD5 md5;
    std::vector<char> buffer(1024);

    while (file.good()) {
        file.read(buffer.data(), buffer.size());
        md5.update(std::string(buffer.data(), file.gcount()));
    }
}

```



```

    }

    file.close();
    return md5.finalize();
}

```

Реализация на Java

Main.java

Этот файл содержит точку входа в приложение (main метод). Он отвечает за запуск графического интерфейса Swing. Внутри потока он пытается установить системный внешний вид для элементов интерфейса, затем создает и отображает главное окно приложения (MainWindow), а также сразу вызывает окно справки.

```

package org.example;

import javax.swing.*;

public class Main {
    public static void main(String[] args) {

        SwingUtilities.invokeLater(() -> {
            try {
                UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
            } catch (Exception ignored) {}
        });

        MainWindow window = new MainWindow();
        window.setVisible(true);
        window.showInfo();

    }
}

```

MainWindow.java

Это основное окно приложения, наследуемое от JFrame. Оно реализует пользовательский интерфейс, позволяющий пользователю взаимодействовать с функционалом хэширования. Окно содержит текстовое поле для ввода строки, кнопки для хэширования текста или файла, поле для отображения результата, кнопку для сохранения хэша в файл и кнопку для вызова справки. Класс обрабатывает нажатия на кнопки, вызывая соответствующие методы для выполнения операций.

```

package org.example;

import javax.swing.*;
import java.awt.event.ActionEvent;
import java.io.File;

public class MainWindow extends JFrame {

    private final JTextField inputField;
    private final JTextField resultField;

    private final JButton hashTextButton;
    private final JButton hashFileButton;
    private final JButton saveButton;
    private final JButton infoButton;

    private String lastHash = "";

    public MainWindow() {
        setTitle("MD5 Hash Analyzer (Java)");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(600, 320);
        setLocationRelativeTo(null);
        setLayout(null);

        JLabel labelInput = new JLabel("Введите текст: ");
        labelInput.setBounds(20, 20, 150, 25);
        add(labelInput);

        inputField = new JTextField();
        inputField.setBounds(20, 50, 540, 25);
        add(inputField);

        hashTextButton = new JButton("Хэшировать текст");
        hashTextButton.setBounds(20, 90, 170, 30);
        hashTextButton.addActionListener(this::onHashText);
        add(hashTextButton);

        hashFileButton = new JButton("Хэшировать файл");
        hashFileButton.setBounds(200, 90, 170, 30);
        hashFileButton.addActionListener(this::onHashFile);
        add(hashFileButton);

        saveButton = new JButton("Сохранить хэш в файл");
        saveButton.setBounds(380, 90, 180, 30);
        saveButton.addActionListener(this::onSaveHash);
        add(saveButton);

        infoButton = new JButton("Справка");
        infoButton.setBounds(20, 220, 120, 30);
        infoButton.addActionListener(e -> showInfo());
        add(infoButton);

        JLabel labelResult = new JLabel("Результат MD5: ");
        labelResult.setBounds(20, 140, 150, 25);
        add(labelResult);

        resultField = new JTextField();
        resultField.setEditable(false);
        resultField.setBounds(20, 170, 540, 25);
        add(resultField);
    }
}

```

```

private void onHashText(ActionEvent e) {
    String text = inputField.getText();
    if (text.isEmpty()) {
        JOptionPane.showMessageDialog(this, "Введите текст для хэширования", "Ошибка",
JOptionPane.WARNING_MESSAGE);
        return;
    }

    lastHash = MD5.hash(text.getBytes());
    resultField.setText(lastHash);
}

private void onHashFile(ActionEvent e) {
    JFileChooser chooser = new JFileChooser();
    int result = chooser.showOpenDialog(this);

    if (result == JFileChooser.APPROVE_OPTION) {
        File file = chooser.getSelectedFile();

        try {
            lastHash = MD5.hashFromFile(file);
            resultField.setText(lastHash);
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(this, "Ошибка при чтении файла: \n" + ex.getMessage(), "Ошибка",
JOptionPane.ERROR_MESSAGE);
        }
    }
}

private void onSaveHash(ActionEvent e) {
    if (lastHash.isEmpty()) {
        JOptionPane.showMessageDialog(this, "Нет результата для сохранения!", "Ошибка",
JOptionPane.WARNING_MESSAGE);
        return;
    }

    MD5.saveToFile(lastHash);
    JOptionPane.showMessageDialog(this, "Хэш успешно сохранён в файл", "Успех",
JOptionPane.INFORMATION_MESSAGE);
}

public void showInfo() {
    JOptionPane.showMessageDialog(this,
        "
        Студент: Белянин Никита Николаевич
        Группа: ПИбд-41
        Вариант №6
    ");
}

    Алгоритм MD5 — криптографическая хэш-функция,
    создающая 128-битное (32-символьное) представление данных.

    Этапы алгоритма:
    - Дополнение данных до длины 512 бит - 64.
    - Разбиение на блоки по 512 бит.
    - Инициализация 4-х регистров (A, B, C, D).
    - 64 раунда с функциями F, G, H, I и сдвигами.
    - Склеивание результата в 128-битный хэш.

    Программа позволяет вычислить MD5 хэш строки или файла
    и по желанию сохранить результат

    "
    ",
    "Справочная информация",

```

```
JOptionPane.INFORMATION_MESSAGE);  
}  
}
```

MD5.java

Класс MD5 реализует алгоритм криптографического хэширования MD5 с нуля, без использования встроенных библиотек Java. Он предоставляет методы для вычисления хэша как из массива байтов, так и из содержимого файла, а также позволяет сохранять полученный хэш в текстовый файл.

- `private static final int[] S` – этот массив содержит 64 значения, определяющих количество битов, на которое нужно циклически сдвигать результат в каждом раунде. Эти значения разделены на 4 группы по 16 значений, каждая из которых используется в одном из четырёх раундов.
- `private static final int[] K` – это массив констант, вычисленных для каждого из 64 раундов. Эти константы добавляются в процессе вычислений для усложнения и усиления криптографической стойкости.
- Метод `hashFromFile(File file)` – принимает файл, читает его содержимое и вызывает основной метод `hash(byte[])`.
- Метод `hash(byte[] input)` – основной метод, реализующий все этапы алгоритма MD5. В нём осуществляется подсчёт длины сообщения, определение количества блоков, добавление padding (дополнения): добавление бита 1, затем нулей, и в конец — длины сообщения в битах. Производится инициализация начальных значений регистров A, B, C, D. Обработка каждого 512-битного (64-байтового) блока. Выполнение 64 раундов с вычислением функций F, G, H, I. Формирование финального 128-битного хэша. Преобразование результата в 32-символьную шестнадцатеричную строку.

Метод `saveToFile(String hash)` – запись полученного хэша в файл `md5_result.txt`.

```

package org.example;

import java.io.File;
import java.io.FileInputStream;
import java.io.FileWriter;
import java.io.IOException;

public class MD5 {
    // Массив сдвигов для каждого из 64 раундов (S-таблица)
    private static final int[] S = {
        7, 12, 17, 22, 7, 12, 17, 22, 7, 12, 17, 22, 7, 12, 17, 22,
        5, 9, 14, 20, 5, 9, 14, 20, 5, 9, 14, 20, 5, 9, 14, 20,
        4, 11, 16, 23, 4, 11, 16, 23, 4, 11, 16, 23, 4, 11, 16, 23,
        6, 10, 15, 21, 6, 10, 15, 21, 6, 10, 15, 21, 6, 10, 15, 21
    };

    private static final int[] K = new int[64];

    static {
        for (int i = 0; i < 64; i++)
            K[i] = (int) (long) ((1L << 32) * Math.abs(Math.sin(i + 1)));
    }

    // Вычисляет MD5-хэш из содержимого файла
    public static String hashFromFile(File file) throws IOException {
        FileInputStream fis = new FileInputStream(file);
        byte[] data = fis.readAllBytes();
        fis.close();
        return hash(data);
    }

    // Основной метод, реализующий алгоритм MD5
    /* Подсчёт длины сообщения. Вычисление количества 512-битных блоков. Подготовка padding
    Запись длины в биты в последние 8 байт padding. */
    public static String hash(byte[] input) {
        int messageLen = input.length;
        int numBlocks = ((messageLen + 8) >>> 6) + 1;
        int totalLen = numBlocks << 6;

        byte[] padding = new byte[totalLen - messageLen];
        padding[0] = (byte) 0x80;

        long messageLenBits = (long) messageLen << 3;
        for (int i = 0; i < 8; i++) {
            padding[padding.length - 8 + i] = (byte) (messageLenBits >>> (8 * i));
        }

        // Инициализация начальных значений регистров (A, B, C, D)
        int A = 0x67452301;
        int B = 0xefcdab89;
        int C = 0x98badcfe;
        int D = 0x10325476;

        // Массив для хранения 64-байтового блока
        byte[] block = new byte[64];

        /* Цикл по всем 512-битным блокам. Заполнение блока данными из input или padding
        Разбиение 64-байтового блока на 16 32-битных слов (M[0]..M[15]).
        Сборка 32-битного слова из 4 байтов (little-endian) */
        for (int i = 0; i < numBlocks; i++) {
            int index = i << 6;
            for (int j = 0; j < 64; j++) {
                block[j] = (index + j < messageLen) ? input[index + j] : padding[index + j - messageLen];
            }
        }
    }
}

```

```
}
```

```
int[] M = new int[16];  
for (int j = 0; j < 16; j++) {  
    M[j] = ((block[j * 4] & 0xff) |  
            ((block[j * 4 + 1] & 0xff) << 8) |  
            ((block[j * 4 + 2] & 0xff) << 16) |  
            ((block[j * 4 + 3] & 0xff) << 24));  
}
```

```
// Сохранение начальных значений для текущего блока и 64 раунда алгоритма  
int a = A, b = B, c = C, d = D;
```

```
for (int j = 0; j < 64; j++) {  
    int F, g;  
    if (j < 16) {  
        F = (b & c) | (~b & d);  
        g = j;  
    } else if (j < 32) {  
        F = (d & b) | (~d & c);  
        g = (5 * j + 1) % 16;  
    } else if (j < 48) {  
        F = b ^ c ^ d;  
        g = (3 * j + 5) % 16;  
    } else {  
        F = c ^ (b | ~d);  
        g = (7 * j) % 16;  
    }  
}
```

```
// Обновление регистров после обработки блока  
int temp = d;  
d = c;  
c = b;  
b = b + Integer.rotateLeft(a + F + K[j] + M[g], S[j]);  
a = temp;  
}
```

```
A += a;  
B += b;  
C += c;  
D += d;  
}
```

```
// Формирование финального 128-битного хэша из регистров  
byte[] digest = new byte[16];  
int[] parts = {A, B, C, D};
```

```
for (int i = 0; i < 4; i++) {  
    for (int j = 0; j < 4; j++) {  
        digest[i * 4 + j] = (byte) (parts[i] >>> (8 * j));  
    }  
}
```

```
// Формирование финального 128-битного хэша из регистров  
StringBuilder sb = new StringBuilder();
```

```
for (byte b : digest) {  
    sb.append(String.format("%02x", b & 0xff));  
}
```

```
return sb.toString();  
}
```

```

public static void saveToFile(String hash) {
    try (FileWriter fw = new FileWriter("md5_result.txt", true)) {
        fw.write("MD5: " + hash + "\n");
    } catch (IOException e) {
        e.printStackTrace();
    }
}
}

```

Демонстрация работы программной системы

Приступим к демонстрации программ. После отладки и запуска программы, нам предоставляется справочная информация об авторе программы, вариант и описание алгоритма, далее по кнопке «Перейти к программе» можно перейти на выполнение программы.

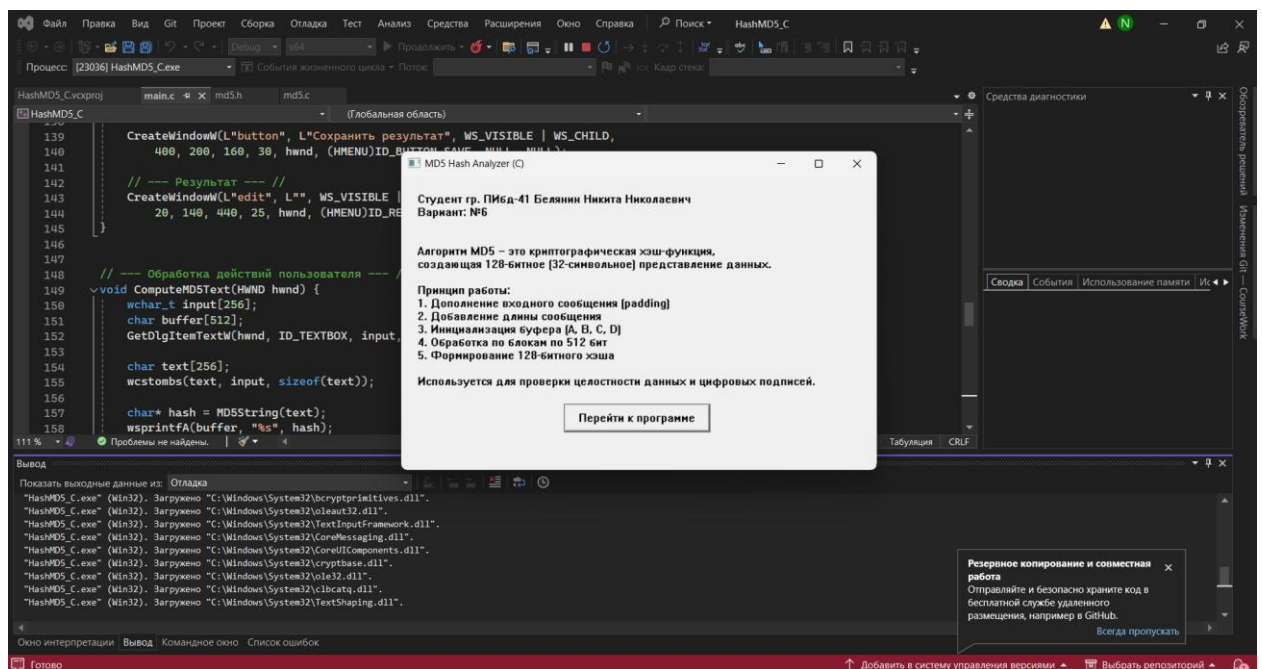


Рисунок 13. Программа запущенная и отлаженная в Vusial Studio.

На экране с работой по хэшированию, есть 3 кнопки: для хэширования текста «Хэшировать текст», для хэширования файла «Хэшировать файл» и сохранения результата в файл «Сохранить в файл». Так же для заполнения текстовых данных есть строка ввода и строка вывода для результата. Для выбора и сохранения файла предлагается выбрать директорию для удобства. Все результаты хэширования будут представлены ниже в разделе результатов. Приступим к демонстрации экранов.

Для реализации на языке C.

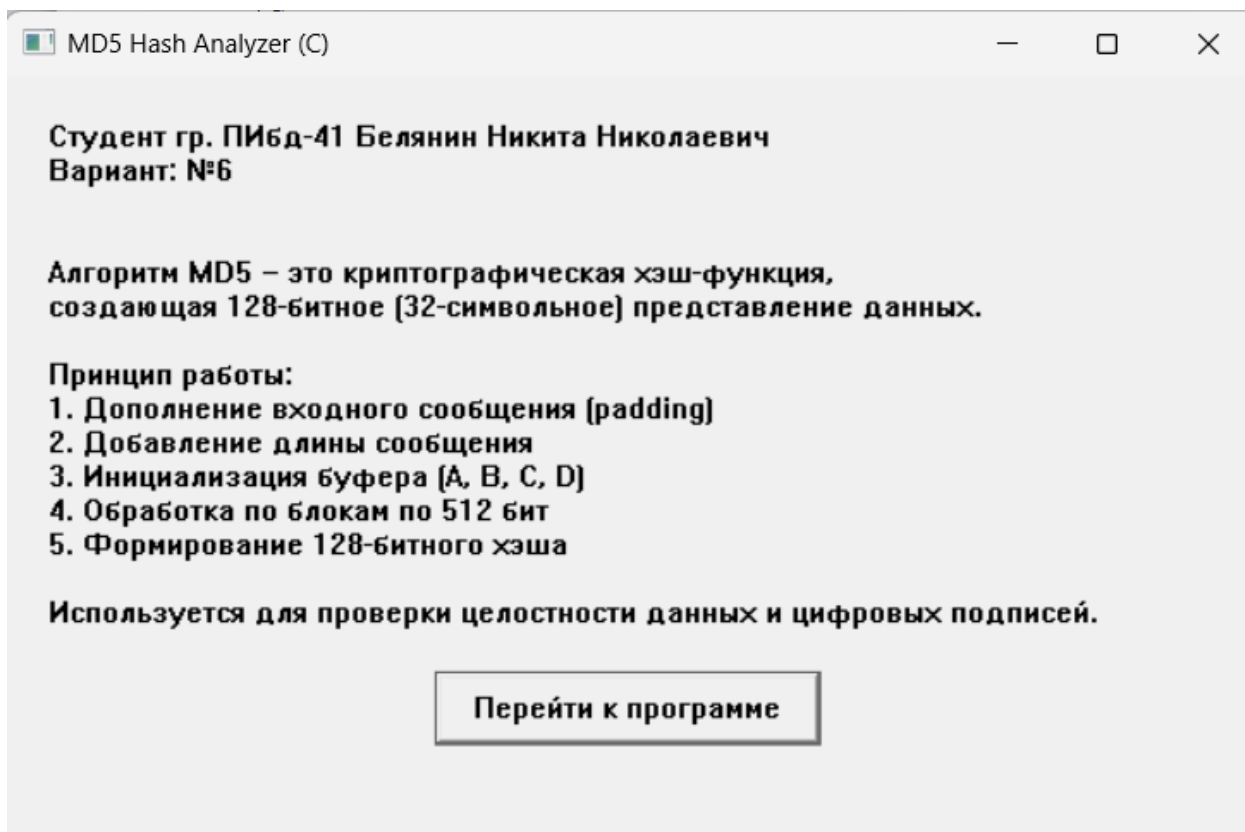


Рисунок 14. Экран со справочной информацией об авторе.

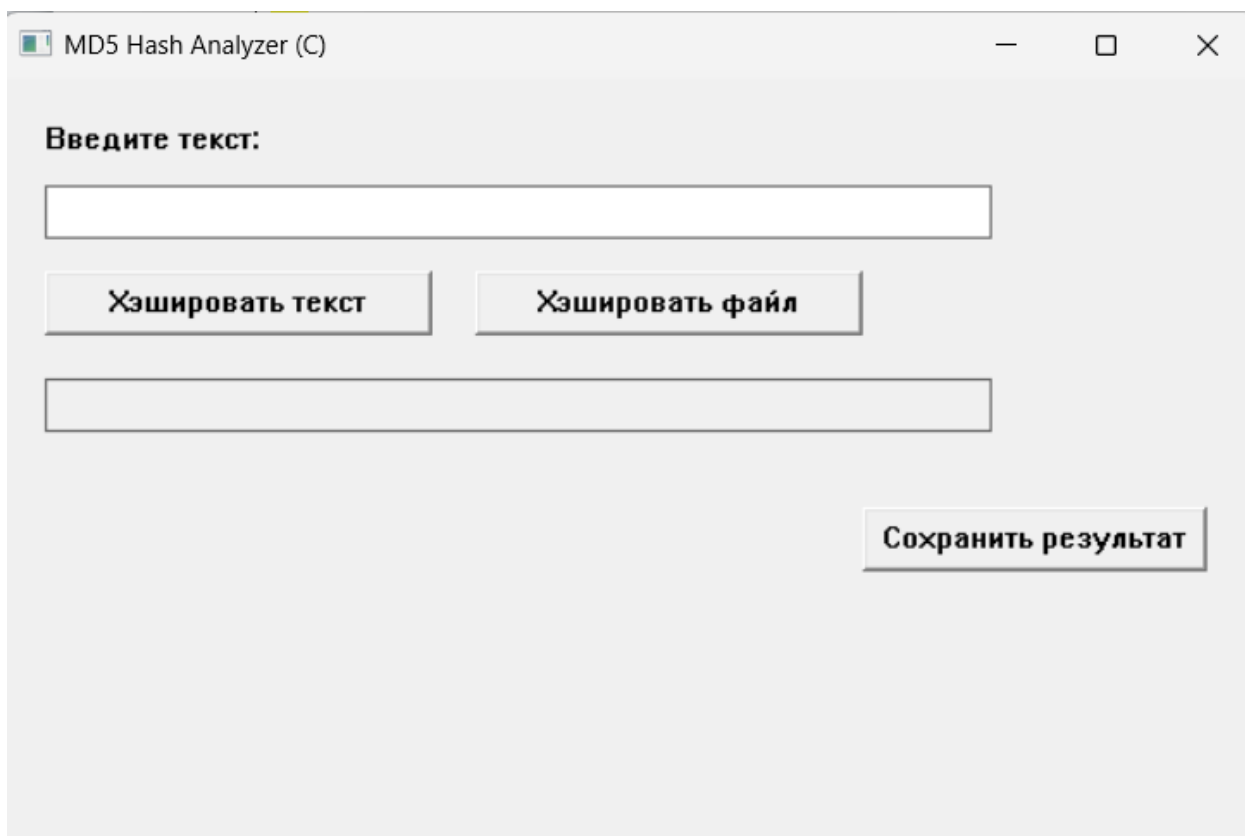


Рисунок 15. Экран для работы с хэшированием.

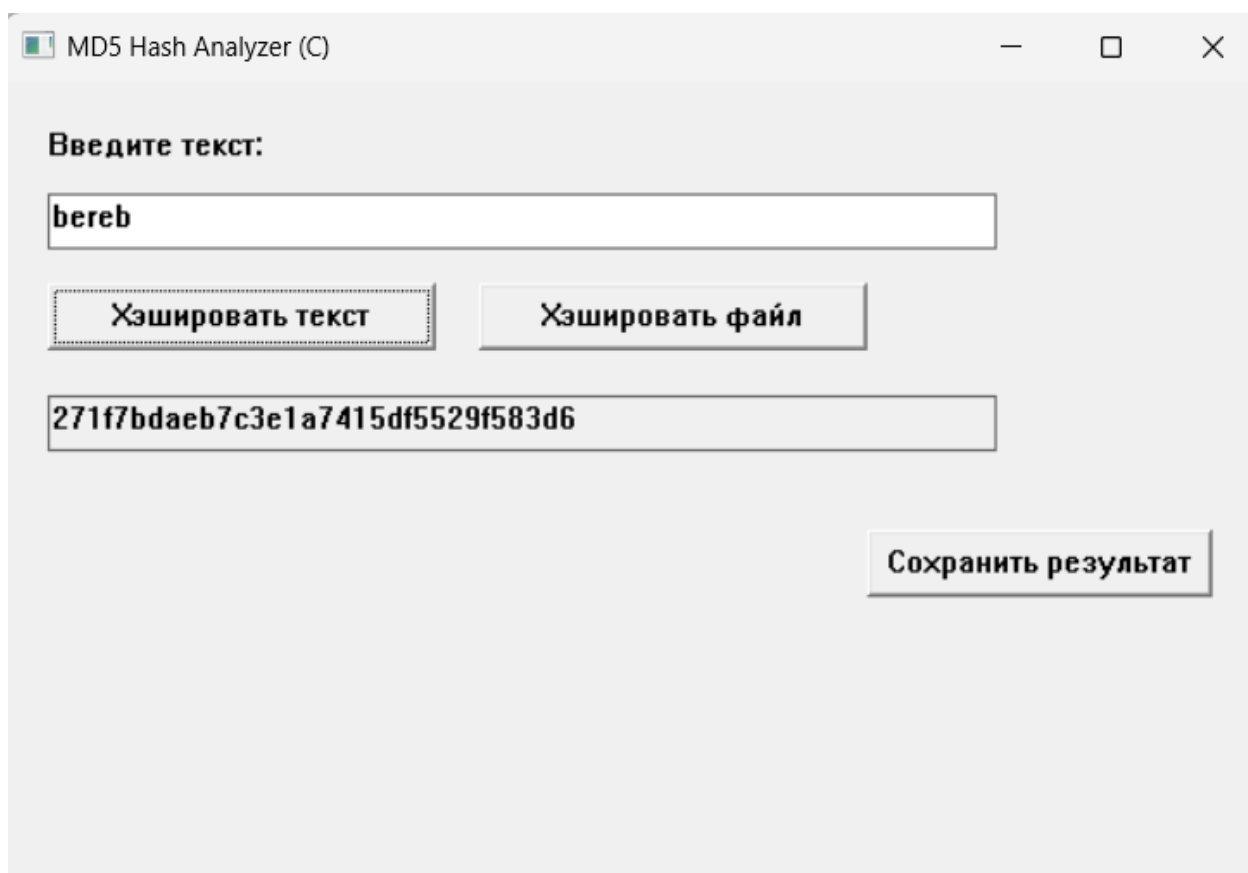


Рисунок 16. Хэширование текста.

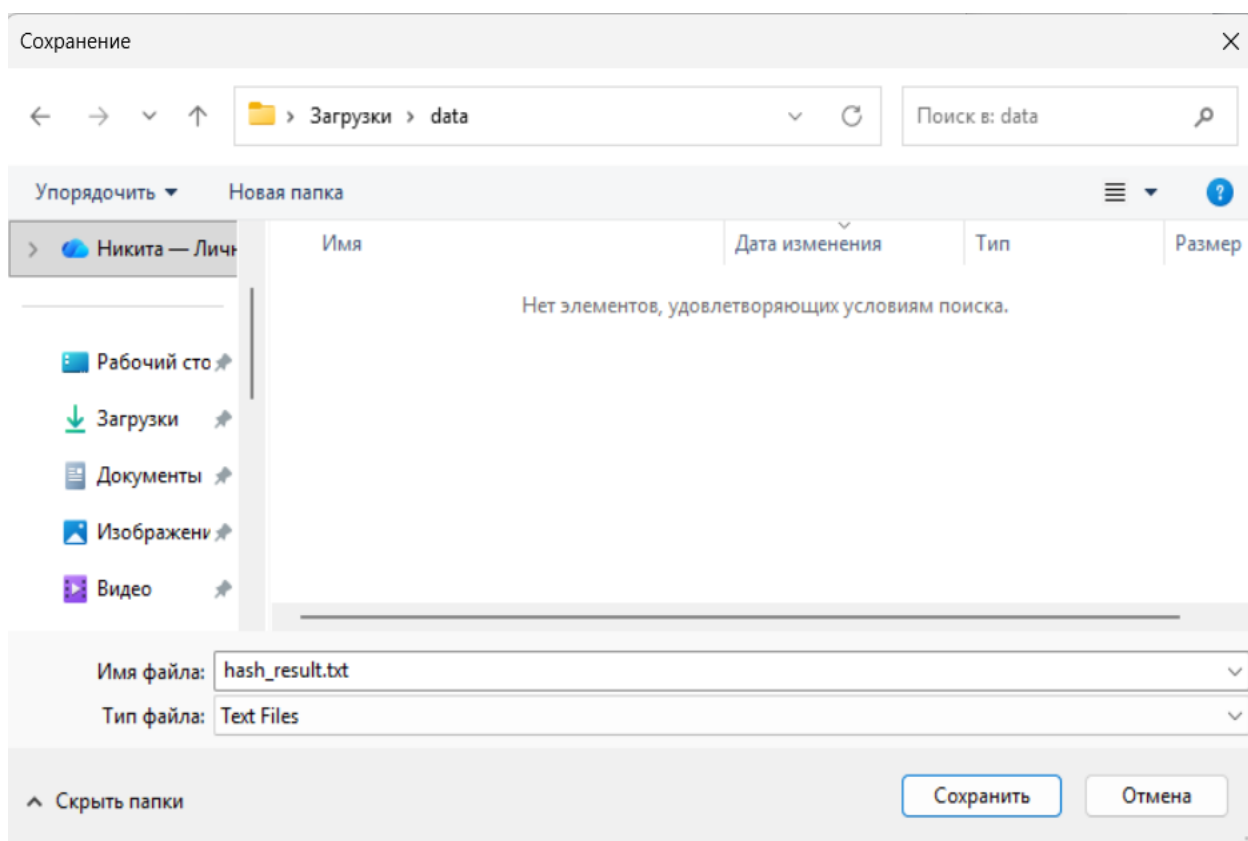


Рисунок 17. Выбор директории для записи результата.

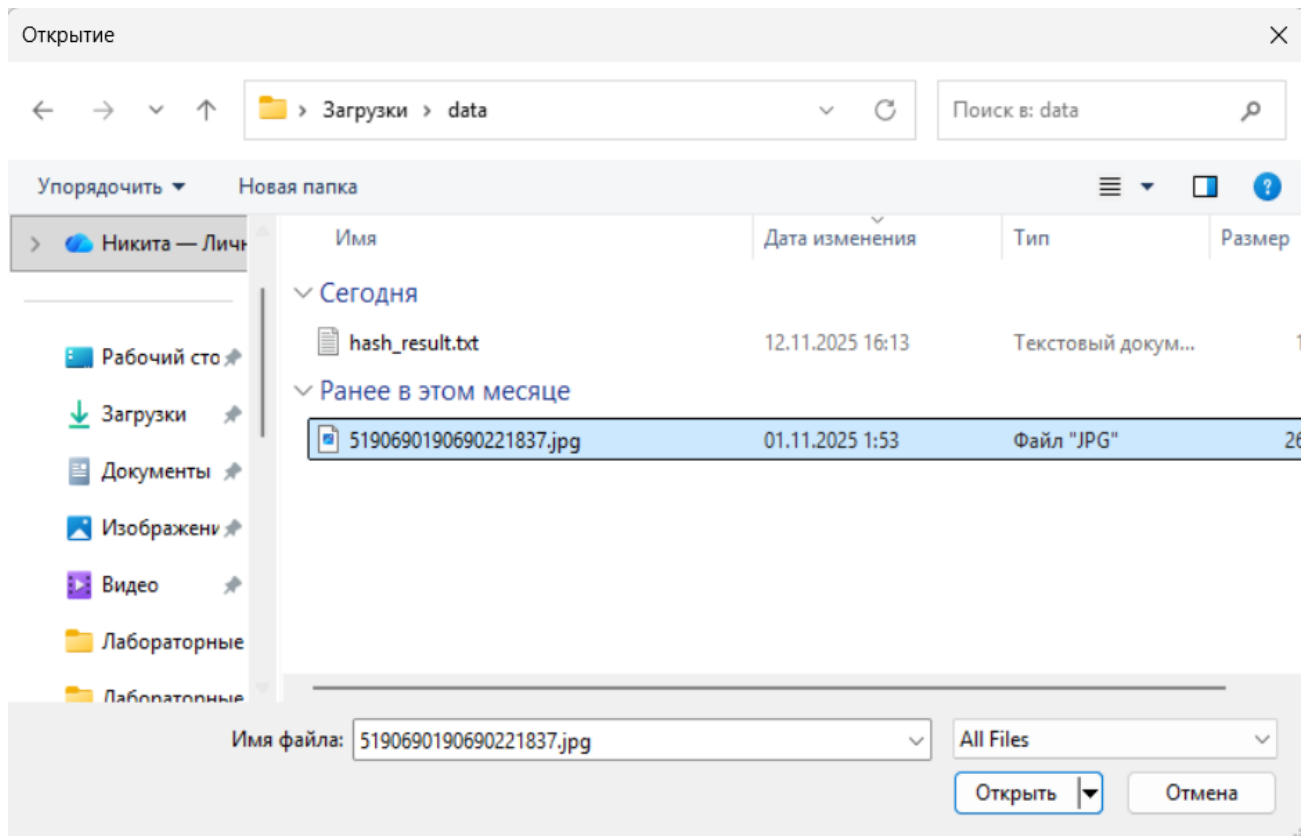


Рисунок 18. Выбор файла из директории для хэширования.

Реализация на языке C++

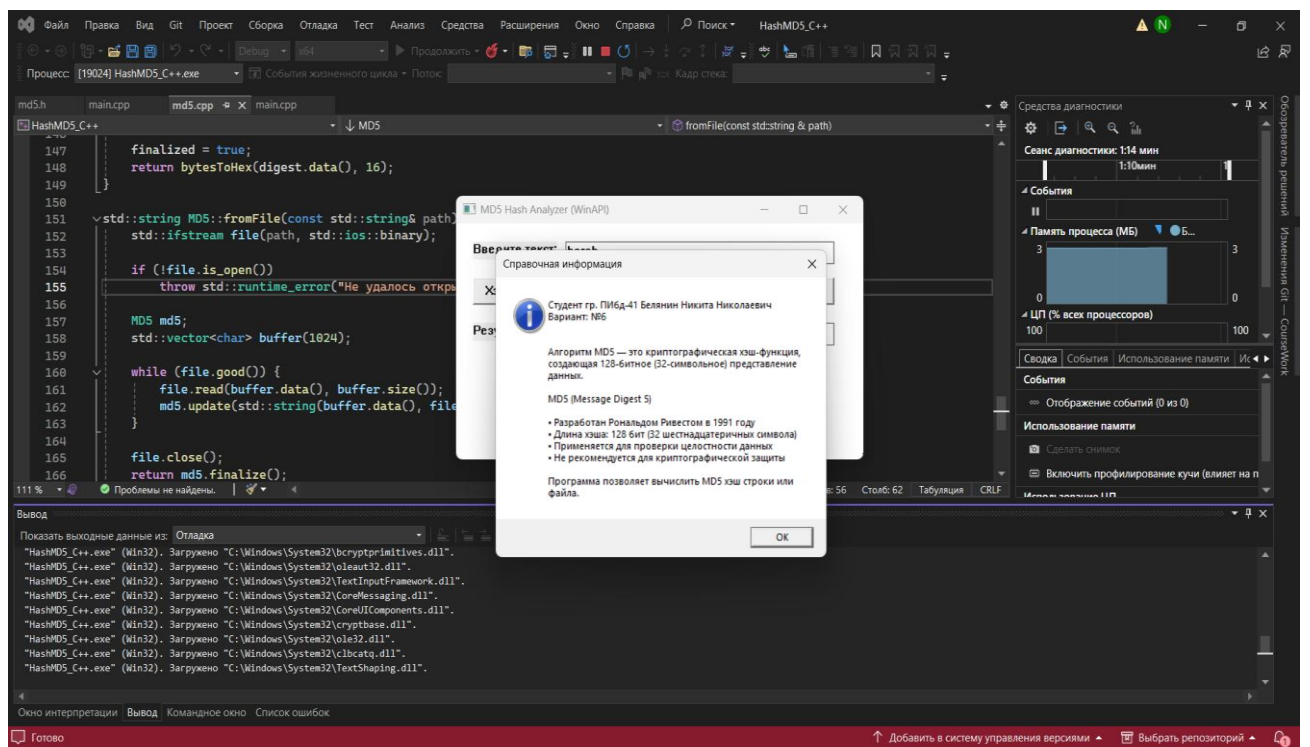


Рисунок 19. Запуск программы (C++).

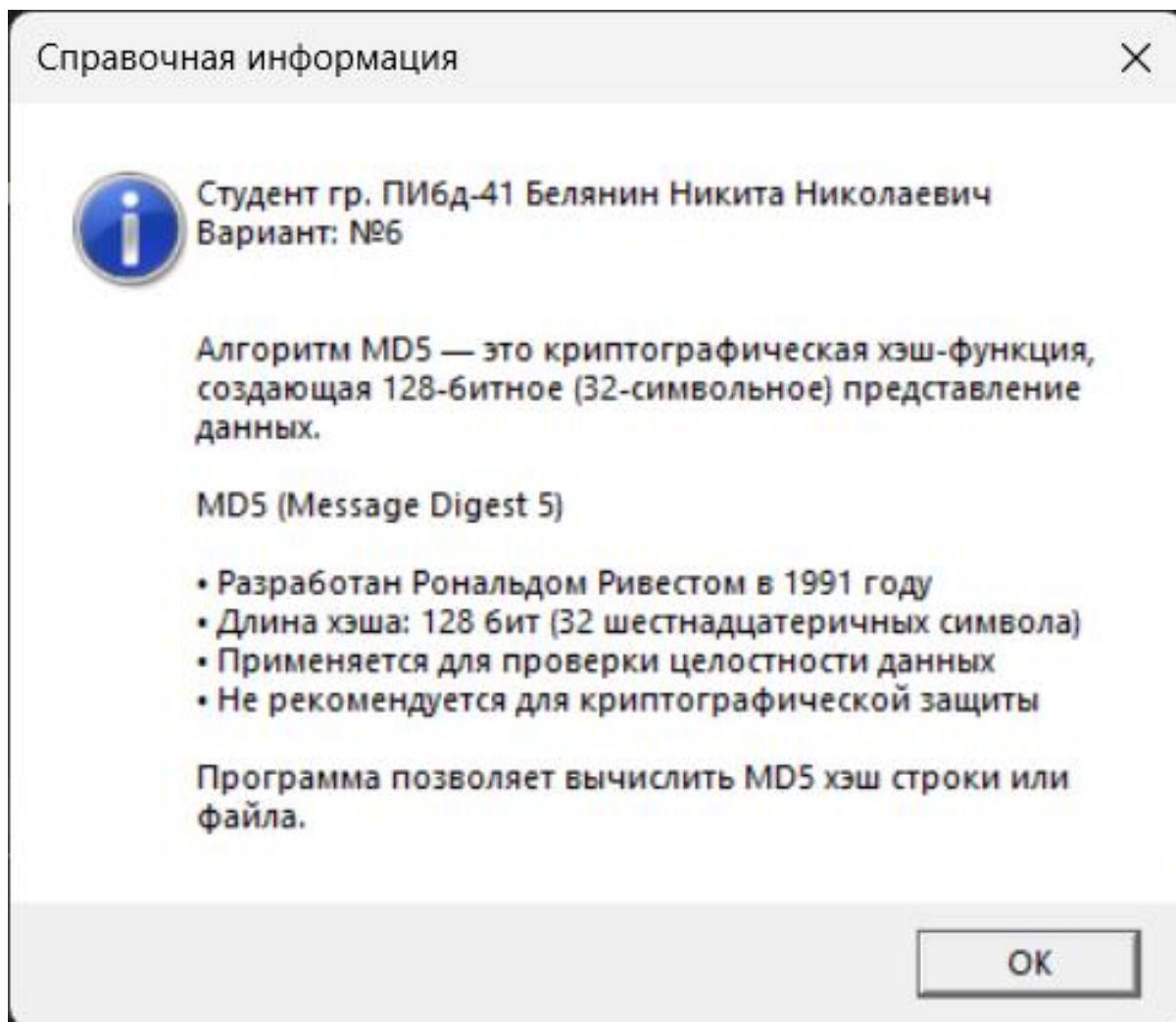


Рисунок 20. Справочная информация (C++).

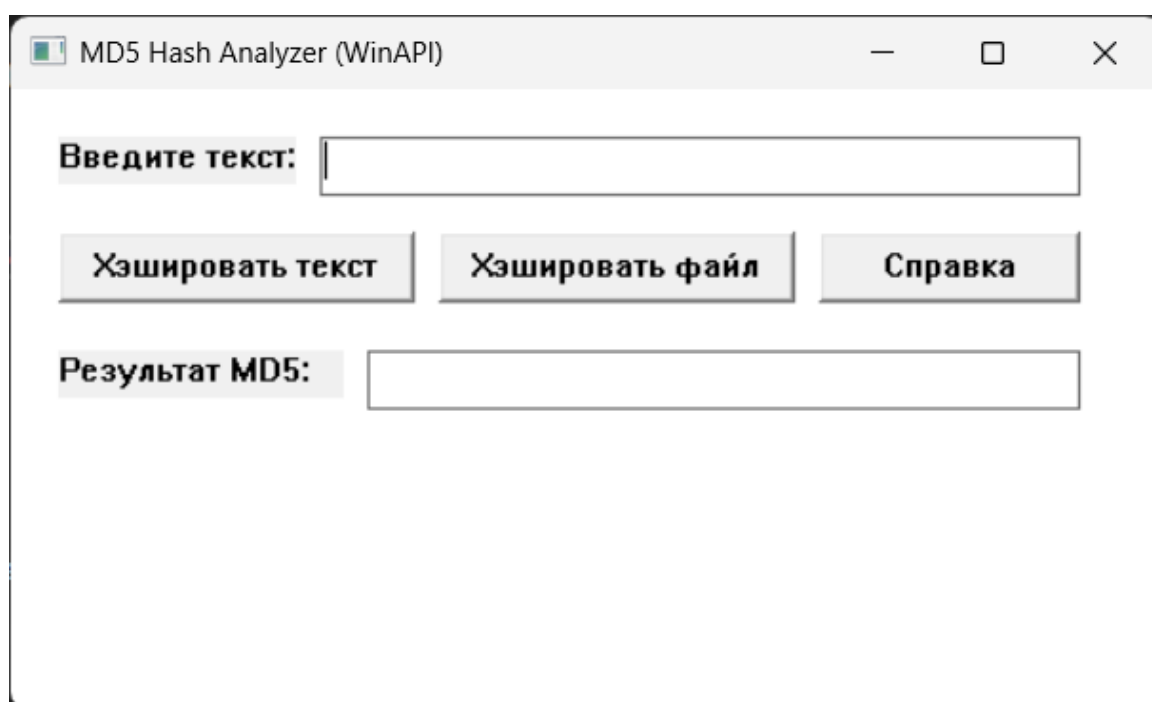


Рисунок 21. Экран хэширования (C++).

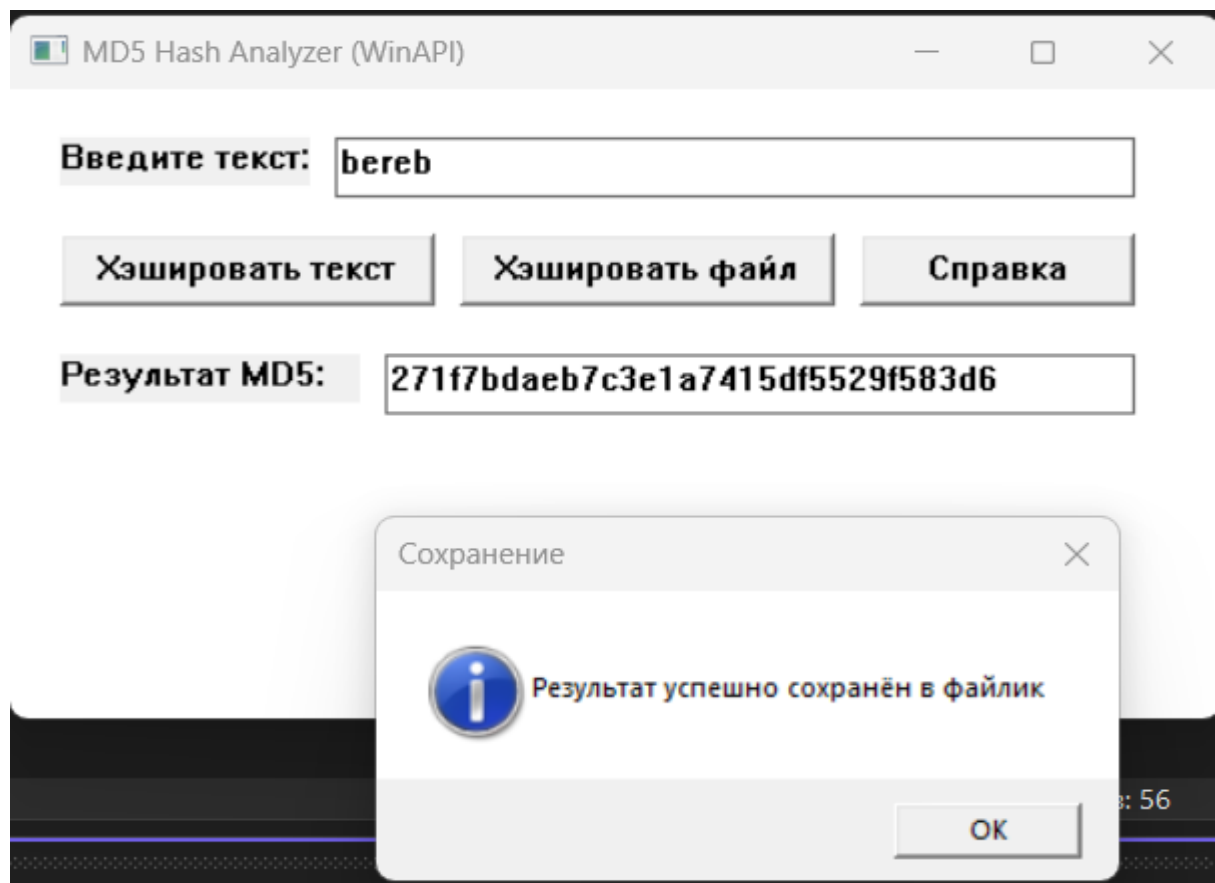


Рисунок 22. Экран хэширования (C++).

Сохранения в файл происходит при любом хэшировании автоматически! Здесь опциональной функции для сохранения не предусмотрено.

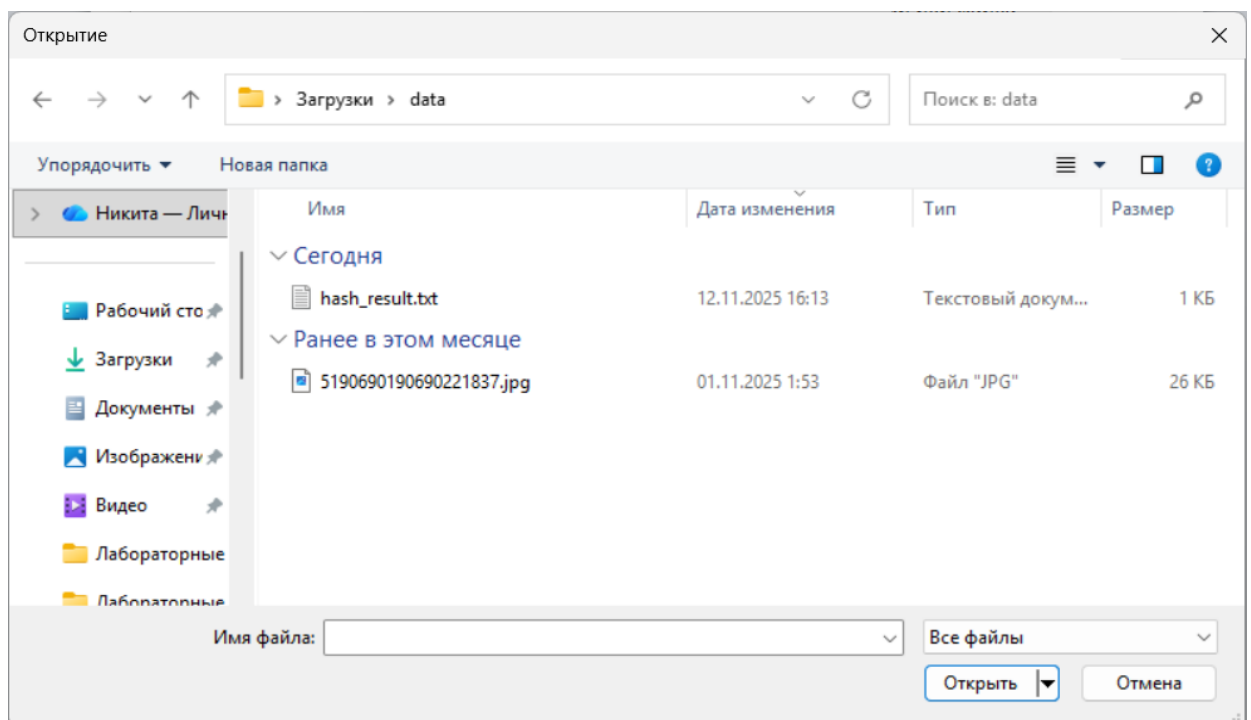


Рисунок 23. Экран выбора файла (C++).

Реализация на языке Java

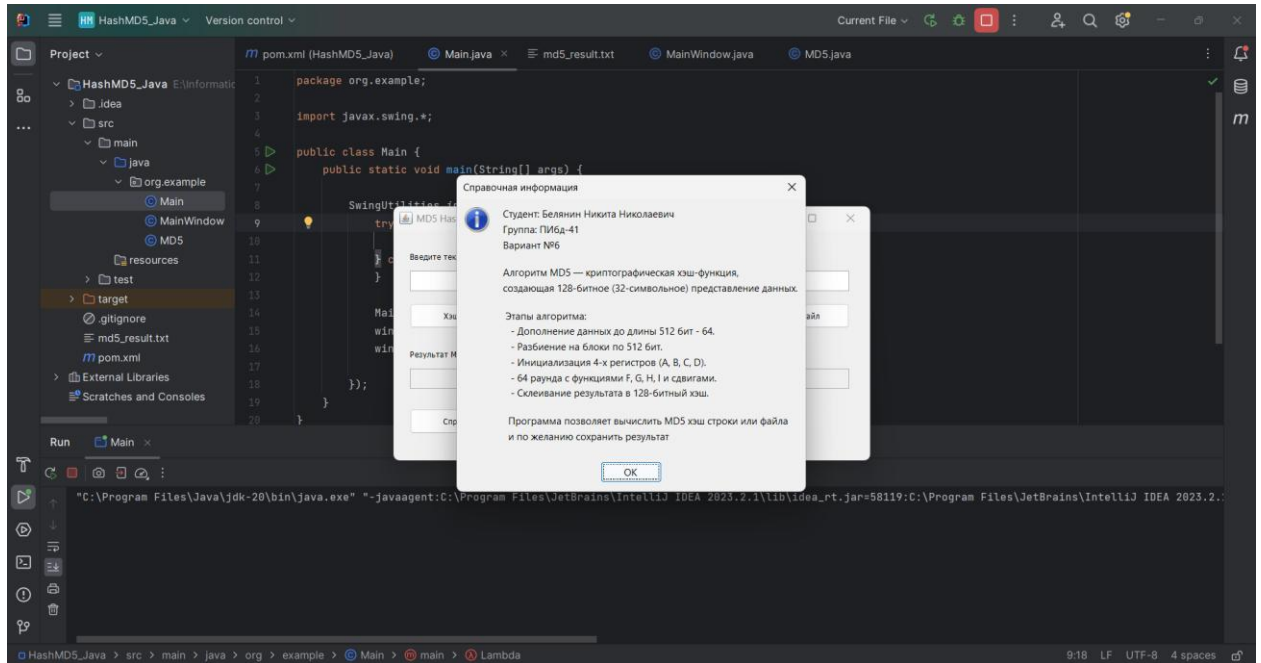


Рисунок 24. Запуск программы (Java).

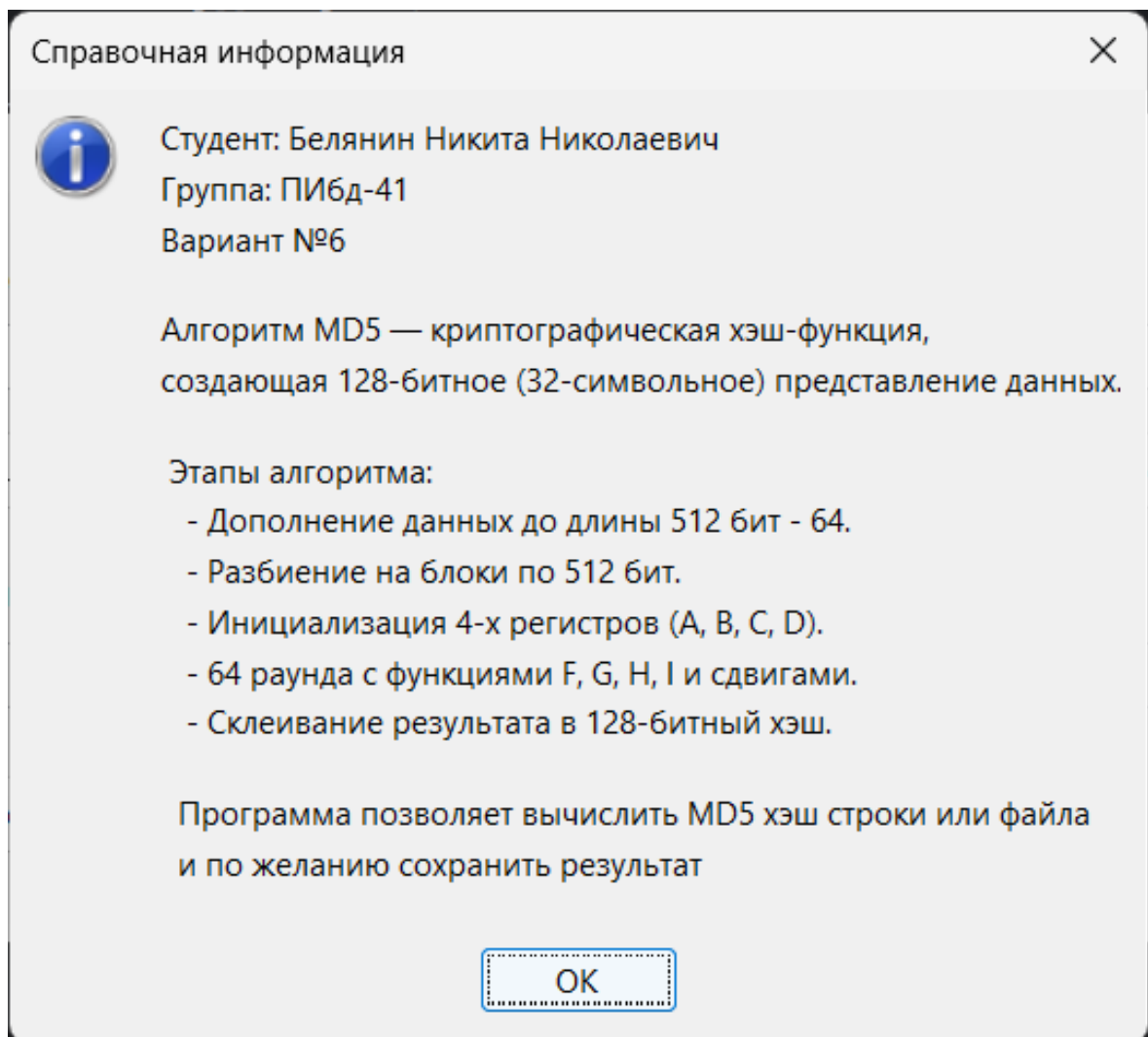


Рисунок 25. Справочная информация (Java).

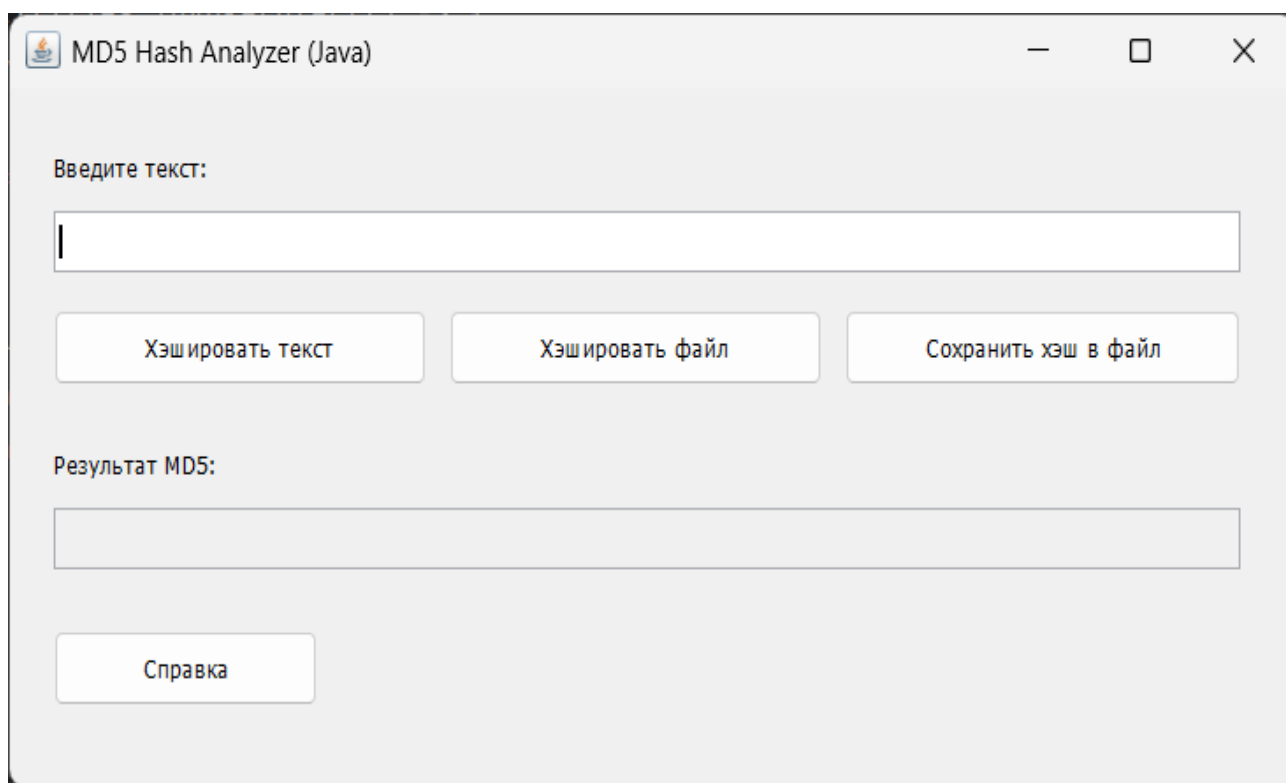


Рисунок 26. Экран работы с хэшированием (Java).

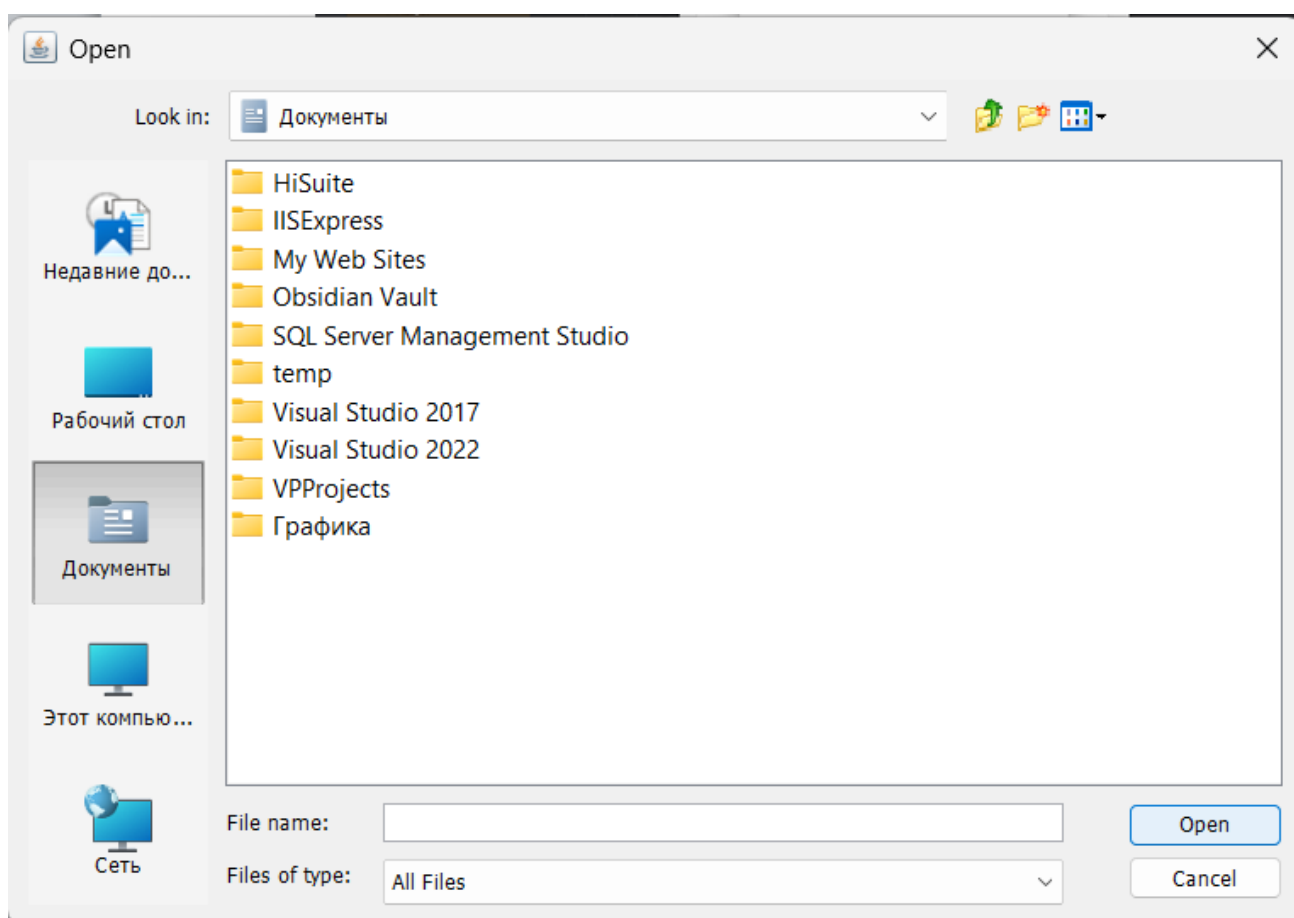


Рисунок 27. Экран выбора файла для хэширования или сохранения (Java).

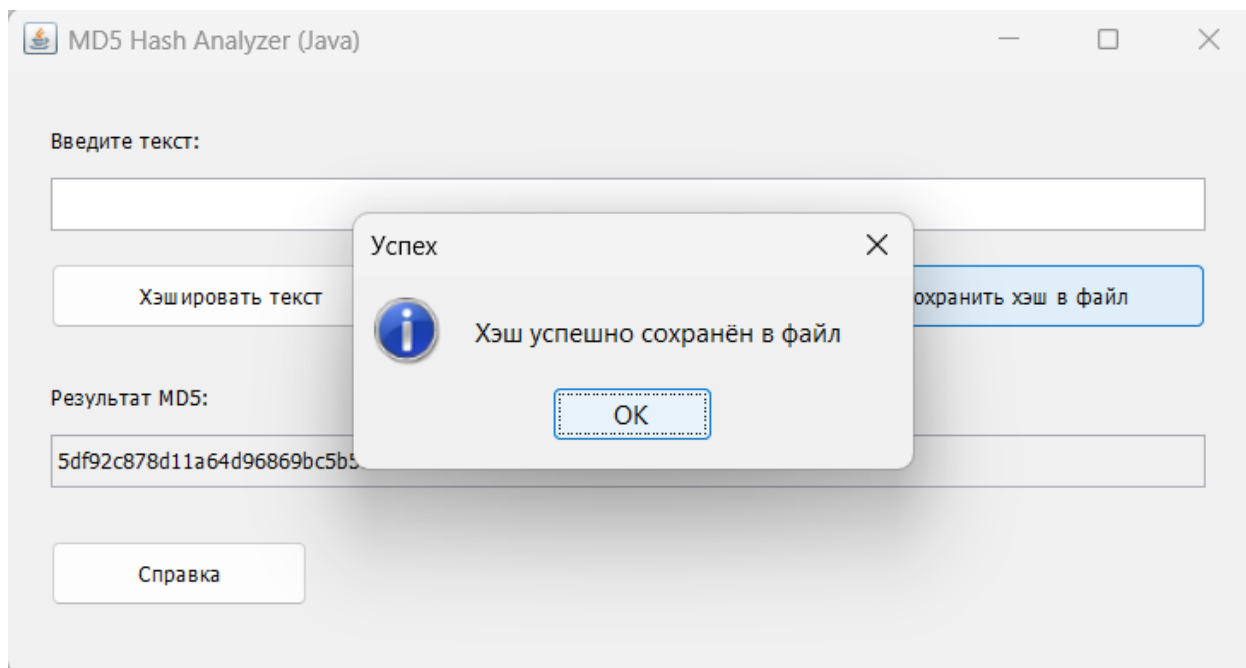


Рисунок 28. Отображение результата (Java).

Результаты хэширования

Хэшировалась строка bereb, результаты ниже:

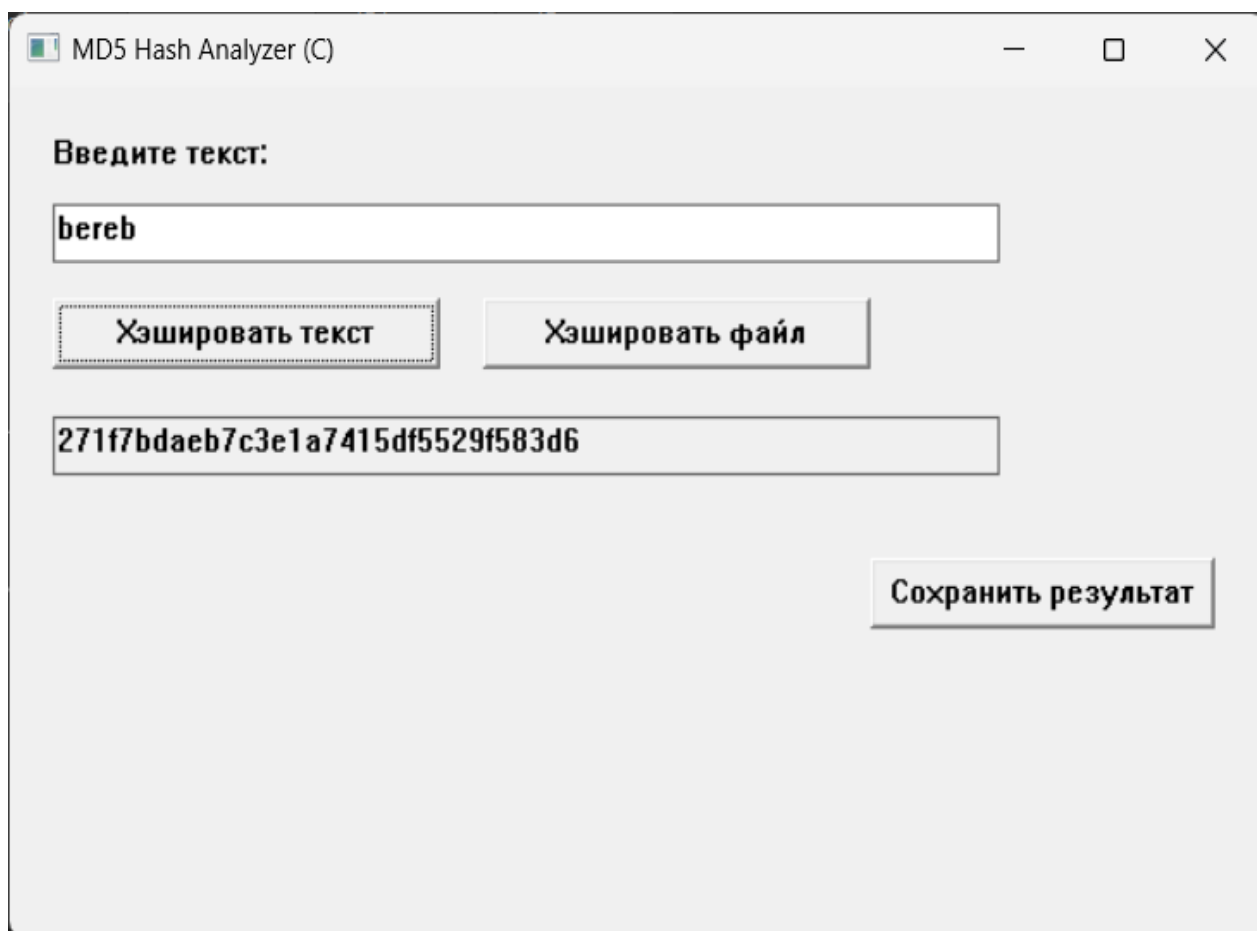


Рисунок 29. Результат хэширования строки bereb (C).

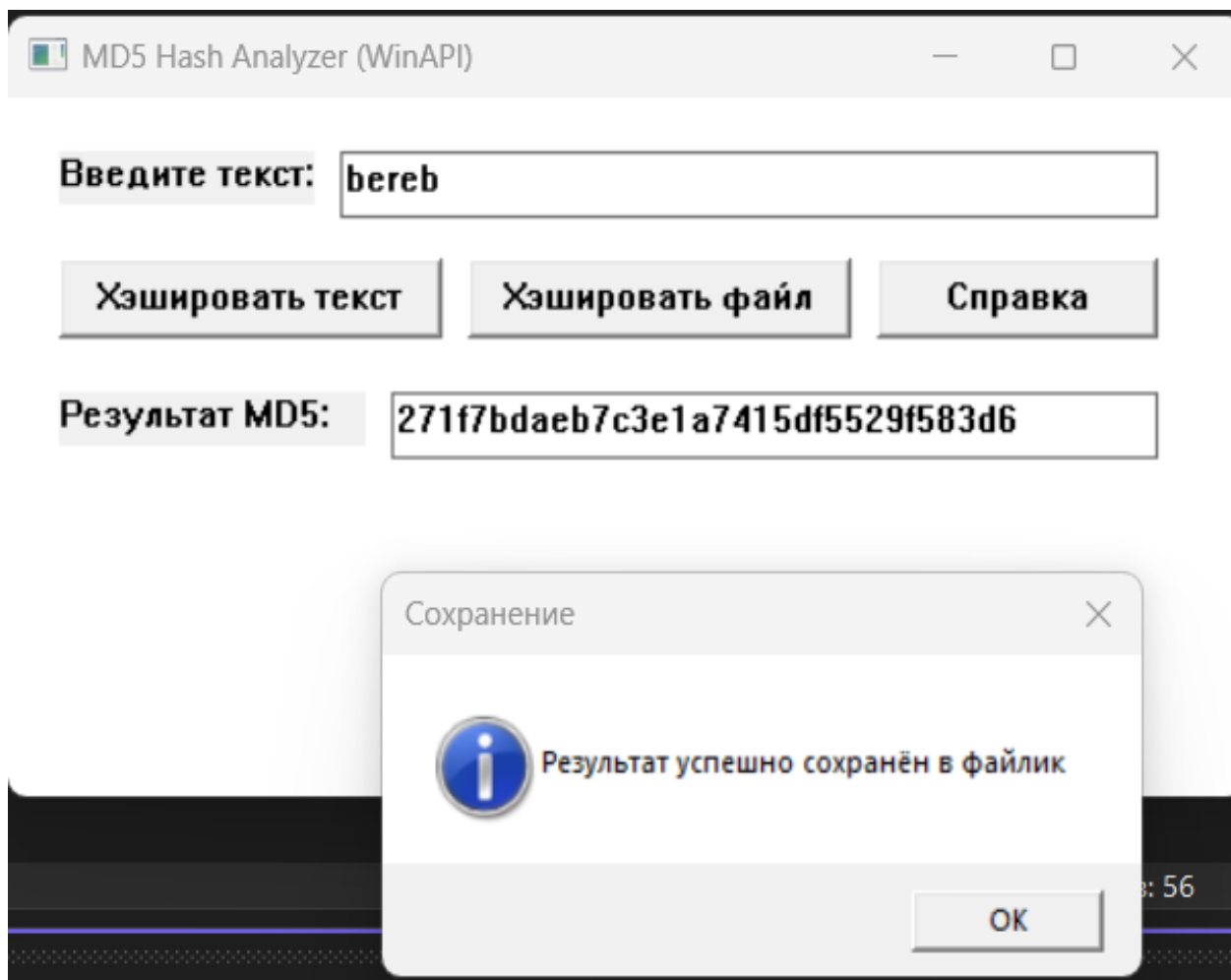


Рисунок 30. Результат хэширования строки bereb (C++).

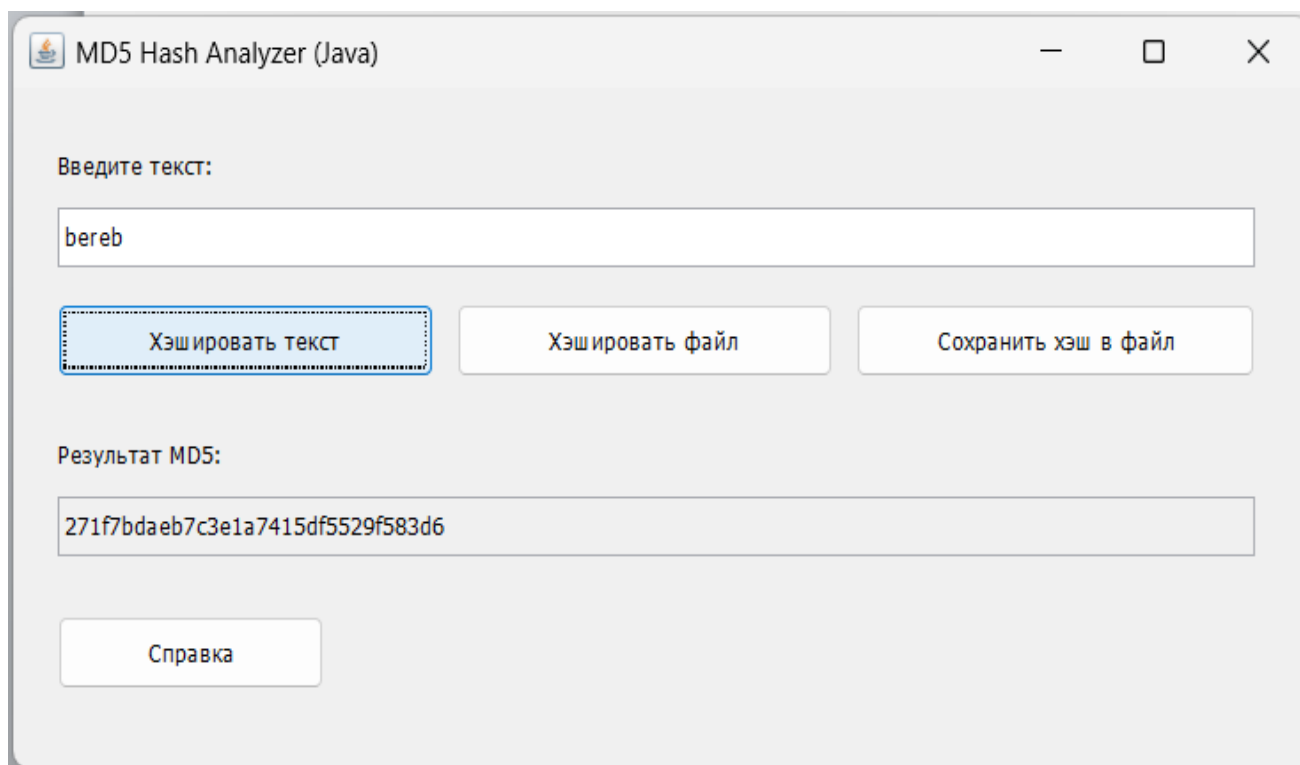


Рисунок 31. Результат хэширования строки bereb (Java).

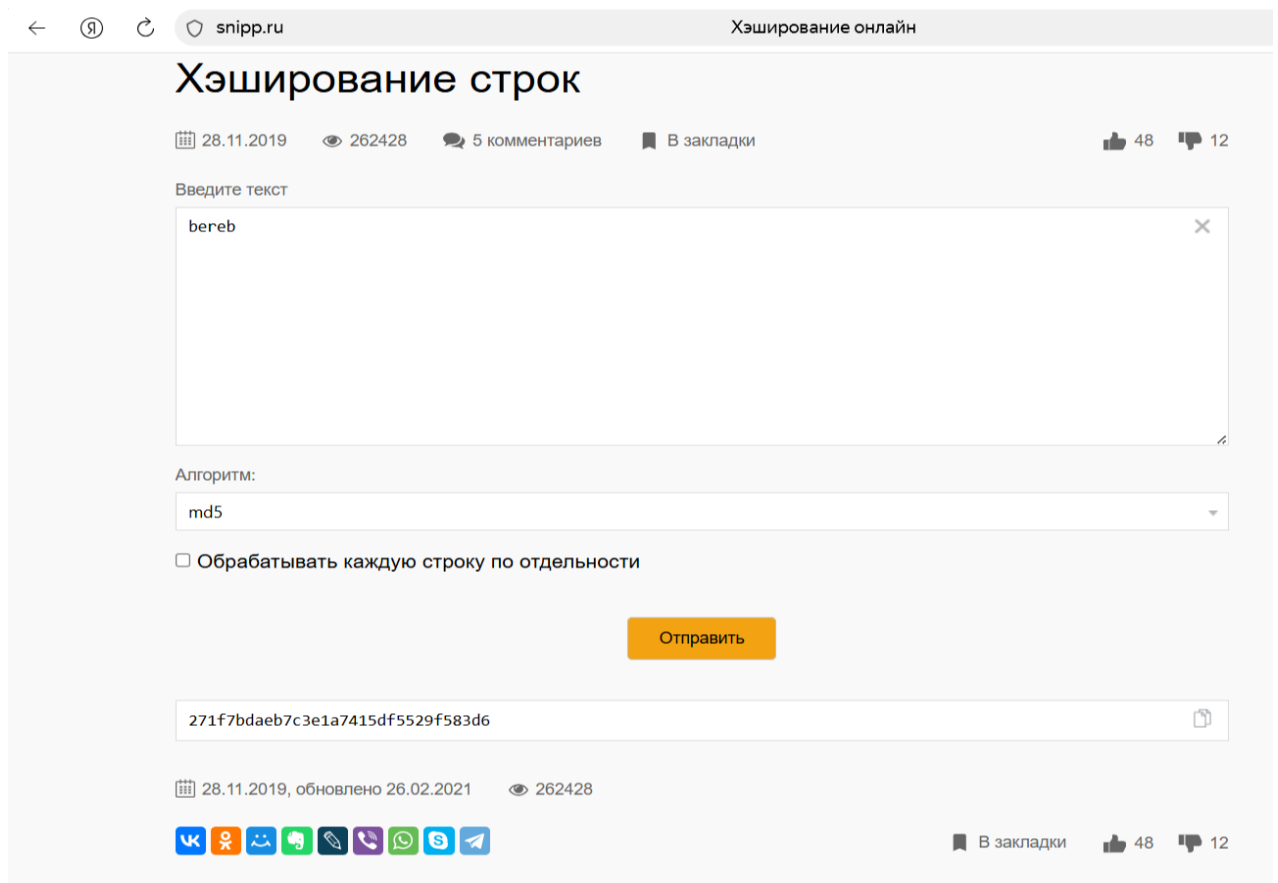


Рисунок 32. Результат хэширования строки bereb (сайт).

Хэширования файла картинки:

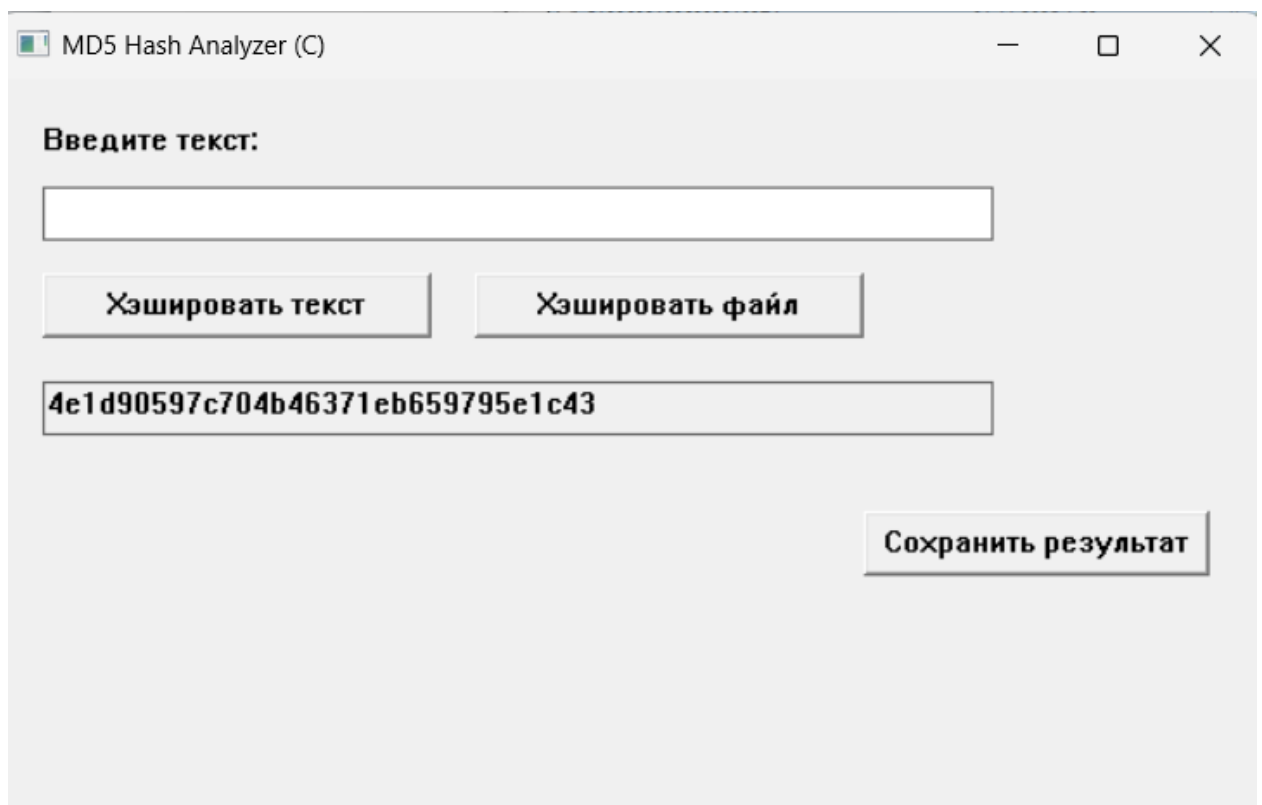


Рисунок 33. Результат хэширования файла картинки (C).

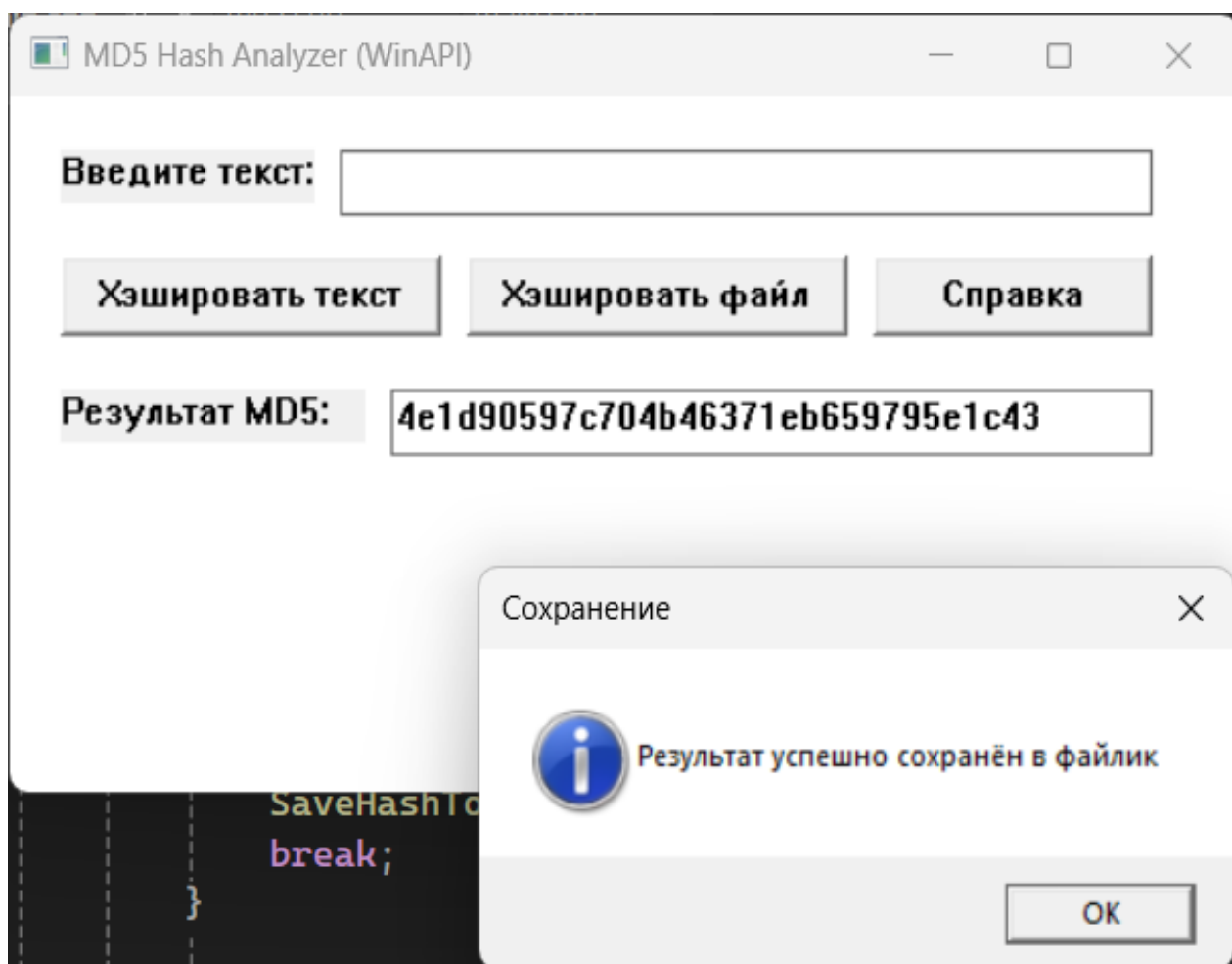


Рисунок 34. Результат хэширования файла картинки (C++).

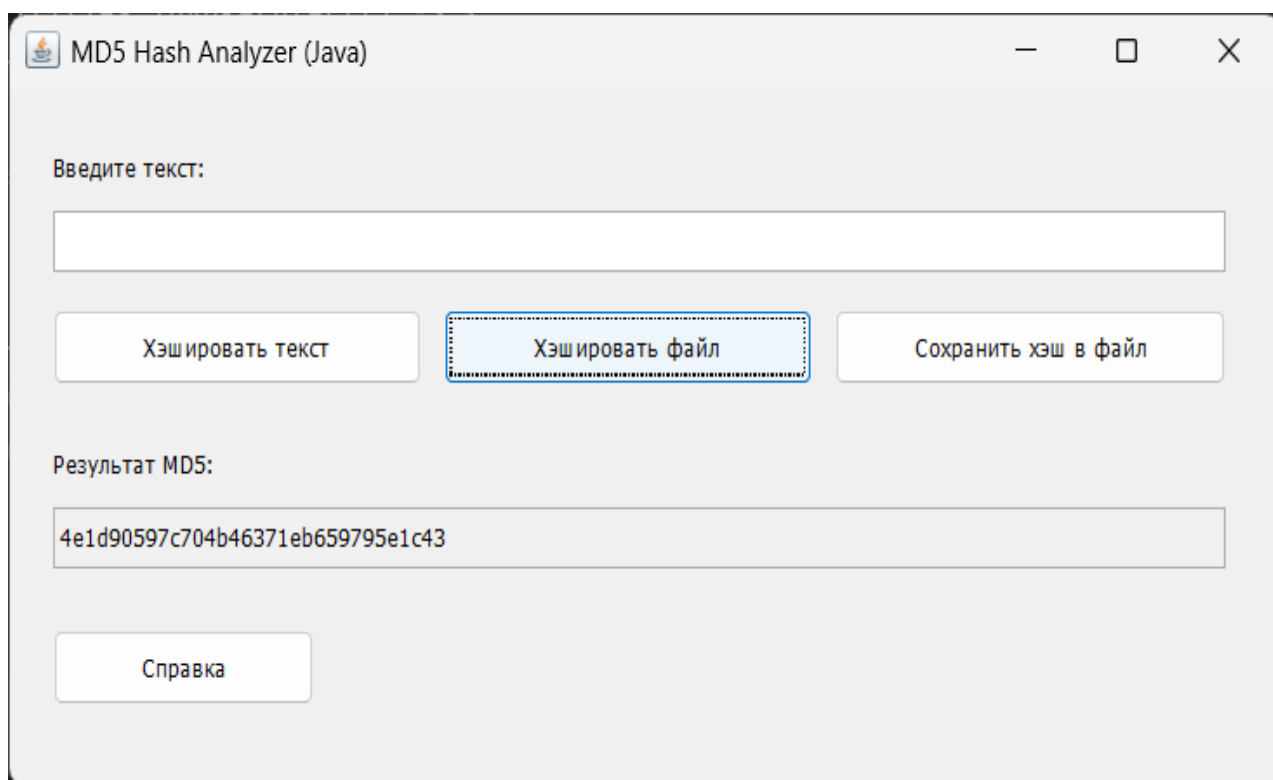


Рисунок 35. Результат хэширования файла картинки (Java).

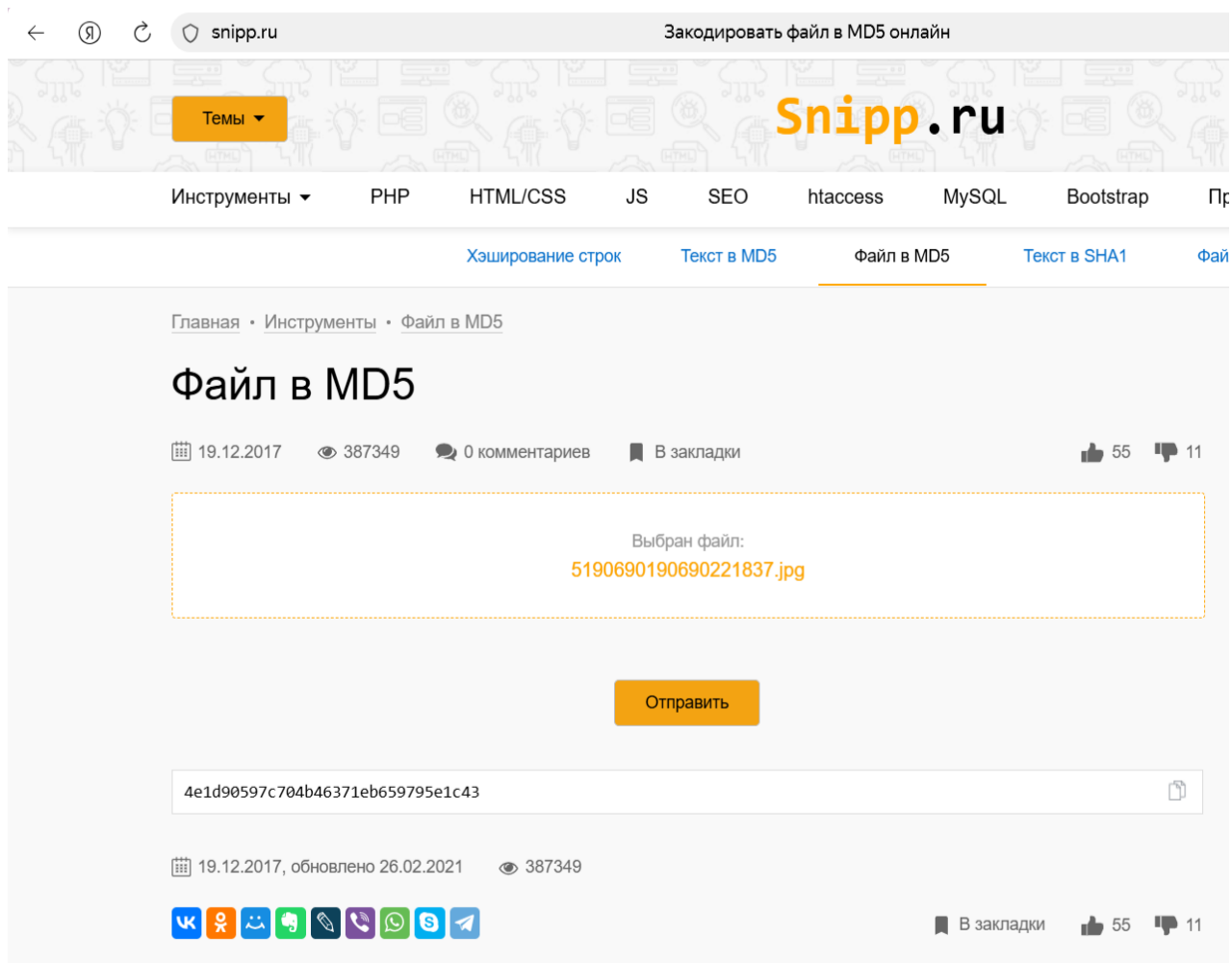


Рисунок 36. Результат хэширования файла картинки (сайт).

Ниже приведены результаты подсчётов хэшей алгоритмом MD5

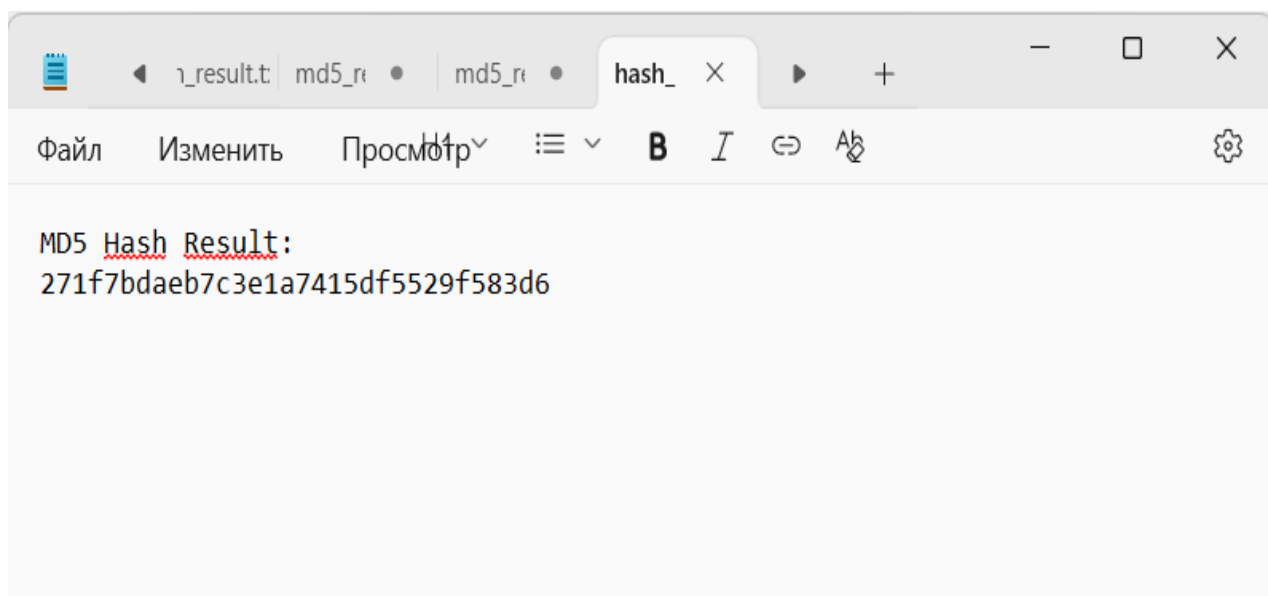
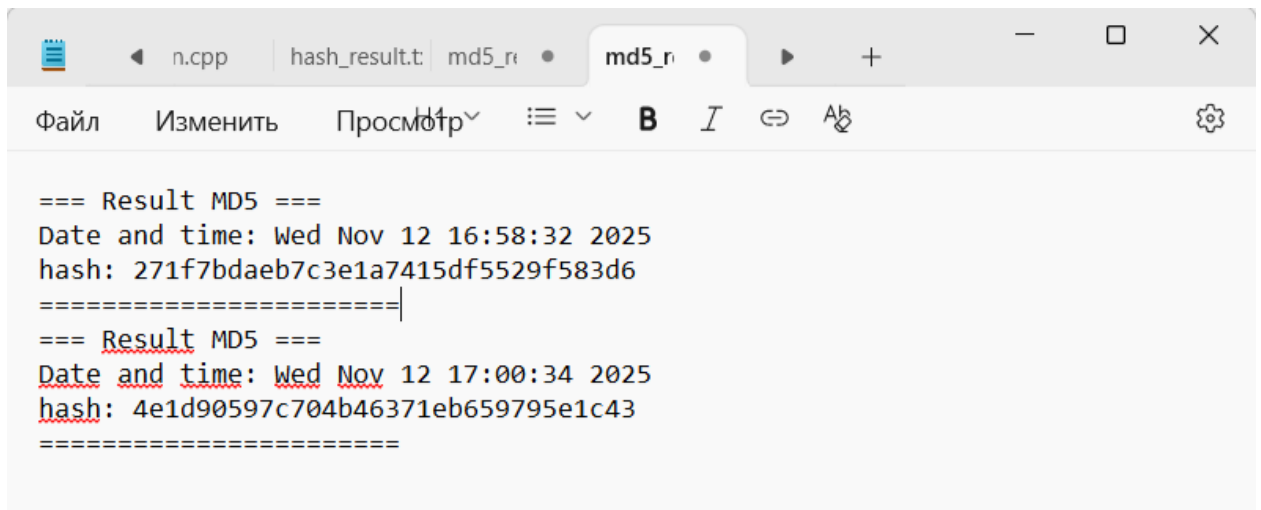


Рисунок 37. Файл с хэшами.



The image shows a text editor window with two tabs: 'n.cpp' and 'hash_result.txt'. The 'hash_result.txt' tab is active and displays the following text:

```
=== Result MD5 ===  
Date and time: Wed Nov 12 16:58:32 2025  
hash: 271f7bdaeb7c3e1a7415df5529f583d6  
=====|  
=== Result MD5 ===  
Date and time: Wed Nov 12 17:00:34 2025  
hash: 4e1d90597c704b46371eb659795e1c43  
=====
```

Рисунок 38. Файл с хэшами.

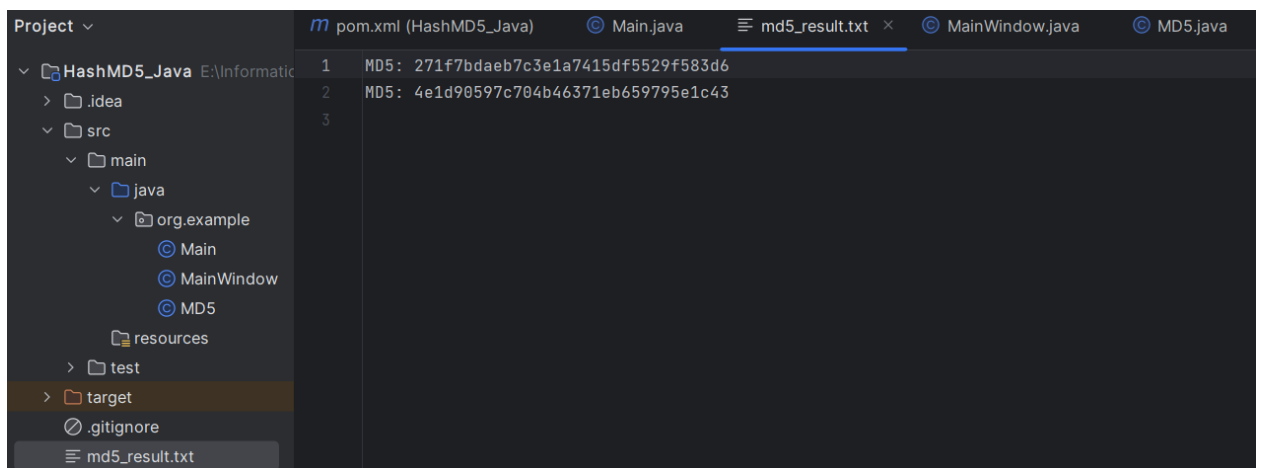


Рисунок 39. Файл с хэшами.

Расчёт одного цикла алгоритма (Трассировка)

Рассмотрим первый раунд итераций алгоритма. Для каждой из 16 операций первого раунда используется соответствующая формула, а после циклический сдвиг. В каждой итерации происходит обновление одного из 4х регистров (A, B, C, D). Вся арифметика выполняется по модулю 2^{32} (чтобы отбросить переполнение). Битовые операции $\&$, $|$, $\sim$ и сдвиги на 32-битных словах. $X[k]$ – 32 битное слово сообщения, $T[i]$ – константа, фиксирована для i . После применения формулы меняются роли регистров (циклическая перестановка).

Шаблон трассировки для каждого шага раунда (итерации 1-16):

1. Взять текущие 32-битные слова a,b,c,d (соответствующие регистрам).

2. Вычислить соответствующую формулу.
3. Вычислить временную сумму (до сдвига).
4. Вычислить сдвиг.
5. Снова обновить сумму.
6. Обновить / присвоить регистрам новое значение, и затем переход на следующий шаг с ротацией ролей.

Алгоритм MD5 оперирует с 512-битными блоками данных, которые разбиваются на 16 32-битных слов, обозначаемых как $X[0]$ до $X[15]$. Первый раунд включает в себя 16 последовательных итераций, в каждой из которых обновляется один из четырёх 32-битных регистров: A, B, C и D. Итерации выполняются по строго определённой схеме, заданной формулой:

$$a = b + ((a + F(b,c,d) + X[k] + T[i]) \lll s),$$

где a , b , c , d — текущие значения регистров, k — индекс слова из блока, s — величина циклического сдвига влево, а $T[i]$ — предварительно вычисленная константа, зависящая от номера итерации. Функция F в первом раунде определяется как $F(b,c,d) = (b \wedge c) \vee (\neg b \wedge d)$, что соответствует логическому выбору между c и d на основе значения b .

Начальные значения регистров перед началом обработки первого блока фиксированы стандартом и равны:

$$A = 0x67452301, B = 0xEFCDAB89, C = 0x98BADCFE, D = 0x10325476.$$

Для первой итерации ($j = 0$) используются параметры: $k = 0$, $s = 7$, $i = 1$, $X[0] = 0xD76AA478$, $T[1] = K[0] = 0xD76AA478$. Вычисление функции F производится над текущими значениями B, C и D:

$$F = (B \wedge C) \vee (\neg B \wedge D) = (0xEFCDAB89 \wedge 0x98BADCFE) \vee (\neg 0xEFCDAB89 \wedge 0x10325476) = 0x77777776.$$

После битовых операций результат равен $0x77777776$. Затем вычисляется промежуточное значение:

$$temp = A + F + X[0] + T[1] = 0x67452301 + 0x77777776 + 0xD76AA478 + 0xD76AA478.$$

Сумма в 32-битной арифметике с переполнением даёт: $temp = 0xA6504F67$. Далее выполняется циклический сдвиг влево на 7 бит:

$$rotate_left(temp, 7) = 0x37725FA0.$$

Новое значение регистра A вычисляется как сумма B и результата сдвига:

$$A_new = B + rotate_left(temp, 7) = 0xEFCDAB89 + 0x37725FA0 = 0x127400B29.$$

После отсечения старших битов (в 32-битной арифметике) $A_new = 0x27400B29$. Остальные регистры (B, C, D) остаются неизменными до следующих итераций. После завершения этой итерации регистры принимают значения:

$$A = 0x27400B29, B = 0xEFCDAB89,$$

$$C = 0x98BADCFE, D = 0x10325476.$$

Процесс повторяется для оставшихся 15 итераций первого раунда, при этом в каждой итерации обновляется один регистр в порядке, заданном схемой [ABCD], [DABC], [CDAB], [BCDA], и так далее. Индекс k для выбора слова $X[k]$ в каждой итерации определяется функцией $g(j)$, а сдвиг s и константа $T[i]$ выбираются из заранее заданных таблиц. Вторая итерация использует $k = 1$, $s = 12$, $i = 2$, и обновляет регистр D, третья — обновляет C, и т.д., обеспечивая равномерное распределение влияния всех слов блока на все регистры.

Теперь надо показать, как именно происходит ротация регистров (т. е. кто станет a,b,c,d в следующем шаге). Стандартная последовательность — после исполнения формулы меняют местами роли: для следующего шага обычно берут $(a,b,c,d) \leftarrow (d, a, b, c)$ либо, эквивалентно, в записи вашего списка: шаг 2 использует [DABC ...]. В шаге 2 равен старому D, b — старому A (который в шаге 1 уже был обновлён).

<i>Итерация j</i>	<i>Порядок</i>	<i>k</i>	<i>s</i>	<i>i</i>	<i>X[k]</i> (шестнадцатеричный)	<i>T[i]=K[j]</i> (шестнадцатеричный)
0	ABCD	0	7	1	0xD76AA478	0xD76AA478
1	DABC	1	12	2	0xE8C7B756	0xE8C7B756
2	CDAB	2	17	3	0x242070DB	0x242070DB
3	BCDA	3	22	4	0xC1BDCCEE	0xC1BDCCEE
4	ABCD	4	7	5	0xF57C0FAF	0xF57C0FAF
5	DABC	5	12	6	0x4787C62A	0x4787C62A
6	CDAB	6	17	7	0xA8304613	0xA8304613
7	BCDA	7	22	8	0xFD469501	0xFD469501
8	ABCD	8	7	9	0x698098D8	0x698098D8
9	DABC	9	12	10	0x8B44F7AF	0x8B44F7AF
10	CDAB	10	17	11	0xFFFFF5BB1	0xFFFFF5BB1
11	BCDA	11	22	12	0x895CD7BE	0x895CD7BE
12	ABCD	12	7	13	0x6B901122	0x6B901122
13	DABC	13	12	14	0xFD987193	0xFD987193
14	CDAB	14	17	15	0xA679438E	0xA679438E
15	BCDA	15	22	16	0x49B40821	0x49B40821

Первый раунд MD5 представляет собой детерминированную последовательность нелинейных преобразований, в которых каждое из 16

слов входного блока влияет на все 4 регистра через комбинацию битовых операций, сложения и циклических сдвигов. Точность вычислений обеспечивается строгим соблюдением порядка обновления регистров, выбора констант и арифметики с переполнением. Результат раунда служит основой для последующих трёх раундов, каждый из которых использует свою уникальную функцию F, но сохраняет ту же структуру вычислений.

Такой подход раньше давал высокую диффузию данных¹⁰ и являлся фундаментом криптографической стойкости алгоритма. Однако уже в 2004 году были найдены практические коллизии, позволяющие намеренно создавать два разных сообщения с одинаковым MD5-хэшем за считанные секунды на обычном компьютере. Сейчас рекомендуется использовать алгоритм MD5 в следующих случаях:

- проверки случайных повреждений данных (например, при скачивании файлов, если нет риска злонамеренной подмены),
- не критичных к безопасности задач (например, простое дедуплицирование файлов¹¹, при котором сохраняется только одна копия уникальных данных, а все остальные ссылки на дубликаты заменяются на указатели (ссылки) на эту единственную копию).

¹⁰ **Диффузия данных** — это свойство криптографической функции, при котором малейшее изменение входных данных (например, одного бита) вызывает значительное и непредсказуемое изменение выходного хэша (в идеале — каждый бит хэша меняется с вероятностью 50%).

¹¹ **Дедуплицирование файлов** - процесс выявления и удаления дубликатов файлов или их фрагментов в системе хранения данных с целью экономии дискового пространства.

Трассировка 1 раунда

j	Схема	k	s	i	B (до)	C (до)	D (до)	F(B,C,D) формула	temp = A+F+X+T	rotate(temp,s) сдвиг	Reg (до) регистр	Reg (новый) регистр
0	ABCD	0	7	1	0xEFCDAB89	0x98BADCFE	0x10325476	0x77777776	0xED8CD97F	0x37725FA0	0xEFCDAB89	0x27400B29
1	DABC	1	12	2	0x98BADCFE	0x10325476	0x27400B29	0x10325476	0x3D661B70	0x5A827999	0x98BADCFE	0xF33C88D7
2	CDAB	2	17	3	0x10325476	0x27400B29	0xF33C88D7	0x27400B29	0x3A827999	0x5A827999	0x10325476	0x6A9E61C9
3	BCDA	3	22	4	0x27400B29	0xF33C88D7	0x6A9E61C9	0xF33C88D7	0x5A827999	0x5A827999	0x27400B29	0x8D962A0E
4	ABCD	4	7	5	0x8D962A0E	0x6A9E61C9	0xF33C88D7	0x6A9E61C9	0x5A827999	0x37725FA0	0x8D962A0E	0xC512F1A7
5	DABC	5	12	6	0x6A9E61C9	0xF33C88D7	0xC512F1A7	0xF33C88D7	0x3D661B70	0x5A827999	0x6A9E61C9	0x2828A286
6	CDAB	6	17	7	0xF33C88D7	0xC512F1A7	0x2828A286	0xC512F1A7	0x3A827999	0x5A827999	0xF33C88D7	0x9B4A0790
7	BCDA	7	22	8	0xC512F1A7	0x2828A286	0x9B4A0790	0x2828A286	0x5A827999	0x5A827999	0xC512F1A7	0x1E25D3D4
8	ABCD	8	7	9	0x1E25D3D4	0x9B4A0790	0x2828A286	0x9B4A0790	0x37725FA0	0x37725FA0	0x1E25D3D4	0x5A827999
9	DABC	9	12	10	0x9B4A0790	0x2828A286	0x5A827999	0x2828A286	0x3D661B70	0x5A827999	0x9B4A0790	0x2828A286
10	CDAB	10	17	11	0x2828A286	0x5A827999	0x2828A286	0x5A827999	0x3A827999	0x5A827999	0x2828A286	0x6ED9EBA1
11	BCDA	11	22	12	0x5A827999	0x2828A286	0x6ED9EBA1	0x2828A286	0x5A827999	0x5A827999	0x5A827999	0x8F1BBCDC
12	ABCD	12	7	13	0x8F1BBCDC	0x6ED9EBA1	0x2828A286	0x6ED9EBA1	0x37725FA0	0x37725FA0	0x8F1BBCDC	0x6D703EF3
13	DABC	13	12	14	0x6ED9EBA1	0x2828A286	0x6D703EF3	0x2828A286	0x3D661B70	0x5A827999	0x6ED9EBA1	0x4787C62A
14	CDAB	14	17	15	0x2828A286	0x6D703EF3	0x4787C62A	0x6D703EF3	0x3A827999	0x5A827999	0x2828A286	0xF429B02B
15	BCDA	15	22	16	0x6D703EF3	0x4787C62A	0xF429B02B	0x4787C62A	0x5A827999	0x5A827999	0x6D703EF3	0x105E5C30

Заключение

В ходе выполнения лабораторной работы был реализован, исследован и подробно изучен алгоритм хэширования MD5 (Message Digest 5), представляющий собой один из наиболее известных и исторически значимых алгоритмов криптографических хэш-функций. Реализация алгоритма позволила глубже понять процесс формирования дайджеста сообщения, продемонстрировала особенности побитовых операций и перестановок, циклических сдвигов и принципы работы с блочной структурой данных. Также были рассмотрены логические функции, лежащие в основе MD5. Программа корректно обрабатывает файлы различных форматов и выводит хэш в шестнадцатеричном виде.

Как отмечалось ранее, основная задача любой хэш-функции сообщений производить образы, которые можно считать относительно случайными. Чтобы считаться криптографически безопасной, хэш-функция должна отвечать двум основным требованиям. Во-первых, злоумышленник не может сгенерировать сообщение, соответствующее определённому хэш-значению. Во-вторых, невозможно создать два сообщения, которые производят одно и то же значение (коллизии в MD5).

Однако, анализ свойств алгоритма показал, что MD5 не удовлетворяет современным требованиям криптографической устойчивости. Исследования последних лет выявили возможность подбора коллизий, что делает невозможным использование MD5 в системах, требующих строгой защиты информации (например, при хранении паролей, цифровой подписи, шифровании и аутентификации). Алгоритм MD5 не отвечает требованиям IETF (Internet Engineering Task Force) рекомендовала новым проектам протоколов не использовать данную хэш-функцию, так как исследовательские атаки предоставили достаточные основания для исключения из списка используемых методов.

Хэши MD5 больше не считаются безопасными, и их не рекомендовано использовать для криптографической аутентификации.

Сегодня, согласно рекомендациям IETF, использование MD5 в новых протоколах не допускается. Ему на смену пришли более надёжные алгоритмы — это SHA-256/384/512, SHA-3, RIPEMD-160, Whirlpool и специализированные схемы (HMAC, Argon2, bcrypt, scrypt), обеспечивающие повышенную стойкость к коллизиям и атакам.

Если вы использовали MD5 в старом проекте, планируйте миграцию: храните указание алгоритма и пересчитайте хэши при первой успешной аутентификации пользователя (реплицируйте с безопасным алгоритмом).

Для хранения паролей — *соль + медленный KDF* (Argon2 рекомендуется).

Для целостности больших файлов — *SHA-256 и подписи*.

- **SHA-256** (256-битный алгоритм безопасного хэширования) является частью семейства SHA-2 и генерирует 256-битное хэш-значение. Он широко используется в блокчейне, SSL-сертификатах, цифровых подписях и сертификатах.

- **SHA-3** представляет собой альтернативу SHA-2 с другой внутренней структурой, что делает его более устойчивым к определённым атакам. Хотя SHA-2 по-прежнему более распространён, SHA-3 используется в безопасных приложениях, требующих соблюдения самых высоких криптографических стандартов.

- **RIPEMD-160** генерирует 160-битное хэш-значение, обеспечивая более высокий уровень безопасности, чем MD5, но не так широко распространён, как SHA-2. Иногда используется в цифровых подписях и сертификатах в определённых системах безопасности.

- **Whirlpool** — это криптографическая хэш-функция, обеспечивающая высокий уровень безопасности и выдающая 512-битный хэш-результат. Она

используется в тех случаях, когда требуется очень высокий уровень безопасности.

Преимущества алгоритма MD5

1. MD5 работает быстрее и проще для понимания.
2. Для интеграции алгоритма MD5 требуется относительно небольшой объём памяти.
3. Создать дайджест исходного сообщения очень просто и быстро.

Недостатки алгоритма MD5

1. MD5 генерирует одну и ту же хэш-функцию для разных входных данных (*хэш-коллизия*).
2. MD5 обеспечивает более низкий уровень безопасности по сравнению с SHA1, SHA256 и другими современными криптографическими алгоритмами.
3. MD5 считался небезопасным алгоритмом. Поэтому теперь мы используем SHA256 вместо MD5.
4. MD5 не является ни симметричным, ни асимметричным алгоритмом.

Сравнение с онлайн-калькуляторами подтвердило правильность вычислений. Также были проанализированы вычислительная сложность, структура цикла и устойчивость алгоритма к коллизиям. Несмотря на то, что MD5 в настоящее время считается криптографически устаревшим, его реализация полезна для понимания принципов построения хэш-функций, операций на уровне битов и формирования контрольных сумм.

Практическая часть включала в себя разработку программы на языке C/C++ и Java, которая обладает следующим функционалом:

- Выбор и загрузка произвольного файла с выбранным форматом.
- Заполнение строки произвольными данными для хэширования.

- Сохранение результатов вычисления в файл.
- Визуальное представление структуры и справочной информации.

У выбранной структуры и архитектуры проекта есть преимущества в том, что функционал программной системы расширяем (можно легко добавить новые алгоритмы, не изменяя базовую структуру), надёжен (при ошибке чтения/записи IO-уровень не нарушает работу алгоритма), а модульность предоставляет удобство в тестировании каждого компонента независимо. Вдобавок код остаётся чистым, есть некоторое разделение ответственности между слоями и логика не привязана к интерфейсу (есть возможность заменить GUI на какое-то консольное или другое графическое решение).

В результате работы были продемонстрированы скорость и простота алгоритма MD5, а также подтверждена его пригодность для задач, связанных с криптографической безопасностью и получением check sum (для проверки целостности файлов, генерация уникальных идентификаторов и контроль изменения данных).

Несмотря на критику, MD5 остаётся востребованным и по-прежнему сохраняет свою актуальность в образовательных целях и в прикладных задачах, где требуются быстроедействие, компактность и простота реализации. Для студентов и начинающих разработчиков этот алгоритм является отличным инструментом для изучения основ хэширования и понимания принципов работы криптографических функций и безопасности. Например, MD5 идеально подходит для проверки целостности больших файлов или создания уникальных идентификаторов. Если вы используете MD5 для проверки целостности, всегда комбинируйте его с другими методами, такими как цифровые подписи, чтобы повысить безопасность. Для криптографических задач переходите на SHA-256 или более современные алгоритмы.

Таким образом, проделанная работа позволила не только реализовать классический алгоритм MD5, но и провести сравнительный анализ его преимуществ и ограничений. Полученные результаты подтвердили важность выбора хэш-функции в зависимости от целей приложения: MD5 целесообразно использовать только в некритичных сценариях, где требуется скорость и уникальность, но не безопасность.

MD5 — это легендарный алгоритм хэширования, который, несмотря на свои недостатки, остаётся полезным в некриптографических задачах. От проверки файлов до создания уникальных идентификаторов, MD5 продолжает находить применение благодаря своей скорости и простоте.

Приложение. Листинг кода

Реализация на С

main.c

```
#define _CRT_SECURE_NO_WARNINGS

#include <windows.h>
#include "md5.h"
#include <stdio.h>

#define ID_TEXTBOX 1
#define ID_BUTTON_TEXT 2
#define ID_BUTTON_FILE 3
#define ID_RESULT 4
#define ID_BUTTON_SAVE 5
#define ID_BUTTON_NEXT 6

LRESULT CALLBACK WndProc(HWND, UINT, WPARAM, LPARAM);
void ComputeMD5Text(HWND);
void ComputeMD5File(HWND);
void SaveResultToFile(HWND);
void ShowInfoScreen(HWND);
void ShowMainScreen(HWND);

BOOL mainScreenShown = FALSE;

char lastHash[256] = "";

// --- Точка входа программы с GUI (Регистрация окна) --- //
int WINAPI WinMain(HINSTANCE hInst, HINSTANCE hPrev,
LPSTR args, int nCmdShow) {
    WNDCLASSW wc = { 0 };
    wc.hbrBackground = (HBRUSH)COLOR_WINDOW;
    wc.hCursor = LoadCursor(NULL, IDC_ARROW);
    wc.hInstance = hInst;
    wc.lpszClassName = L"MD5Application";
    wc.lpfnWndProc = WndProc;
    RegisterClassW(&wc);

    CreateWindowW(L"MD5Application", L"MD5 Hash
Analyzer (C)",
WS_OVERLAPPEDWINDOW |
WS_VISIBLE,
100, 100, 600, 400, NULL, NULL, NULL,
NULL);

    MSG msg = { 0 };
    while (GetMessage(&msg, NULL, 0, 0)) {
        TranslateMessage(&msg);
        DispatchMessage(&msg);
    }

    return 0;
}

// --- Обработчик событий (кнопки, окна, ввод) --- //
LRESULT CALLBACK WndProc(HWND hwnd, UINT msg,
WPARAM wp, LPARAM lp) {
    static HWND hTextBox, hButtonText, hButtonFile,
hResult, hButtonSave, hInfo;

    switch (msg) {
        case WM_CREATE:
            ShowInfoScreen(hwnd);
            break;

        case WM_COMMAND:
            switch (LOWORD(wp)) {
                case ID_BUTTON_NEXT:
                    if (!mainScreenShown) {
```

```
TRUE;

            ShowMainScreen(hwnd);
        }

        break;

        case ID_BUTTON_TEXT:
            ComputeMD5Text(hwnd);
            break;

        case ID_BUTTON_FILE:
            ComputeMD5File(hwnd);
            break;

        case ID_BUTTON_SAVE:
            SaveResultToFile(hwnd);
            break;
    }

    break;

    case WM_DESTROY:
        PostQuitMessage(0);
        break;
    }

    return DefWindowProcW(hwnd, msg, wp, lp);
}

// === Экран со справочной информацией === //
void ShowInfoScreen(HWND hwnd) {
    CreateWindowW(L"static",
L"Студент гр. ПИБд-41 Белянин Никита
Николаевич\nВариант: №6\n\n"
L"Алгоритм MD5 — это
криптографическая хэш-функция,\n"
L"создающая 128-битное (32-символьное)
представление данных.\n\n"
L"Принцип работы:\n"
L"1. Дополнение входного сообщения
(padding)\n"
L"2. Добавление длины сообщения\n"
L"3. Инициализация буфера (A, B, C,
D)\n"
L"4. Обработка по блокам по 512 бит\n"
L"5. Формирование 128-битного
хэша\n\n"
L"Используется для проверки
целостности данных и цифровых подписей.",
WS_VISIBLE | WS_CHILD | ES_LEFT, 20,
20, 540, 240,
hwnd, NULL, NULL, NULL);

    CreateWindowW(L"button", L"Перейти к
программе",
WS_VISIBLE | WS_CHILD |
WS_BORDER,
200, 280, 180, 35, hwnd,
(HMENU)ID_BUTTON_NEXT, NULL, NULL);
}

// === Основное окно для хэширования === //
void ShowMainScreen(HWND hwnd) {
    // Очистить предыдущее содержимое
    HWND hChild = GetWindow(hwnd, GW_CHILD);

    while (hChild) {
        DestroyWindow(hChild);
        hChild = GetWindow(hwnd, GW_CHILD);
    }
}
```

```

// === Поле ввода текста === //
CreateWindowW(L"static", L"Введите текст: ",
              WS_VISIBLE | WS_CHILD, 20, 20, 120,
20, hwnd, NULL, NULL, NULL);

CreateWindowW(L"edit", L"",
              WS_VISIBLE | WS_CHILD |
WS_BORDER, 20, 50, 440, 25, hwnd, (HMENU)ID_TEXTBOX,
NULL, NULL);

// --- Кнопки --- //
CreateWindowW(L"button", L"Хэшировать текст",
              WS_VISIBLE | WS_CHILD, 20, 90, 180,
30, hwnd, (HMENU)ID_BUTTON_TEXT, NULL, NULL);

CreateWindowW(L"button", L"Хэшировать файл",
              WS_VISIBLE | WS_CHILD, 220, 90, 180,
30, hwnd, (HMENU)ID_BUTTON_FILE, NULL, NULL);

CreateWindowW(L"button", L"Сохранить результат",
WS_VISIBLE | WS_CHILD,
              400, 200, 160, 30, hwnd,
(HMENU)ID_BUTTON_SAVE, NULL, NULL);

// --- Результат --- //
CreateWindowW(L"edit", L"", WS_VISIBLE |
WS_CHILD | WS_BORDER | ES_READONLY,
              20, 140, 440, 25, hwnd,
(HMENU)ID_RESULT, NULL, NULL);
}

// --- Обработка действий пользователя --- //
void ComputeMD5Text(HWND hwnd) {
    wchar_t input[256];
    char buffer[512];
    GetDlgItemTextW(hwnd, ID_TEXTBOX, input, 256);

    char text[256];
    wcstombs(text, input, sizeof(text));

    char* hash = MD5String(text);
    wprintfA(buffer, "%s", hash);
    SetDlgItemTextA(hwnd, ID_RESULT, buffer);
    strncpy_s(lastHash, sizeof(lastHash), hash,
_TRUNCATE);
    free(hash);
}

void ComputeMD5File(HWND hwnd) {
    OPENFILENAMEW ofn = { 0 };
    wchar_t file[260] = L"";
    file[0] = '\0';

    ofn.lStructSize = sizeof(ofn);
    ofn.hwndOwner = hwnd;
    ofn.lpstrFilter = L"All Files\0*.*\0";
    ofn.lpstrFile = file;
    ofn.nMaxFile = sizeof(file);
    ofn.Flags = OFN_FILEMUSTEXIST |
OFN_PATHMUSTEXIST;

    if (GetOpenFileNameW(&ofn)) {
        char path[260];
        wcstombs(path, file, sizeof(path));
        char* hash = MD5File(path);
        SetDlgItemTextA(hwnd, ID_RESULT,
hash);
        strncpy_s(lastHash, sizeof(lastHash), hash,
_TRUNCATE);
        free(hash);
    }
}

void SaveResultToFile(HWND hwnd) {
    if (strlen(lastHash) == 0) {

```

```

        MessageBoxW(hwnd, L"Нет данных для
сохранения!", L"Ошибка", MB_ICONERROR);
        return;
    }

    OPENFILENAMEW ofn = { 0 };
    wchar_t file[260] = L"hash_result.txt";
    ofn.lStructSize = sizeof(ofn);
    ofn.hwndOwner = hwnd;
    ofn.lpstrFilter = L"Text Files\0*.txt\0All Files\0*.*\0";
    ofn.lpstrFile = file;
    ofn.nMaxFile = sizeof(file);
    ofn.Flags = OFN_OVERWRITEPROMPT;

    if (GetSaveFileNameW(&ofn)) {
        FILE* f;
        char path[260];
        wcstombs(path, file, sizeof(path));
        f = fopen(path, "w");

        if (f) {
            fprintf(f, "MD5 Hash
Result:\n%s\n", lastHash);
            fclose(f);
            MessageBoxW(hwnd,
L"Результат успешно сохранён!", L"Готово",
MB_ICONINFORMATION);
        }
    }
}

```

md5.h

```
#pragma once
```

```
#ifndef MD5_H
#define MD5_H
```

```
#include <stdint.h>
#include <stddef.h>
```

```

// === Контекст MD5 (состояние вычислений) === //
/*
    state - 4 регистра (a,b,c,d);
    count - количество обработанных бит
    buffer - текущий блок данных (512 бит) */
typedef struct {
    uint32_t state[4];
    uint32_t count[2];
    unsigned char buffer[64];
} MD5_CTX;

```

```
// ===== Основные функции ===== //
```

```
// Инициализация контекста (регистры)
void MD5Init(MD5_CTX* context);
```

```
// Добавление данных
void MD5Update(MD5_CTX* context, const unsigned char*
input, size_t inputLen);
```

```
// Завершение вычисления
void MD5Final(unsigned char digest[16], MD5_CTX* context);
```

```
// Один раунд преобразования блока
void MD5Transform(uint32_t state[4], const unsigned char
block[64]);
```

```
// Хэширование строки и хэширование файла
void *MD5String(const char* string);
void *MD5File(const char* filename);
```

```
#endif
```

md5.c

```
#define _CRT_SECURE_NO_WARNINGS
```

```
#include "md5.h"
```



```

#include <string.h>
#include <stdlib.h>
#include <stdio.h>

#define F(x,y,z) ((x & y) | (~x & z))
#define G(x,y,z) ((x & z) | (y & ~z))
#define H(x,y,z) (x ^ y ^ z)
#define I(x,y,z) (y ^ (x | ~z))
#define ROTATE_LEFT(x,n) ((x << n) | (x >> (32 - n)))

#define FF(a,b,c,d,x,s,ac) { (a) += F((b),(c),(d)) + (x) +
(uint32_t)(ac); (a) = ROTATE_LEFT((a), (s)); (a) += (b); }

#define GG(a,b,c,d,x,s,ac) { (a) += G((b),(c),(d)) + (x) +
(uint32_t)(ac); (a) = ROTATE_LEFT((a),(s)); (a) += (b); }

#define HH(a,b,c,d,x,s,ac) { (a) += H((b),(c),(d)) + (x) +
(uint32_t)(ac); (a) = ROTATE_LEFT((a),(s)); (a) += (b); }

#define II(a,b,c,d,x,s,ac) { (a) += I((b),(c),(d)) + (x) +
(uint32_t)(ac); (a) = ROTATE_LEFT((a),(s)); (a) += (b); }

static void Encode(unsigned char* output, const uint32_t* input,
size_t len);
static void Decode(uint32_t* output, const unsigned char* input,
size_t len);
static void MD5_memcpy(unsigned char* output, const unsigned
char* input, size_t len);
static void MD5_memset(unsigned char* output, int value, size_t
len);

static const unsigned char PADDING[64] = { 0x80, 0 };

// === Инициализация контекста === //
void MD5Init(MD5_CTX* context) {
    context->count[0] = context->count[1] = 0;
    context->state[0] = 0x67452301;
    context->state[1] = 0xefcdab89;
    context->state[2] = 0x98badcfe;
    context->state[3] = 0x10325476;
}

// === Обновление (добавление данных, делятся входные
данные на блоки по 512 бит) === //
void MD5Update(MD5_CTX* context, const unsigned char*
input, size_t inputLen) {
    size_t i, index, partLen;

    index = (unsigned int)((context->count[0] >> 3) &
0x3f);
    if ((context->count[0] += ((uint32_t)inputLen << 3)) <
((uint32_t)inputLen << 3))
        context->count[1]++;
    context->count[1] += ((uint32_t)inputLen >> 29);

    partLen = 64 - index;

    if (inputLen >= partLen) {
        MD5_memcpy(&context->buffer[index],
input, partLen);
        MD5Transform(context->state, context-
>buffer);

        for (i = partLen; i + 63 < inputLen; i += 64)
            MD5Transform(context->state,
&input[i]);

        index = 0;
    }
    else i = 0;

    MD5_memcpy(&context->buffer[index], &input[i],
inputLen - i);
}

// === Завершение (финализация) === //

```

```

void MD5Final(unsigned char digest[16], MD5_CTX* context) {
    unsigned char bits[8];
    unsigned int index, padLen;

    Encode(bits, context->count, 8);

    index = (unsigned int)((context->count[0] >> 3) &
0x3f);
    padLen = (index < 56) ? (56 - index) : (120 - index);
    MD5Update(context, PADDING, padLen);

    MD5Update(context, bits, 8);

    Encode(digest, context->state, 16);

    MD5_memset((unsigned char*)context, 0,
sizeof(*context));
}

// === Преобразование блока (ядро, где 64 шага, 4 раунда * 16
операций для функций F,G,H,I) === //
/* Шаг: перемешивание A,B,C,D; добавление констант, сдвиги
и суммирование в 32-битной арифметике */
void MD5Transform(uint32_t state[4], const unsigned char
block[64]) {
    uint32_t a = state[0], b = state[1], c = state[2], d =
state[3], x[16];
    Decode(x, block, 64);

    // === Раунд 1 === //
    FF(a, b, c, d, x[0], 7, 0xd76aa478);
    FF(d, a, b, c, x[1], 12, 0xe8c7b756);
    FF(c, d, a, b, x[2], 17, 0x242070db);
    FF(b, c, d, a, x[3], 22, 0xc1bdceee);
    FF(a, b, c, d, x[4], 7, 0xf57c0faf);
    FF(d, a, b, c, x[5], 12, 0x4787c62a);
    FF(c, d, a, b, x[6], 17, 0xa8304613);
    FF(b, c, d, a, x[7], 22, 0xfd469501);
    FF(a, b, c, d, x[8], 7, 0x698098d8);
    FF(d, a, b, c, x[9], 12, 0x8b44f7af);
    FF(c, d, a, b, x[10], 17, 0xffff5bb1);
    FF(b, c, d, a, x[11], 22, 0x895cd7be);
    FF(a, b, c, d, x[12], 7, 0x6b901122);
    FF(d, a, b, c, x[13], 12, 0xfd987193);
    FF(c, d, a, b, x[14], 17, 0xa679438e);
    FF(b, c, d, a, x[15], 22, 0x49b40821);

    // === Раунд 2 === //
    GG(a, b, c, d, x[1], 5, 0xf61e2562);
    GG(d, a, b, c, x[6], 9, 0x040b340);
    GG(c, d, a, b, x[11], 14, 0x265e5a51);
    GG(b, c, d, a, x[0], 20, 0xe9b6c7aa);
    GG(a, b, c, d, x[5], 5, 0xd62f105d);
    GG(d, a, b, c, x[10], 9, 0x02441453);
    GG(c, d, a, b, x[15], 14, 0xd8a1e681);
    GG(b, c, d, a, x[4], 20, 0xe7d3fbc8);
    GG(a, b, c, d, x[9], 5, 0x21e1cde6);
    GG(d, a, b, c, x[14], 9, 0xc33707d6);
    GG(c, d, a, b, x[3], 14, 0xf4d50d87);
    GG(b, c, d, a, x[8], 20, 0x455a14ed);
    GG(a, b, c, d, x[13], 5, 0xa9e3e905);
    GG(d, a, b, c, x[2], 9, 0xfcefa3f8);
    GG(c, d, a, b, x[7], 14, 0x676f02d9);
    GG(b, c, d, a, x[12], 20, 0x8d2a4c8a);

    // === Раунд 3 === //
    HH(a, b, c, d, x[5], 4, 0xffff3942);
    HH(d, a, b, c, x[8], 11, 0x8771f681);
    HH(c, d, a, b, x[11], 16, 0x6d9d6122);
    HH(b, c, d, a, x[14], 23, 0xfde5380c);
    HH(a, b, c, d, x[1], 4, 0xa4beea44);
    HH(d, a, b, c, x[4], 11, 0x4bdecfa9);
    HH(c, d, a, b, x[7], 16, 0xf6bb4b60);
    HH(b, c, d, a, x[10], 23, 0xbee5fbcb);
    HH(a, b, c, d, x[13], 4, 0x289b7ec6);
    HH(d, a, b, c, x[0], 11, 0xeaa127fa);
    HH(c, d, a, b, x[3], 16, 0xd4ef3085);
    HH(b, c, d, a, x[6], 23, 0x04881d05);
}

```

```

HH(a, b, c, d, x[9], 4, 0xd9d4d039);
HH(d, a, b, c, x[12], 11, 0xe6db99e5);
HH(c, d, a, b, x[15], 16, 0x1fa27cf8);
HH(b, c, d, a, x[2], 23, 0xc4ac5665);

// == Раунд 4 == //
Π(a, b, c, d, x[0], 6, 0xf4292244);
Π(d, a, b, c, x[7], 10, 0x432aff97);
Π(c, d, a, b, x[14], 15, 0xab9423a7);
Π(b, c, d, a, x[5], 21, 0xfc93a039);
Π(a, b, c, d, x[12], 6, 0x655b59c3);
Π(d, a, b, c, x[3], 10, 0x8f0ccc92);
Π(c, d, a, b, x[10], 15, 0xffeff47d);
Π(b, c, d, a, x[1], 21, 0x85845dd1);
Π(a, b, c, d, x[8], 6, 0x6fa87e4f);
Π(d, a, b, c, x[15], 10, 0xfe2ce6e0);
Π(c, d, a, b, x[6], 15, 0xa3014314);
Π(b, c, d, a, x[13], 21, 0x4e0811a1);
Π(a, b, c, d, x[4], 6, 0xf7537e82);
Π(d, a, b, c, x[11], 10, 0xbd3af235);
Π(c, d, a, b, x[2], 15, 0x2ad7d2bb);
Π(b, c, d, a, x[9], 21, 0xeb86d391);

state[0] += a;
state[1] += b;
state[2] += c;
state[3] += d;

MD5_memset((unsigned char*)x, 0, sizeof(x));
}

// == Утилиты для конвертации uint32 и байты == //
static void Encode(unsigned char* output, const uint32_t* input,
size_t len) {
    for (size_t i = 0, j = 0; j < len; i++, j += 4) {
        output[j] = (unsigned char)(input[i] & 0xff);
        output[j + 1] = (unsigned char)((input[i] >>
8) & 0xff);
        output[j + 2] = (unsigned char)((input[i] >>
16) & 0xff);
        output[j + 3] = (unsigned char)((input[i] >>
24) & 0xff);
    }
}

static void Decode(uint32_t* output, const unsigned char* input,
size_t len) {
    for (size_t i = 0, j = 0; j < len; i++, j += 4)
        output[i] = ((uint32_t)input[j]) |
        (((uint32_t)input[j + 1] << 8) |
        (((uint32_t)input[j + 2] << 16) |
        (((uint32_t)input[j + 3] << 24);
}

static void MD5_memcpy(unsigned char* output, const unsigned
char* input, size_t len) {
    memcpy(output, input, len);
}

static void MD5_memset(unsigned char* output, int value, size_t
len) {
    memset(output, value, len);
}

char* MD5String(const char* string) {
    MD5_CTX context;
    unsigned char digest[16];
    char* output = (char*)malloc(33);
    if (!output) return NULL;

    MD5Init(&context);
    MD5Update(&context, (const unsigned char*)string,
strlen(string));
    MD5Final(digest, &context);

```

```

for (int i = 0; i < 16; ++i)
    sprintf(&output[i * 2], "%02x", digest[i]);

output[32] = '\0';
return output;
}

char* MD5File(const char* filename) {
    FILE* file = fopen(filename, "rb");
    if (!file) return _strdup("Ошибка открытия файла");

    MD5_CTX context;
    unsigned char buffer[1024], digest[16];
    size_t bytes;
    char* output = (char*)malloc(33);
    if (!output) return NULL;

    MD5Init(&context);
    while ((bytes = fread(buffer, 1, sizeof(buffer), file)) !=
0)
        MD5Update(&context, buffer, bytes);
    MD5Final(digest, &context);
    fclose(file);

    for (int i = 0; i < 16; ++i)
        sprintf(&output[i * 2], "%02x", digest[i]);

    output[32] = '\0';
    return output;
}

```

Реализация на C++

main.cpp

```

#define _CRT_SECURE_NO_WARNINGS
#include <windows.h>
#include <commdlg.h>
#include <string>
#include <fstream>
#include <chrono>
#include <ctime>
#include "md5.h"

LRESULT CALLBACK WndProc(HWND, UINT, WPARAM,
LPARAM);
void ShowReference();
void SaveHashToFile(const std::string& hash, const std::wstring&
source);

// элементы интерфейса
HWND hEditInput, hEditOutput, hBtnText, hBtnFile, hBtnInfo;

int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE,
LPSTR, int nCmdShow) {
    const wchar_t CLASS_NAME[] = L"MD5HashApp";

    WNDCLASS wc = {};
    wc.lpfnWndProc = WndProc;
    wc.hInstance = hInstance;
    wc.lpszClassName = CLASS_NAME;
    wc.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);

    RegisterClass(&wc);

    HWND hwnd = CreateWindowEx(
        0,
        CLASS_NAME,
        L"MD5 Hash Analyzer (WinAPI)",
        WS_OVERLAPPEDWINDOW ^ WS_THICKFRAME,
        CW_USEDEFAULT, CW_USEDEFAULT, 500, 300,
        nullptr, nullptr, hInstance, nullptr
    );

```

```

if (!hwnd) return 0;
ShowWindow(hwnd, nCmdShow);

MSG msg = {};
while (GetMessage(&msg, nullptr, 0, 0)) {
    TranslateMessage(&msg);
    DispatchMessage(&msg);
}
return (int)msg.wParam;
}

// === создание окна и элементов ===
LRESULT CALLBACK WndProc(HWND hwnd, UINT msg,
WPARAM wParam, LPARAM lParam) {
    switch (msg) {
        case WM_CREATE: {
            CreateWindow(L"STATIC", L"Введите текст:",
WS_VISIBLE | WS_CHILD,
                20, 20, 100, 20, hwnd, nullptr, nullptr, nullptr);

            hEditInput = CreateWindow(L"EDIT", L"", WS_VISIBLE |
WS_CHILD | WS_BORDER,
                130, 20, 320, 25, hwnd, nullptr, nullptr, nullptr);

            hBtnText = CreateWindow(L"BUTTON", L"Хэшировать
текст", WS_VISIBLE | WS_CHILD,
                20, 60, 150, 30, hwnd, (HMENU)1, nullptr, nullptr);

            hBtnFile = CreateWindow(L"BUTTON", L"Хэшировать
файл", WS_VISIBLE | WS_CHILD,
                180, 60, 150, 30, hwnd, (HMENU)2, nullptr, nullptr);

            hBtnInfo = CreateWindow(L"BUTTON", L"Справка",
WS_VISIBLE | WS_CHILD,
                340, 60, 110, 30, hwnd, (HMENU)3, nullptr, nullptr);

            CreateWindow(L"STATIC", L"Результат MD5:",
WS_VISIBLE | WS_CHILD,
                20, 110, 120, 20, hwnd, nullptr, nullptr, nullptr);

            hEditOutput = CreateWindow(L"EDIT", L"", WS_VISIBLE |
WS_CHILD | WS_BORDER | ES_AUTOHSCROLL,
                150, 110, 300, 25, hwnd, nullptr, nullptr, nullptr);
            break;
        }

        case WM_COMMAND: {
            switch (LOWORD(wParam)) {
                case 1: { // хэшировать текст
                    wchar_t buffer[1024];
                    GetWindowText(hEditInput, buffer, 1024);
                    std::wstring text(buffer);
                    if (text.empty()) {
                        MessageBox(hwnd, L"Введите текст для
хэширования!", L"Ошибка", MB_ICONWARNING);
                        break;
                    }

                    MD5 md5;
                    md5.update(std::string(text.begin(), text.end()));
                    std::string result = md5.finalize();
                    std::wstring wresult(result.begin(), result.end());
                    SetWindowText(hEditOutput, wresult.c_str());
                    SaveHashToFile(result, L"Введённый текст");
                    break;
                }

                case 2: { // хэшировать файл
                    OPENFILENAME ofn = {};
                    wchar_t fileName[MAX_PATH] = L"";
                    ofn.lStructSize = sizeof(ofn);
                    ofn.hwndOwner = hwnd;
                    ofn.lpstrFile = fileName;
                    ofn.nMaxFile = MAX_PATH;
                    ofn.lpstrFilter = L"Все файлы\0*.*\0";
                    ofn.Flags = OFN_FILEMUSTEXIST;

                    if (GetOpenFileName(&ofn)) {

```

```

                        try {
                            std::string path(fileName, fileName +
wcslen(fileName));
                            std::string result = MD5::fromFile(path);
                            std::wstring wresult(result.begin(), result.end());
                            SetWindowText(hEditOutput, wresult.c_str());
                            SaveHashToFile(result, fileName);
                        }
                        catch (const std::exception& e) {
                            MessageBox(hwnd, e.what(), "Ошибка",
MB_ICONERROR);
                        }
                    }
                    break;
                }
            }

            case 3:
                ShowReference();
                break;
            }
            break;
        }

        case WM_DESTROY:
            PostQuitMessage(0);
            break;
    }
    return DefWindowProc(hwnd, msg, wParam, lParam);
}

// === окно со справочной информацией ===
void ShowReference() {
    MessageBox(
        nullptr,
        L"Студент гр. ПИбд-41 Белянин Никита
Николаевич\nВариант: №6\n\n"
        L"\nАлгоритм MD5 — это криптографическая хэш-
функция,\n"
        L"создающая 128-битное (32-символьное) представление
данных.\n\n"
        L"MD5 (Message Digest 5)\n\n"
        L"• Разработан Рональдом Ривестом в 1991 году\n"
        L"• Длина хэша: 128 бит (32 шестнадцатеричных
символа)\n"
        L"• Применяется для проверки целостности данных\n"
        L"• Не рекомендуется для криптографической защиты\n"
        L"\nПрограмма позволяет вычислить MD5 хэш строки
или файла.",
        L"Справочная информация",
        MB_ICONINFORMATION
    );
}

// === сохранение хэша в файл ===
void SaveHashToFile(const std::string& hash, const std::wstring&
source) {
    std::ofstream file("md5_result.txt", std::ios::app);
    if (!file.is_open()) return;

    // текущее время
    auto now = std::chrono::system_clock::now();
    std::time_t now_c = std::chrono::system_clock::to_time_t(now);

    file << "=== Result MD5 ===\n";
    file << "Date and time: " << std::ctime(&now_c);

    file << "hash: " << hash << "\n";
    file << "=====\n\n";

    file.close();

    MessageBox(nullptr, L"Результат успешно сохранён в
файлик",
        L"Сохранение", MB_OK | MB_ICONINFORMATION);
    file << "MD5: " << hash << "\n";
}

```

md5.h

```
#pragma once
```

```
#ifndef MD5_H
#define MD5_H
```

```
#include <string>
#include <array>
#include <stdint>
#include <vector>
```

```
class MD5 {
public:
    MD5();
    void update(const std::string& data);
    std::string finalize();
    static std::string fromFile(const std::string& path);
```

```
private:
    void transform(const uint8_t block[64]);
    void encode(uint8_t* output, const uint32_t* input,
size_t len);
    void decode(uint32_t* output, const uint8_t* input,
size_t len);
    void reset();
    static std::string bytesToHex(const uint8_t* bytes,
size_t length);
```

```
    bool finalized;
    uint64_t count;
    std::array<uint32_t, 4> state;
    std::array<uint8_t, 64> buffer;
    std::array<uint8_t, 16> digest;
```

```
};
```

```
#endif
```

md5.cpp

```
#include "md5.h"
#include <fstream>
#include <cstring>
#include <cmath>
#include <sstream>
#include <iomanip>
#include <stdexcept>
```

```
static inline uint32_t F(uint32_t x, uint32_t y, uint32_t z) { return
(x & y) | (~x & z); }
static inline uint32_t G(uint32_t x, uint32_t y, uint32_t z) { return
(x & z) | (y & ~z); }
static inline uint32_t H(uint32_t x, uint32_t y, uint32_t z) { return
x ^ y ^ z; }
static inline uint32_t I(uint32_t x, uint32_t y, uint32_t z) { return y
^ (x | ~z); }
```

```
static inline uint32_t ROTATE_LEFT(uint32_t x, uint32_t n) {
return (x << n) | (x >> (32 - n)); }
```

```
#define FF(a,b,c,d,x,s,ac) { a += F(b, c, d) + x + ac; a =
ROTATE_LEFT(a, s); a += b; }
```

```
#define GG(a,b,c,d,x,s,ac) { a += G(b,c,d) + x + ac; a =
ROTATE_LEFT(a, s); a += b; }
```

```
#define HH(a,b,c,d,x,s,ac) { a += H(b,c,d) + x + ac; a =
ROTATE_LEFT(a, s); a += b; }
```

```
#define II(a,b,c,d,x,s,ac) { a += I(b,c,d) + x + ac; a =
ROTATE_LEFT(a, s); a += b; }
```

```
MD5::MD5() { reset(); }
```

```
void MD5::reset() {
    finalized = false;
    count = 0;
    state = { 0x67452301, 0xefcdab89, 0x98badcfe,
0x10325476 };
    std::fill(buffer.begin(), buffer.end(), 0);
}
```

```
void MD5::update(const std::string& data) {
    if (finalized) throw std::runtime_error("MD5 already
finalized");
    const uint8_t* input = reinterpret_cast<const
uint8_t*>(data.data());
    size_t len = data.size();
```

```
    size_t index = (count / 8) % 64;
    count += static_cast<uint64_t>(len) * 8;
```

```
    size_t partLen = 64 - index;
    size_t i = 0;
```

```
    if (len >= partLen) {
        std::memcpy(&buffer[index], input,
partLen);
        transform(buffer.data());
        for (i = partLen; i + 63 < len; i += 64)
            transform(&input[i]);
        index = 0;
    }
    std::memcpy(&buffer[index], &input[i], len - i);
}
```

```
void MD5::transform(const uint8_t block[64]) {
    uint32_t a = state[0], b = state[1], c = state[2], d =
state[3], x[16];
    decode(x, block, 64);
```

```
    // === Раунд 1 === //
    FF(a, b, c, d, x[0], 7, 0xd76aa478); FF(d, a, b, c, x[1],
12, 0xe8c7b756);
    FF(c, d, a, b, x[2], 17, 0x242070db); FF(b, c, d, a, x[3],
22, 0xc1bdcee);
    FF(a, b, c, d, x[4], 7, 0xf57c0faf); FF(d, a, b, c, x[5], 12,
0x4787c62a);
    FF(c, d, a, b, x[6], 17, 0xa8304613); FF(b, c, d, a, x[7],
22, 0xfd469501);
    FF(a, b, c, d, x[8], 7, 0x698098d8); FF(d, a, b, c, x[9],
12, 0x8b44f7af);
    FF(c, d, a, b, x[10], 17, 0xffff5bb1); FF(b, c, d, a, x[11],
22, 0x895cd7be);
    FF(a, b, c, d, x[12], 7, 0x6b901122); FF(d, a, b, c,
x[13], 12, 0xfd987193);
    FF(c, d, a, b, x[14], 17, 0xa679438e); FF(b, c, d, a,
x[15], 22, 0x49b40821);
```

```
    // === Раунд 2 === //
    GG(a, b, c, d, x[1], 5, 0xf61e2562); GG(d, a, b, c, x[6],
9, 0xc040b340);
    GG(c, d, a, b, x[11], 14, 0x265e5a51); GG(b, c, d, a,
x[0], 20, 0xe9b6c7aa);
    GG(a, b, c, d, x[5], 5, 0xd62f105d); GG(d, a, b, c,
x[10], 9, 0x02441453);
    GG(c, d, a, b, x[15], 14, 0xd8a1e681); GG(b, c, d, a,
x[4], 20, 0xe7d3fbc8);
    GG(a, b, c, d, x[9], 5, 0x21e1cde6); GG(d, a, b, c,
x[14], 9, 0xc33707d6);
    GG(c, d, a, b, x[3], 14, 0xf4d50d87); GG(b, c, d, a,
x[8], 20, 0x455a14ed);
    GG(a, b, c, d, x[13], 5, 0xa9e3e905); GG(d, a, b, c,
x[2], 9, 0xfcefa3f8);
    GG(c, d, a, b, x[7], 14, 0x676f02d9); GG(b, c, d, a,
x[12], 20, 0x8d2a4c8a);
```

```
    // === Раунд 3 === //
    HH(a, b, c, d, x[5], 4, 0xffffa3942); HH(d, a, b, c, x[8],
11, 0x8771f681);
    HH(c, d, a, b, x[11], 16, 0x6d9d6122); HH(b, c, d, a,
x[14], 23, 0xfde5380c);
    HH(a, b, c, d, x[1], 4, 0xa4beea44); HH(d, a, b, c, x[4],
11, 0x4bdecfa9);
    HH(c, d, a, b, x[7], 16, 0xf6bb4b60); HH(b, c, d, a,
x[10], 23, 0x9bebfb70);
    HH(a, b, c, d, x[13], 4, 0x289b7ec6); HH(d, a, b, c,
x[0], 11, 0xea127fa);
```

```

        HH(c, d, a, b, x[3], 16, 0xd4ef3085); HH(b, c, d, a, x[6],
23, 0x04881d05);
        HH(a, b, c, d, x[9], 4, 0xd9d4d039); HH(d, a, b, c,
x[12], 11, 0xe6db99e5);
        HH(c, d, a, b, x[15], 16, 0x1fa27cf8); HH(b, c, d, a,
x[2], 23, 0xc4ac5665);

        // ==== Раунд 4 ==== //
        II(a, b, c, d, x[0], 6, 0xf4292244); II(d, a, b, c, x[7], 10,
0x432aff97);
        II(c, d, a, b, x[14], 15, 0xab9423a7); II(b, c, d, a, x[5],
21, 0xfc93a039);
        II(a, b, c, d, x[12], 6, 0x655b59c3); II(d, a, b, c, x[3],
10, 0x8f0ccc92);
        II(c, d, a, b, x[10], 15, 0xffeff47d); II(b, c, d, a, x[1], 21,
0x85845dd1);
        II(a, b, c, d, x[8], 6, 0x6fa87e4f); II(d, a, b, c, x[15], 10,
0xfe2ce6e0);
        II(c, d, a, b, x[6], 15, 0xa3014314); II(b, c, d, a, x[13],
21, 0x4e0811a1);
        II(a, b, c, d, x[4], 6, 0xf7537e82); II(d, a, b, c, x[11], 10,
0xbd3af235);
        II(c, d, a, b, x[2], 15, 0x2ad7d2bb); II(b, c, d, a, x[9],
21, 0xeb86d391);

        state[0] += a; state[1] += b; state[2] += c; state[3] += d;
}

void MD5::encode(uint8_t* output, const uint32_t* input, size_t
len) {
    for (size_t i = 0, j = 0; j < len; ++i, j += 4) {
        output[j] = (uint8_t)(input[i] & 0xff);
        output[j + 1] = (uint8_t)((input[i] >> 8) &
0xff);
        output[j + 2] = (uint8_t)((input[i] >> 16) &
0xff);
        output[j + 3] = (uint8_t)((input[i] >> 24) &
0xff);
    }
}

void MD5::decode(uint32_t* output, const uint8_t* input, size_t
len) {
    for (size_t i = 0, j = 0; j < len; ++i, j += 4)
        output[i] = ((uint32_t)input[j]) |

        (((uint32_t)input[j + 1] << 8) |

        (((uint32_t)input[j + 2] << 16) |

        (((uint32_t)input[j + 3] << 24);
}

std::string MD5::bytesToHex(const uint8_t* bytes, size_t length) {
    std::ostringstream oss;
    oss << std::hex << std::setfill('0');

    for (size_t i = 0; i < length; ++i)
        oss << std::setw(2) <<
static_cast<int>(bytes[i] & 0xff);

    return oss.str();
}

std::string MD5::finalize() {
    if (finalized) return bytesToHex(digest.data(), 16);

    uint8_t bits[8];

    encode(bits, (uint32_t*)&count, 8);

    // вычисление кол-ва байт в буфере
    size_t index = (count / 8) % 64;
    size_t padLen = (index < 56) ? (56 - index) : (120 -
index);

    static const uint8_t PADDING[64] = { 0x80 };

    update(std::string((char*)PADDING, padLen));
    update(std::string((char*)bits, 8));
}

```

```

// Формирование итогового ответа (state -> digest)
encode(digest.data(), state.data(), 16);

finalized = true;
return bytesToHex(digest.data(), 16);
}

std::string MD5::fromFile(const std::string& path) {
    std::ifstream file(path, std::ios::binary);

    if (!file.is_open())
        throw std::runtime_error("Не удалось
открыть файл.");

    MD5 md5;
    std::vector<char> buffer(1024);

    while (file.good()) {
        file.read(buffer.data(), buffer.size());
        md5.update(std::string(buffer.data(),
file.gcount()));
    }

    file.close();
    return md5.finalize();
}

```

Реализация на Java

Main.java

```

package org.example;

import javax.swing.*;

public class Main {
    public static void main(String[] args) {

        SwingUtilities.invokeLater() -> {
            try {

                UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
            } catch (Exception ignored) {
            }

            MainWindow window = new MainWindow();
            window.setVisible(true);
            window.showInfo();

        });
    }
}

```

Md5.java

```

package org.example;

import java.io.File;
import java.io.FileInputStream;
import java.io.FileWriter;
import java.io.IOException;

public class MD5 {
    private static final int[] S = {
        7, 12, 17, 22, 7, 12, 17, 22, 7, 12, 17, 22, 7, 12, 17, 22,
        5, 9, 14, 20, 5, 9, 14, 20, 5, 9, 14, 20, 5, 9, 14, 20,
        4, 11, 16, 23, 4, 11, 16, 23, 4, 11, 16, 23, 4, 11, 16, 23,
        6, 10, 15, 21, 6, 10, 15, 21, 6, 10, 15, 21, 6, 10, 15, 21
    };
}

```

```

private static final int[] K = new int[64];

static {
    for (int i = 0; i < 64; i++)
        K[i] = (int) (long) ((1L << 32) * Math.abs(Math.sin(i +
1)));
}

public static String hashFromFile(File file) throws IOException
{
    FileInputStream fis = new FileInputStream(file);
    byte[] data = fis.readAllBytes();
    fis.close();
    return hash(data);
}

public static String hash(byte[] input) {
    int messageLen = input.length;
    int numBlocks = ((messageLen + 8) >>> 6) + 1;
    int totalLen = numBlocks << 6;

    byte[] padding = new byte[totalLen - messageLen];
    padding[0] = (byte) 0x80;

    long messageLenBits = (long) messageLen << 3;
    for (int i = 0; i < 8; i++) {
        padding[padding.length - 8 + i] = (byte) (messageLenBits
>>> (8 * i));
    }

    int A = 0x67452301;
    int B = 0xefcdab89;
    int C = 0x98badcfe;
    int D = 0x10325476;

    byte[] block = new byte[64];
    for (int i = 0; i < numBlocks; i++) {
        int index = i << 6;
        for (int j = 0; j < 64; j++) {
            block[j] = (index + j < messageLen) ? input[index + j] :
padding[index + j - messageLen];
        }

        int[] M = new int[16];
        for (int j = 0; j < 16; j++) {
            M[j] = ((block[j * 4] & 0xff) |
((block[j * 4 + 1] & 0xff) << 8) |
((block[j * 4 + 2] & 0xff) << 16) |
((block[j * 4 + 3] & 0xff) << 24);
        }

        int a = A, b = B, c = C, d = D;

        for (int j = 0; j < 64; j++) {
            int F, g;
            if (j < 16) {
                F = (b & c) | (~b & d);
                g = j;
            } else if (j < 32) {
                F = (d & b) | (~d & c);
                g = (5 * j + 1) % 16;
            } else if (j < 48) {
                F = b ^ c ^ d;
                g = (3 * j + 5) % 16;
            } else {
                F = c ^ (b | ~d);
                g = (7 * j) % 16;
            }

            int temp = d;
            d = c;
            c = b;
            b = b + Integer.rotateLeft(a + F + K[j] + M[g], S[j]);
            a = temp;
        }

        A += a;
        B += b;

```

```

        C += c;
        D += d;
    }

    byte[] digest = new byte[16];
    int[] parts = {A, B, C, D};

    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4; j++) {
            digest[i * 4 + j] = (byte) (parts[i] >>> (8 * j));
        }
    }

    StringBuilder sb = new StringBuilder();

    for (byte b : digest) {
        sb.append(String.format("%02x", b & 0xff));
    }

    return sb.toString();
}

public static void saveToFile(String hash) {
    try (FileWriter fw = new FileWriter("md5_result.txt", true)) {
        fw.write("MD5: " + hash + "\n");
    } catch (IOException e) {
        e.printStackTrace();
    }
}
}

```

MainWindow.java

```
package org.example;
```

```
import javax.swing.*;
import java.awt.event.ActionEvent;
import java.io.File;
```

```
public class MainWindow extends JFrame {
```

```
    private final JTextField inputField;
    private final JTextField resultField;
```

```
    private final JButton hashTextButton;
    private final JButton hashFileButton;
    private final JButton saveButton;
    private final JButton infoButton;
```

```
    private String lastHash = "";
```

```
    public MainWindow() {
        setTitle("MD5 Hash Analyzer (Java)");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(600, 320);
        setLocationRelativeTo(null);
        setLayout(null);
    }

```

```
        JLabel labelInput = new JLabel("Введите текст: ");
        labelInput.setBounds(20, 20, 150, 25);
        add(labelInput);
    }

```

```
        inputField = new JTextField();
        inputField.setBounds(20, 50, 540, 25);
        add(inputField);
    }

```

```
        hashTextButton = new JButton("Хэшировать текст");
        hashTextButton.setBounds(20, 90, 170, 30);
        hashTextButton.addActionListener(this::onHashText);
        add(hashTextButton);
    }

```

```
        hashFileButton = new JButton("Хэшировать файл");
        hashFileButton.setBounds(200, 90, 170, 30);
        hashFileButton.addActionListener(this::onHashFile);
        add(hashFileButton);
    }
}

```

```

saveButton = new JButton("Сохранить хэш в файл");
saveButton.setBounds(380, 90, 180, 30);
saveButton.addActionListener(this::onSaveHash);
add(saveButton);

infoButton = new JButton("Справка");
infoButton.setBounds(20, 220, 120, 30);
infoButton.addActionListener(e -> showInfo());
add(infoButton);

JLabel labelResult = new JLabel("Результат MD5: ");
labelResult.setBounds(20, 140, 150, 25);
add(labelResult);

resultField = new JTextField();
resultField.setEditable(false);
resultField.setBounds(20, 170, 540, 25);
add(resultField);
}

private void onHashText(ActionEvent e) {
    String text = inputField.getText();
    if (text.isEmpty()) {
        JOptionPane.showMessageDialog(this, "Введите текст
для хэширования", "Ошибка",
JOptionPane.WARNING_MESSAGE);
        return;
    }

    lastHash = MD5.hash(text.getBytes());
    resultField.setText(lastHash);
}

private void onHashFile(ActionEvent e) {
    JFileChooser chooser = new JFileChooser();
    int result = chooser.showOpenDialog(this);

    if (result == JFileChooser.APPROVE_OPTION) {
        File file = chooser.getSelectedFile();

        try {
            lastHash = MD5.hashFromFile(file);
            resultField.setText(lastHash);
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(this, "Ошибка при
чтении файла: \n" + ex.getMessage(), "Ошибка",
JOptionPane.ERROR_MESSAGE);
        }
    }
}

private void onSaveHash(ActionEvent e) {
    if (lastHash.isEmpty()) {
        JOptionPane.showMessageDialog(this, "Нет результата
для сохранения!", "Ошибка",
JOptionPane.WARNING_MESSAGE);
        return;
    }

    MD5.saveToFile(lastHash);
    JOptionPane.showMessageDialog(this, "Хэш успешно
сохранён в файл", "Успех",
JOptionPane.INFORMATION_MESSAGE);
}

public void showInfo() {
    JOptionPane.showMessageDialog(this,
        "Студент: Белянин Никита Николаевич
        Группа: ПИБд-41
        Вариант №6

        Алгоритм MD5 — криптографическая хэш-
        функция,
        создающая 128-битное (32-символьное)
        представление данных.

        Этапы алгоритма:

```

- Дополнение данных до длины 512 бит - 64.
- Разбиение на блоки по 512 бит.
- Инициализация 4-х регистров (A, B, C, D).
- 64 раунда с функциями F, G, H, I и сдвигами.
- Склеивание результата в 128-битный хэш.

Программа позволяет вычислить MD5 хэш
строки или файла
и по желанию сохранить результат

```

        """,
        "Справочная информация",
        JOptionPane.INFORMATION_MESSAGE);
    }
}

```

Алгоритм MD5 — криптографическая хэш-
функция,
создающая 128-битное (32-символьное)
представление данных.

Этапы алгоритма: